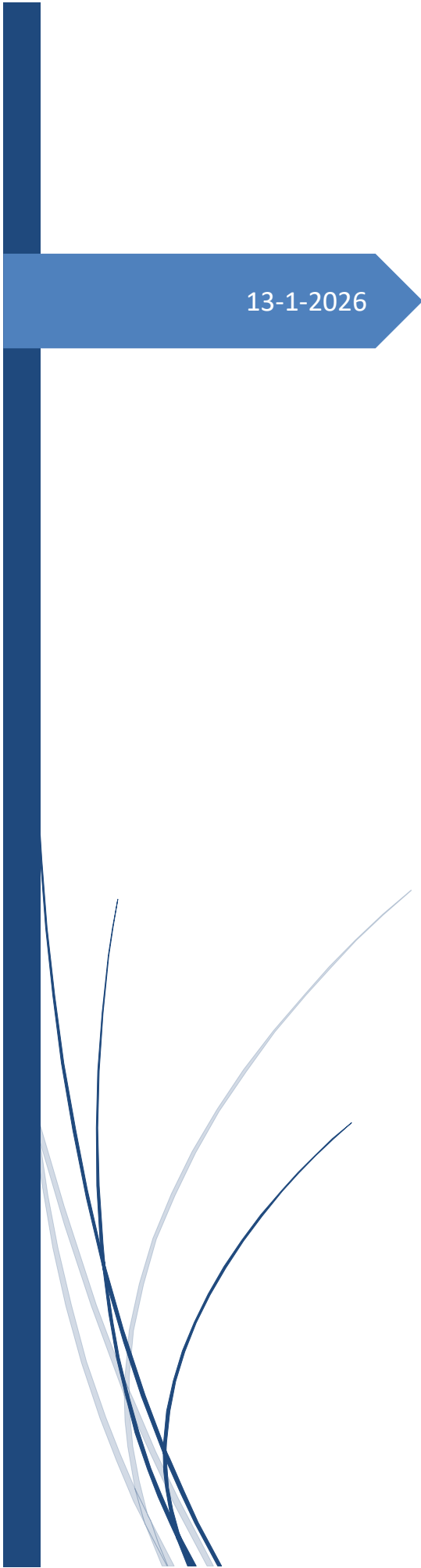




13-1-2026

Tutorial de PHP 8

Rafael Alberto Moreno Parra

Contenido

Tabla de ilustraciones	6
Página Web del autor	8
Canal de Youtube	8
Licencia de este libro	9
Licencia del software	9
Marcas registradas.....	9
Introducción	10
Iniciando.....	13
Software de servidor.....	13
Navegadores	13
Editores	13
Código fuente.....	13
Descargando e instalando el servidor de aplicaciones	14
Probando que el servidor de aplicaciones funciona bien.....	22
Configurando Visual Studio Code	24
¿Dónde se almacenan esos archivos?	26
Comentarios en el código	27
El tradicional “Hola Mundo”	27
Uso de variables.....	29
Operaciones matemáticas	30
Asignación	30
Operaciones con cadenas	31
Impresión de caracteres propios del idioma español.....	32
Constantes matemáticas	33
Funciones trigonométricas	33
Funciones Matemáticas	34
Ejemplos de cálculos	34
Corrigiendo errores en PHP	35
Sí condicional	36
Operadores lógicos: Conjunción (Y, &&) y Disyunción (O,)	37
Uso del switch	38

Ciclos. Uso del “for”	39
Ciclo while	41
Ciclo do - while	44
Uso del break	47
Uso del continue	48
Ciclos anidados y salir de estos	49
Funciones o subrutinas	50
Ejemplos de uso de la librería	58
Método de Bisección	65
Ámbito (scope) de las variables y funciones	66
Funciones recursivas	69
Uso de eval como evaluador de expresiones algebraicas	71
Números aleatorios	72
Distribución Normal	72
Distribución Triangular	73
Distribución Uniforme	73
Manejo de errores con números	74
is_nan	74
is_finite	74
Try...catch	75
Arreglos unidimensionales o vectores	76
Uso de llaves	77
Adicionar elementos con array_push	77
Operaciones con arreglos	78
Ordenar arreglos	79
Fusionar, extraer y retirar elementos de un arreglo	80
Funciones genéricas para arreglos	81
Retornar arreglos desde funciones	83
Implementación de algoritmos de ordenación	84
Algoritmo de burbuja	84
Algoritmo de ordenación por selección	85
Algoritmo de ordenación por inserción	86

Algoritmo de ordenación QuickSort	86
Algoritmo de ordenación MergeSort.....	87
Manejo de Fechas	88
Manejo de cadenas o strings	89
Vocales tildadas, Ñs y otros caracteres del idioma español.....	90
Ejemplos de programas usando cadenas	93
Invertir Cadena.....	93
Quitar los espacios de una cadena	93
Quitar las vocales de una cadena	94
Quitar caracteres no permitidos.....	94
Un sencillo cifrador / descifrador	95
Un sencillo cifrador / descifrador con clave de cifrado	96
Sumar largos números enteros almacenados en cadenas	97
Arreglos bidimensionales.....	98
Ejemplos de programas usando arreglos bidimensionales	102
Poner una torre al azar en un tablero de ajedrez y mostrar su ataque	102
Poner un Rey al azar en un tablero de ajedrez y mostrar su ataque	103
Poner un alfil al azar en un tablero de ajedrez y mostrar su ataque.....	104
Poner una Reina al azar en un tablero de ajedrez y mostrar su ataque	105
Ruta del menor costo.....	106
Resolver Sudokus	109
Poner los barcos en el juego batalla naval	112
Funciones que reciben distinto número de parámetros	114
Funciones que reciben a su vez funciones por parámetros	115
Operaciones de bit.....	117
Convertir un entero a su representación binaria	117
Operaciones OR, AND y XOR directas.....	117
Multiplicar usando operaciones de bit	118
Dividir usando operaciones de bit	119
Intercambiar valores usando operaciones de bit	119
Programación orientada a objetos	120
Definiendo clases y atributos.....	120

Definiendo métodos para acceder a atributos privados	120
Dos variables, misma instancia	121
Dos variables y dos instancias distintas	121
Constantes	122
Constructores y destructores.....	122
Un constructor con parámetros con valores por defecto	124
Instancia un objeto dentro de otra clase y accede a los atributos.....	125
Recibe un objeto como parámetro de una función.....	126
Clonando objetos	126
Detectar instancias	127
Métodos estáticos.....	127
Serializar un objeto	128
Comparando objetos	130
Herencia	131
Tipos de acceso en la herencia	132
Reacción a mismo método en clase padre e hija	133
Reacción a constructores en clase padre e hija.....	135
Clases abstractas.....	137
Métodos abstractos	138
Evitar la herencia	139
Lista de objetos	140
Cambiar un ítem en una lista de objetos.....	142
Polimorfismo.....	143
Uso de gettype para detectar el tipo de dato de una variable.....	144
Árbol binario	145
Recorrido en pre-orden	146
Recorrido en in-orden.....	148
Recorrido en post-orden.....	149
Ordenación usando un árbol binario.....	150
Lista de objetos. Pilas.....	152
Lista de objetos. Colas.	153
Archivos Planos	155

Uso de JSON	155
Añadiendo saltos de línea	156
Añadiendo datos a un archivo	156
Leyendo un archivo plano	157
Modelo Vista Controlador (MVC)	158
Formularios	164
Método GET	164
Método POST	168
Uso de bases de datos	170
Versiones de elaboración de CRUDs	171
Referencias.....	172

Tabla de ilustraciones

Ilustración 1: Lenguajes de programación para aplicaciones Web en el lado del servidor	10
Ilustración 2: Tendencia en el uso de PHP	11
Ilustración 3: Tendencia en el uso de PHP	11
Ilustración 4: Tendencia en el uso de PHP	12
Ilustración 5: Sitio web para la descarga de XAMPP	14
Ilustración 6: Pantalla de advertencia sobre el User Account Control	14
Ilustración 7: Solo Apache, MySQL, PHP y phpMyAdmin.	15
Ilustración 8: El directorio donde se instalará XAMPP.	16
Ilustración 9: Apache HTTP Server requiere permisos del Firewall, se presiona "Permitir acceso"	17
Ilustración 10: Finaliza la instalación	18
Ilustración 11: Panel de control de XAMPP, los servicios están inactivos.....	19
Ilustración 12: Iniciando el servidor web Apache.	20
Ilustración 13: MySQL requiere permisos en el firewall, clic en "Permitir acceso".	20
Ilustración 14: Los dos servicios están activos.....	21
Ilustración 15: ¿Apache funciona? abre un navegador y se escribe <code>http://localhost</code>	22
Ilustración 16: En <code>phpinfo</code> aparece el estado de PHP	22
Ilustración 17: En <code>phpMyAdmin</code> aparece el estado de MySQL.	23
Ilustración 18: Instalar PHP Intelephense.....	24
Ilustración 19: Instalar <code>php cs fixer</code>	24
Ilustración 20: Instalar PHP Debug	24
Ilustración 21: Etiquetas de inicio y cierre de script PHP	25
Ilustración 22: Se recomienda dejar solo la etiqueta de apertura	25
Ilustración 23: En <code>C:\xampp\htdocs</code> o <code>D:\xampp\htdocs</code> se ubican los archivos PHP	26
Ilustración 24: Imprimir un texto	27
Ilustración 25: Inicia el navegador seleccionado para mostrar la página.....	28
Ilustración 26: En vez de <code>localhost</code> , se puede usar <code>127.0.0.1</code>	28
Ilustración 27: El navegador muestra los errores enviados por el servidor	35
Ilustración 28: Ámbito (scope) de una variable	66
Ilustración 29: Ámbito (scope) de una variable	67
Ilustración 30: Ámbito (scope) de una variable	68
Ilustración 31: Comportamiento con letras del idioma español	91
Ilustración 32: Recorriendo la cadena con caracteres multibyte	91
Ilustración 33: Recorriendo la cadena con caracteres multibyte	92
Ilustración 34: Sudoku resuelto	111
Ilustración 35: Árbol binario	146
Ilustración 36: Recorrido en pre-orden	147
Ilustración 37: Recorrido en in-orden.....	148
Ilustración 38: Recorrido en post-orden.....	149

Ilustración 39: Ordenación usando un árbol binario	151
Ilustración 40: Página mostrada	158
Ilustración 41: Código HTML de la página mostrada	158
Ilustración 42: Mejora del código en HTML.....	159
Ilustración 43: Mejorando con una tabla	160
Ilustración 44: Página mejorada	163
Ilustración 45: Formulario donde el usuario digita los datos de base y altura	164
Ilustración 46: Se digitan los datos y se presiona el botón "Enviar consulta"	164
Ilustración 47: Esta es la respuesta del programa	165
Ilustración 48: Datos enviados por la URL	165
Ilustración 49: Dos archivos en el servidor de aplicaciones	166
Ilustración 50: El formulario luce igual para el usuario final	168
Ilustración 51: La respuesta es igual para el usuario final	168
Ilustración 52: Base de datos "educar"	170

Página Web del autor

Investigación sobre Evolución y Vida Artificial: <http://darwin.50webs.com>

Correo: ramsoftware@gmail.com

Sitio GitHub: <https://github.com/ramsoftware/LibroPHP>

Canal de Youtube

Canal en Youtube: <http://www.youtube.com/user/RafaelMorenoP> (dedicado a C#)

Licencia de este libro



Licencia del software

Todo el software desarrollado aquí tiene licencia LGPL “Lesser General Public License”



Marcas registradas

En este libro se hace uso de las siguientes tecnologías registradas:

Microsoft ® Windows ® Enlace: <http://windows.microsoft.com/en-US/windows/home>

Introducción

En el desarrollo de aplicaciones Web, en el lado del servidor, PHP [1] [2] es el lenguaje de programación con una gran mayoría. Sólo es ver el siguiente gráfico a 08 de enero de 2026 de cómo está repartido los lenguajes de programación en el servidor [3]:

El enlace es: https://w3techs.com/technologies/overview/programming_language/all

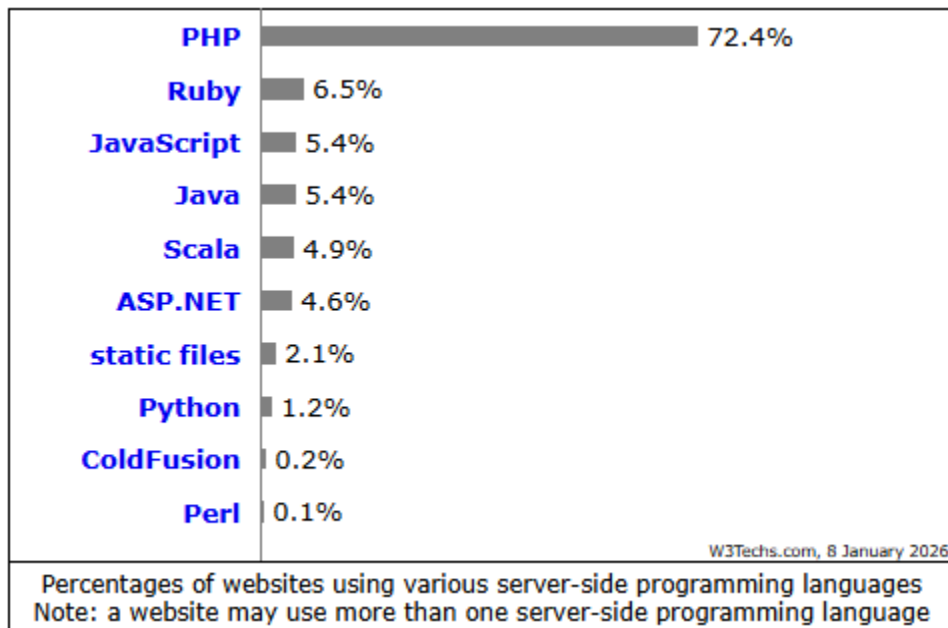


Ilustración 1: Lenguajes de programación para aplicaciones Web en el lado del servidor

Y su tendencia desde 2010:

https://w3techs.com/technologies/history_overview/programming_language/ms/y

	2010 1 Jan	2011 1 Jan	2012 1 Jan	2013 1 Jan	2014 1 Jan	2015 1 Jan	2016 1 Jan	2017 1 Jan	2018 1 Jan	2019 1 Jan	2019 12 Aug
PHP	72.5%	74.8%	76.6%	77.7%	80.3%	80.6%	80.0%	80.0%	80.2%	78.9%	79.1%
ASP.NET	24.4%	23.2%	21.4%	19.9%	17.8%	16.7%	15.6%	14.8%	13.5%	11.8%	10.9%
Java	4.0%	3.8%	3.9%	4.0%	2.6%	2.8%	3.1%	3.3%	3.4%	4.0%	3.9%
Ruby	0.5%	0.5%	0.6%	0.5%	0.6%	0.9%	1.1%	1.3%	1.6%	2.4%	2.6%
static files							1.5%	1.5%	1.6%	2.1%	2.0%
Scala						0.2%	0.2%	0.3%	0.5%	1.2%	1.4%
Python	0.3%	1.0%	1.3%	1.5%	1.7%	1.6%	1.7%	1.6%	1.3%	1.1%	1.2%
JavaScript			<0.1%	<0.1%	0.1%	0.1%	0.2%	0.3%	0.4%	0.7%	0.7%
ColdFusion		1.3%	1.2%	1.1%	0.8%	0.7%	0.7%	0.6%	0.6%	0.5%	0.5%
Perl		1.1%	1.0%	0.8%	0.6%	0.5%	0.5%	0.4%	0.3%	0.3%	0.3%
Erlang						0.1%	0.1%	0.1%	0.1%	0.1%	0.1%
Miva Script							0.1%	<0.1%	<0.1%	<0.1%	<0.1%

Ilustración 2: Tendencia en el uso de PHP

Historical yearly trends in the usage statistics of server-side programming languages for websites

This report shows the historical trends in the usage of the top server-side programming languages since January 2013.

	2013 1 Jan	2014 1 Jan	2015 1 Jan	2016 1 Jan	2017 1 Jan	2018 1 Jan	2019 1 Jan	2020 1 Jan	2021 1 Jan	2022 1 Jan	2023 1 Jan	2024 1 Jan	2024 15 Jul
PHP	77.7%	80.3%	80.6%	80.0%	80.0%	80.2%	78.9%	78.9%	79.1%	78.1%	77.7%	76.7%	76.1%
ASP.NET	19.9%	17.8%	16.7%	15.6%	14.8%	13.5%	11.8%	10.6%	9.3%	8.0%	7.4%	6.6%	5.9%
Ruby	0.5%	0.6%	0.9%	1.1%	1.3%	1.6%	2.4%	3.0%	4.3%	6.0%	5.3%	5.6%	5.9%
Java	4.0%	2.6%	2.8%	3.1%	3.3%	3.4%	4.0%	3.7%	3.2%	3.7%	4.6%	4.7%	4.9%
JavaScript	<0.1%	0.1%	0.1%	0.2%	0.3%	0.4%	0.7%	0.8%	1.2%	1.8%	2.3%	3.1%	3.5%
Scala			0.2%	0.2%	0.3%	0.5%	1.2%	1.6%	1.8%	2.3%	2.8%	3.0%	3.2%
static files				1.5%	1.5%	1.6%	2.1%	1.8%	1.6%	1.5%	1.9%	1.8%	1.8%
Python	1.5%	1.7%	1.6%	1.7%	1.6%	1.3%	1.1%	1.3%	1.4%	1.4%	1.3%	1.4%	1.3%
ColdFusion	1.1%	0.8%	0.7%	0.7%	0.6%	0.6%	0.5%	0.5%	0.3%	0.3%	0.3%	0.3%	0.2%
Perl	0.8%	0.6%	0.5%	0.5%	0.4%	0.3%	0.3%	0.2%	0.2%	0.2%	0.2%	0.1%	0.1%
Erlang			0.1%	0.1%	0.1%	0.1%	0.1%	0.1%	0.1%	0.1%	0.1%	0.1%	0.1%
Miva Script				0.1%	<0.1%	<0.1%	<0.1%	<0.1%	<0.1%	<0.1%	<0.1%	<0.1%	<0.1%

Ilustración 3: Tendencia en el uso de PHP

	2015 1 Jan	2016 1 Jan	2017 1 Jan	2018 1 Jan	2019 1 Jan	2020 1 Jan	2021 1 Jan	2022 1 Jan	2023 1 Jan	2024 1 Jan	2025 1 Jan	2026 1 Jan	2026 8 Jan
PHP	80.6%	80.0%	80.0%	80.2%	78.9%	78.9%	79.1%	78.1%	77.7%	76.7%	75.2%	72.4%	72.4%
Ruby	0.9%	1.1%	1.3%	1.6%	2.4%	3.0%	4.3%	6.0%	5.3%	5.6%	6.2%	6.5%	6.5%
JavaScript	0.1%	0.2%	0.3%	0.4%	0.7%	0.8%	1.2%	1.8%	2.3%	3.1%	4.0%	5.4%	5.4%
Java	2.8%	3.1%	3.3%	3.4%	4.0%	3.7%	3.2%	3.7%	4.6%	4.7%	5.0%	5.4%	5.4%
Scala	0.2%	0.2%	0.3%	0.5%	1.2%	1.6%	1.8%	2.3%	2.8%	3.0%	3.8%	4.8%	4.9%
ASP.NET	16.7%	15.6%	14.8%	13.5%	11.8%	10.6%	9.3%	8.0%	7.4%	6.6%	5.4%	4.6%	4.6%
static files		1.5%	1.5%	1.6%	2.1%	1.8%	1.6%	1.5%	1.9%	1.8%	1.7%	2.1%	2.1%
Python	1.6%	1.7%	1.6%	1.3%	1.1%	1.3%	1.4%	1.4%	1.3%	1.4%	1.3%	1.2%	1.2%
ColdFusion	0.7%	0.7%	0.6%	0.6%	0.5%	0.5%	0.3%	0.3%	0.3%	0.3%	0.2%	0.2%	0.2%
Perl	0.5%	0.5%	0.4%	0.3%	0.3%	0.2%	0.2%	0.2%	0.2%	0.1%	0.1%	0.1%	0.1%
Erlang	0.1%	0.1%	0.1%	0.1%	0.1%	0.1%	0.1%	0.1%	0.1%	0.1%	0.1%	<0.1%	<0.1%
Miva Script		0.1%	<0.1%	<0.1%	<0.1%	<0.1%	<0.1%	<0.1%	<0.1%	<0.1%	<0.1%	<0.1%	<0.1%

Ilustración 4: Tendencia en el uso de PHP

Es necesario programar en el lado del servidor para las siguientes tareas:

1. Programar la lógica de negocios de la aplicación Web.
2. Validaciones de datos en el servidor, por ejemplo, que el usuario final ingrese datos válidos.
3. Interactuar con la capa de persistencia: Conectarse a la base de datos y hacer operaciones de lectura y escritura en esta.

El presente libro es sobre PHP 8.4.

Iniciando

Empezar a programar en PHP es fácil y gratuito. Sólo se requiere de un servidor de aplicaciones, tener un navegador moderno y un editor de texto, todos pueden estar instalados en el mismo PC. Este manual mostrará capturas de pantalla en el ambiente de Microsoft Windows 11, pero lo expuesto es aplicable en otros sistemas operativos porque PHP es multiplataforma.

Software de servidor

Enlace para XAMPP: <https://www.apachefriends.org/es/index.html>

Navegadores

Enlace para Mozilla Firefox: <https://www.mozilla.org/es-ES/firefox/new/>

Enlace para Opera: <https://www.opera.com/es-419>

Enlace para Google Chrome: <https://www.google.com.mx/chrome/>

Enlace para Vivaldi: <https://vivaldi.com/es/>

Enlace para Brave: <https://brave.com/download/>

Editores

Enlace para Visual Studio Code: <https://code.visualstudio.com/>

Código fuente

El código fuente se puede descargar de: <https://github.com/ramsoftware/LibroPHP>

Descargando e instalando el servidor de aplicaciones

Para trabajar en PHP, es necesario que el computador se convierta en un servidor de aplicaciones que debe tener los siguientes componentes:

1. Un servidor Web [4]
2. Un gestor de base de datos [5]
3. Poder ejecutar PHP

En el caso del servidor Web se hace uso de Apache, el gestor de base de datos es MariaDB [6] o MySQL [7] (ambos son compatibles) y que en el servidor se ejecute PHP. Esto último es un componente que se añade a Apache. Los tres son software libre, no hay que pagar un licenciamiento por usarlos.

Los tres: Apache, MariaDB/MySQL, PHP se pueden instalar por aparte, directamente de los sitios oficiales, pero hay una manera mucho más rápida y fácil y es usando paquete de instaladores que instalan esos tres componentes y los configura automáticamente. Uno muy conocido es XAMPP.

Instalar XAMPP es sencillo, se mostrará paso a paso desde su descarga hasta su instalación y ejecución.



Ilustración 5: Sitio web para la descarga de XAMPP

Inicia la instalación

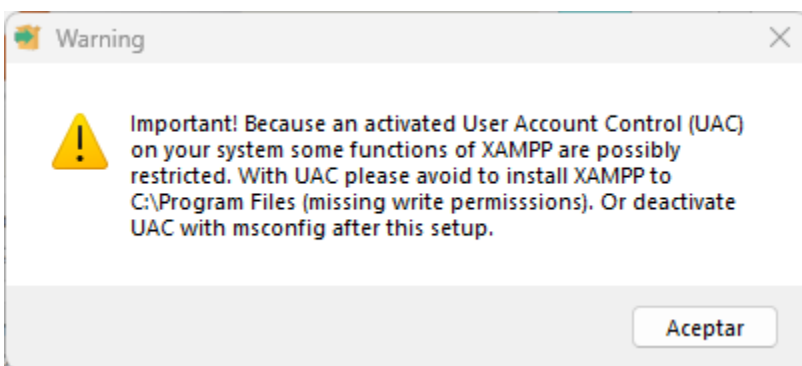


Ilustración 6: Pantalla de advertencia sobre el User Account Control

UAC significa Control de Cuentas de Usuario y es para que las aplicaciones ejecuten en un modo NO administrador del sistema. XAMPP requiere tener acceso de administrador, luego Windows 11 preguntará si va a ser instalarlo, se responde afirmativamente.

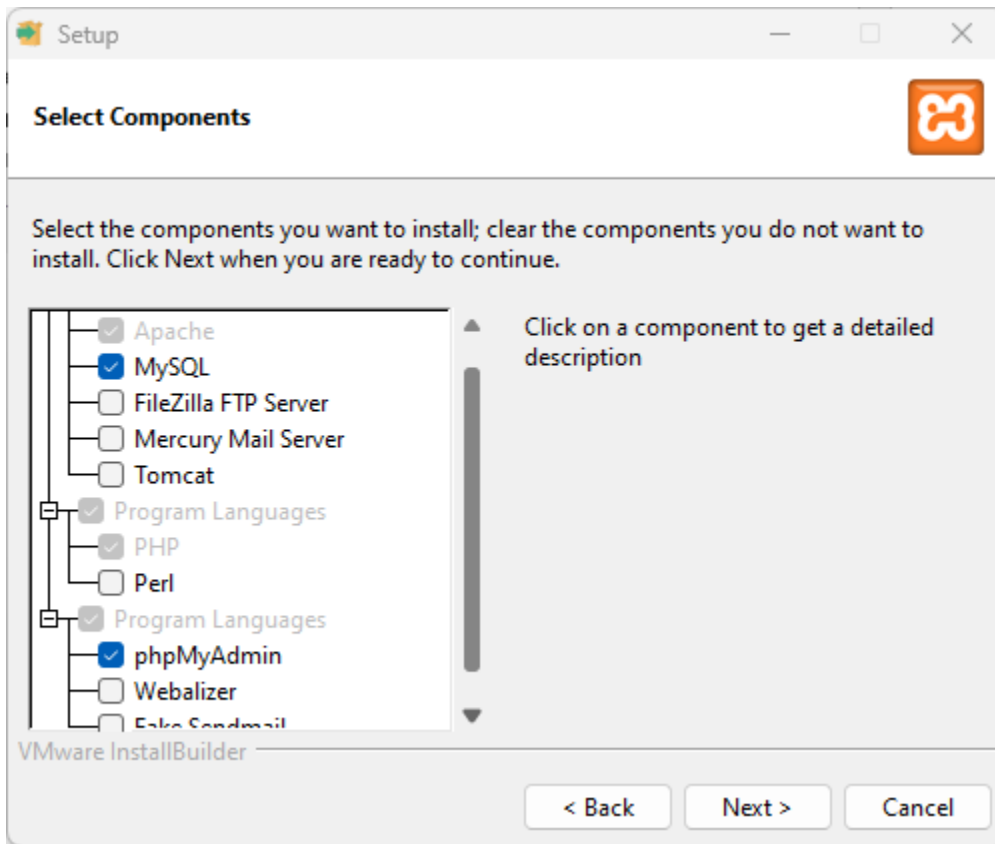


Ilustración 7: Solo Apache, MySQL, PHP y phpMyAdmin.

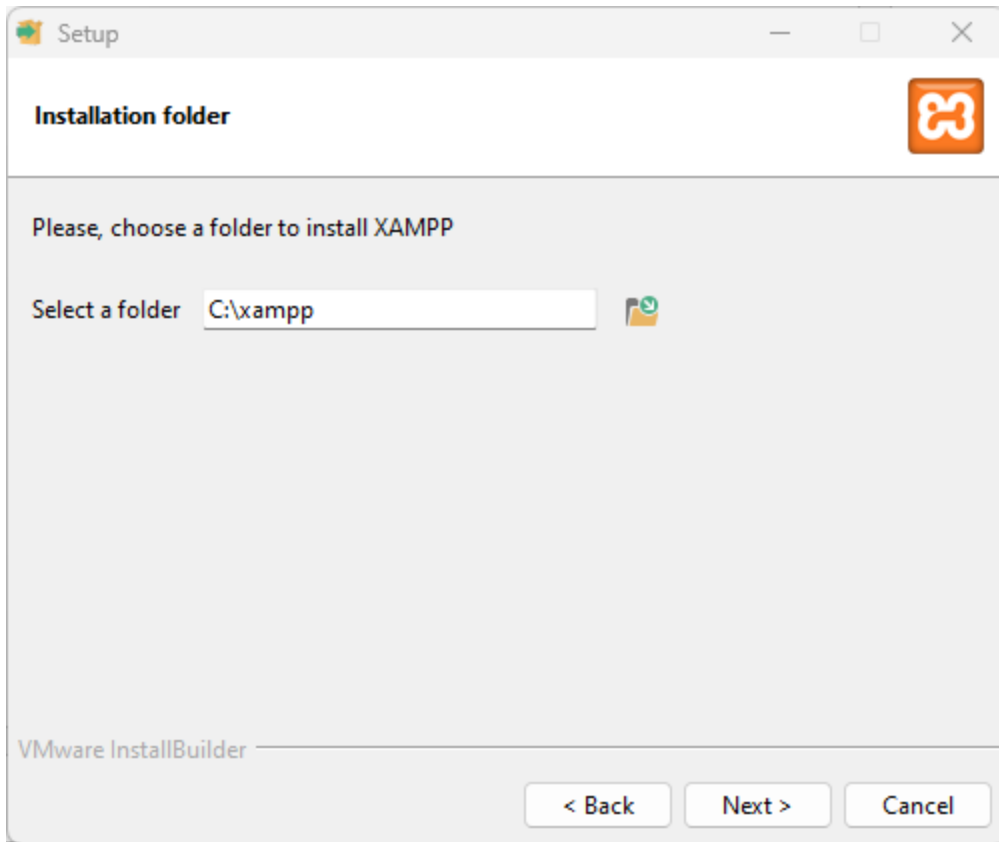


Ilustración 8: El directorio donde se instalará XAMPP.

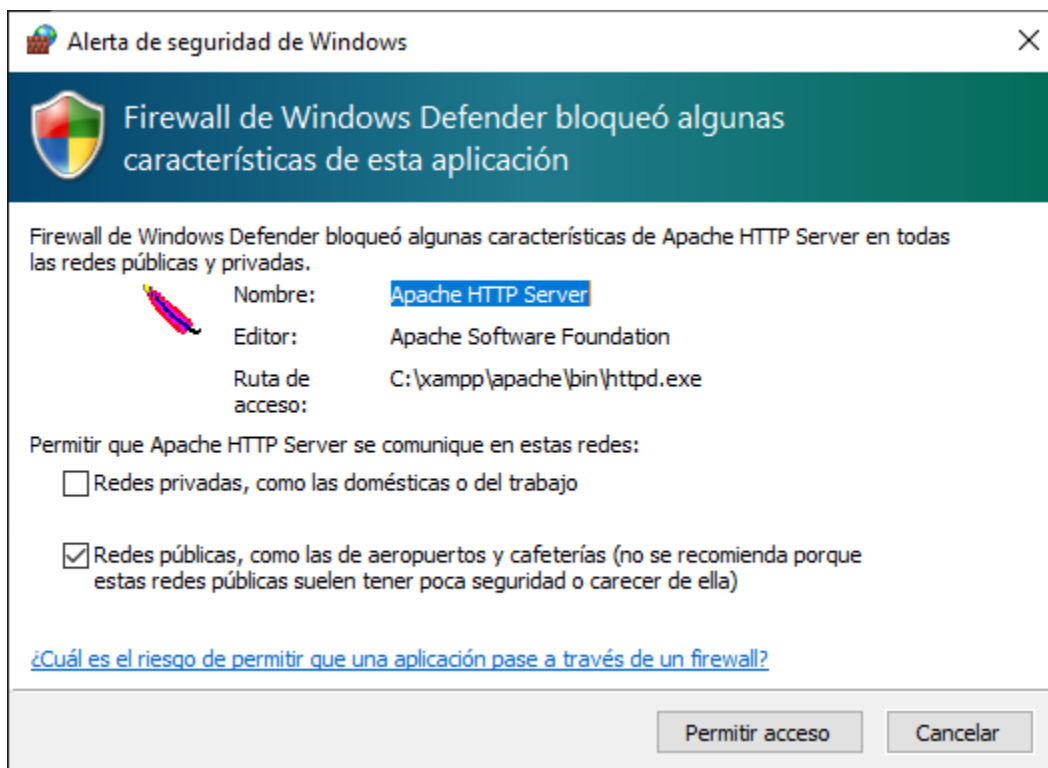


Ilustración 9: Apache HTTP Server requiere permisos del Firewall, se presiona "Permitir acceso"

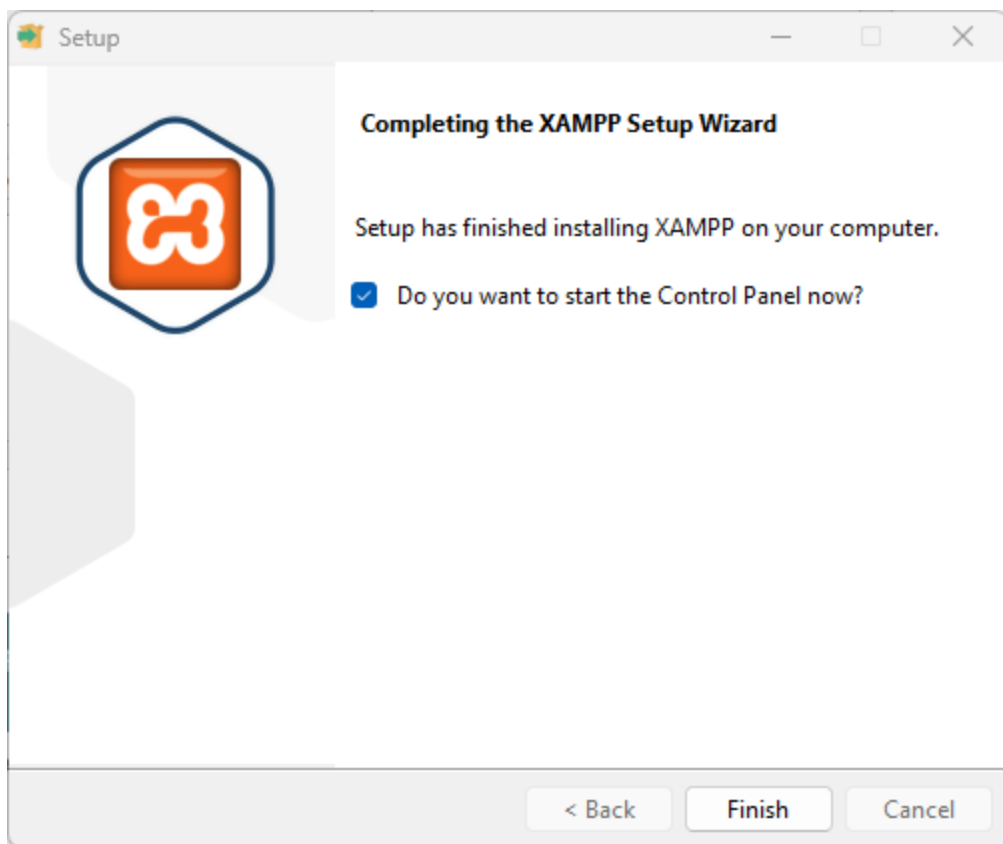


Ilustración 10: Finaliza la instalación

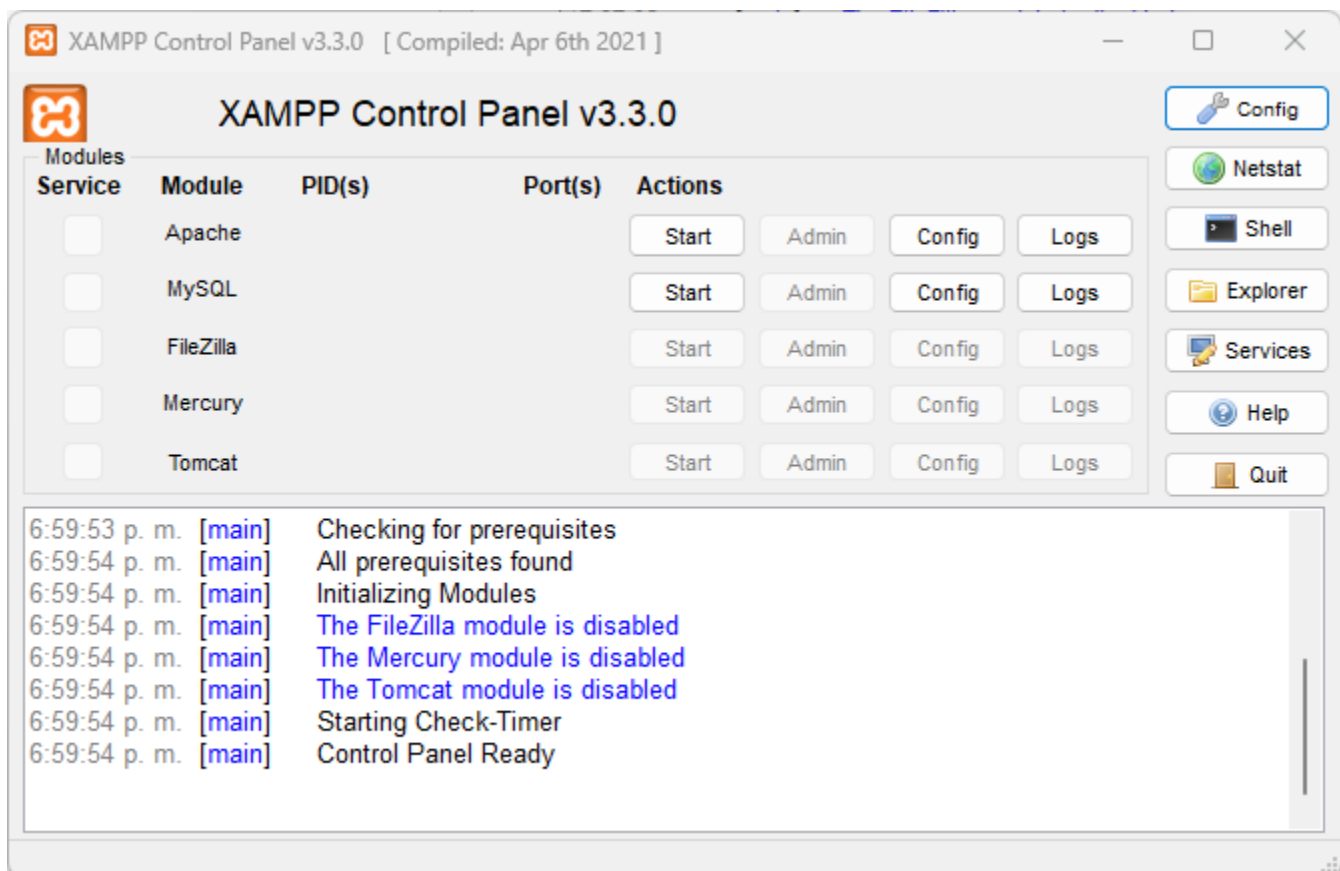


Ilustración 11: Panel de control de XAMPP, los servicios están inactivos.

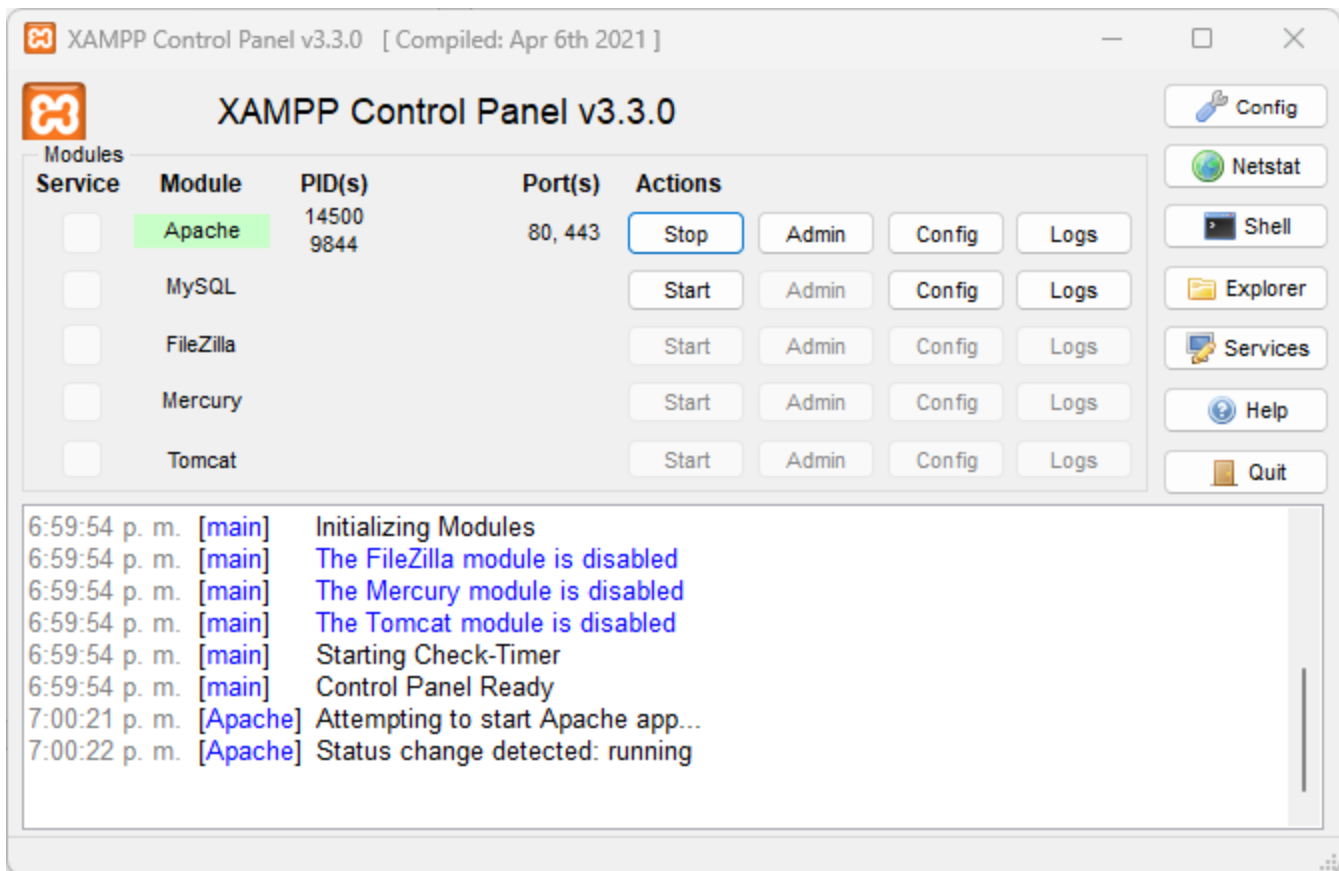


Ilustración 12: Iniciando el servidor web Apache.

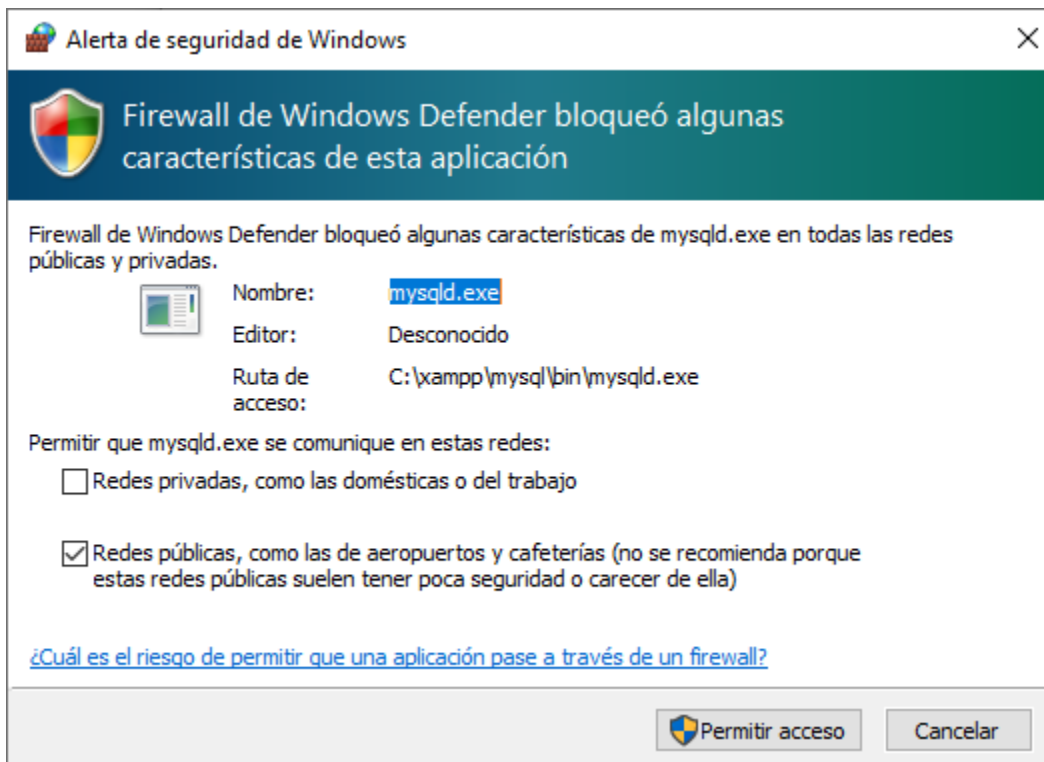


Ilustración 13: MySQL requiere permisos en el firewall, clic en "Permitir acceso".

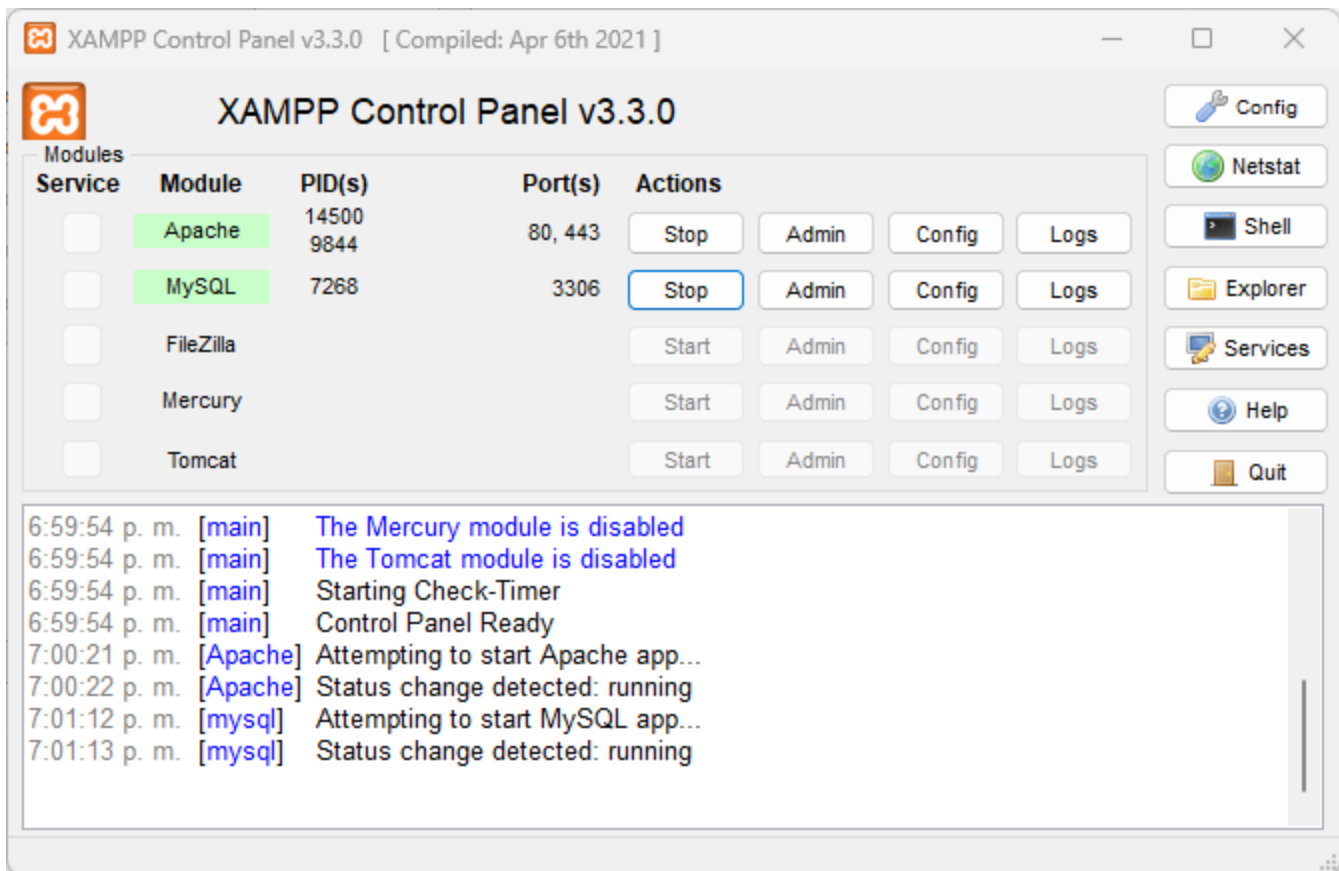


Ilustración 14: Los dos servicios están activos.

Probando que el servidor de aplicaciones funciona bien

Para saber si el servidor de aplicaciones funciona correctamente, se abre un navegador y en <http://localhost>, debería verse esta pantalla:

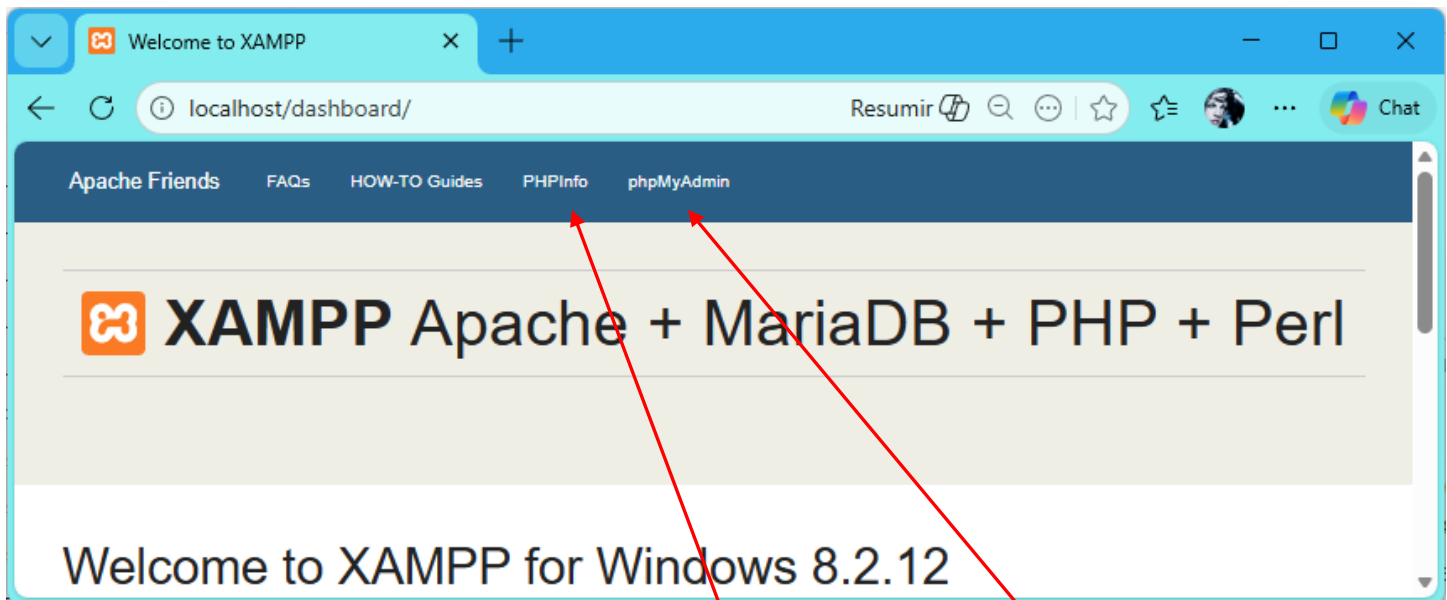


Ilustración 15: ¿Apache funciona? abre un navegador y se escribe <http://localhost>.

Para ver que funciona PHP entonces se selecciona PHPInfo, y más adelante phpMyAdmin

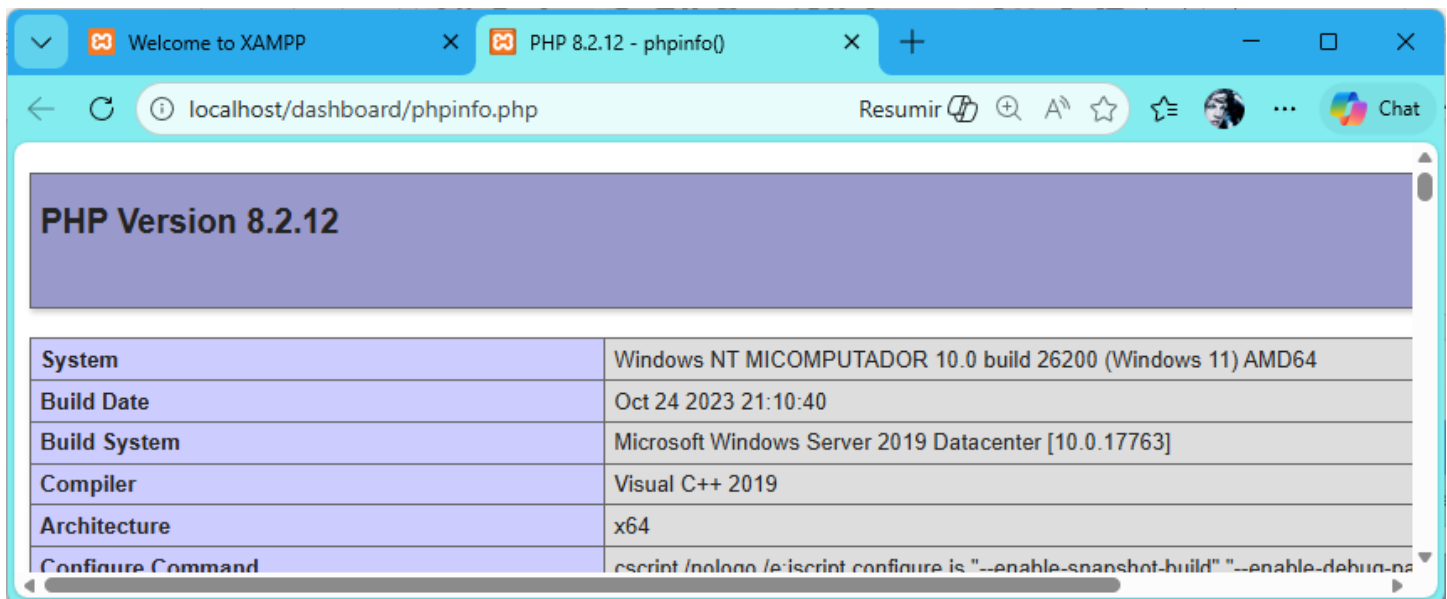


Ilustración 16: En *phpinfo* aparece el estado de PHP

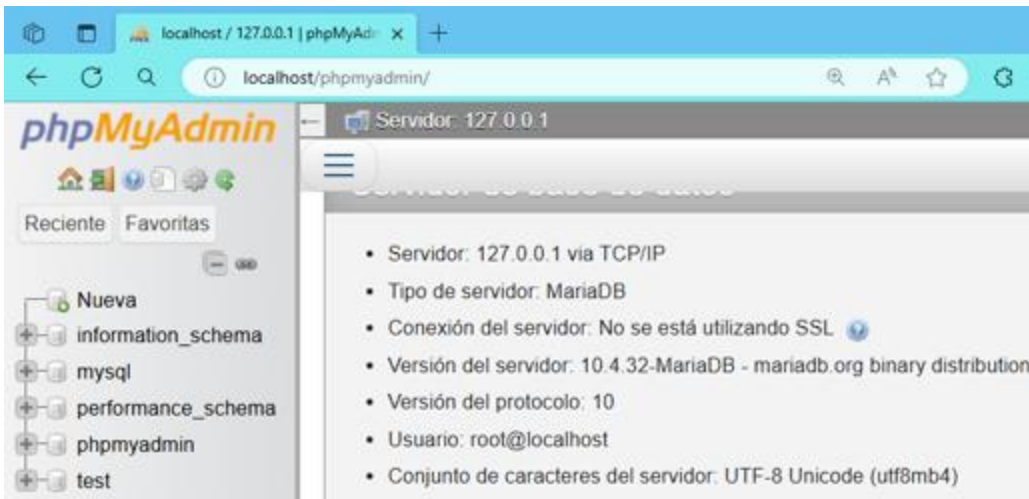


Ilustración 17: En phpMyAdmin aparece el estado de MySQL.

Si todo está funcionando correctamente entonces es momento de trabajar con PHP.

Configurando Visual Studio Code

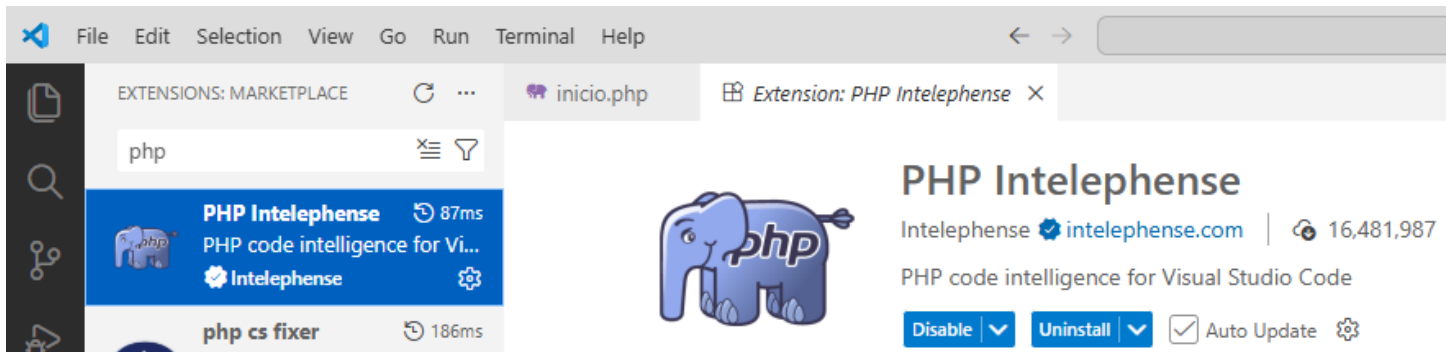


Ilustración 18: Instalar PHP Intelephense



Ilustración 19: Instalar php cs fixer

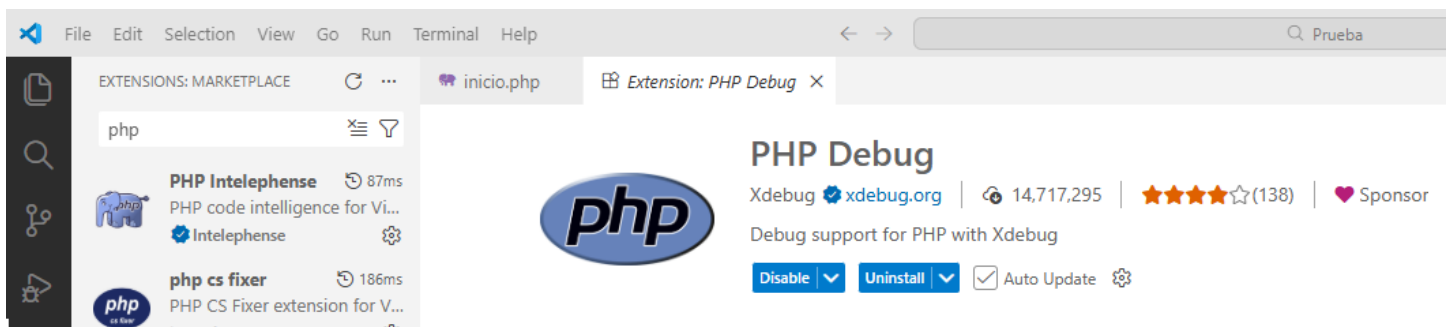


Ilustración 20: Instalar PHP Debug

Los programas en PHP inician con el juego de caracteres `<?php` y cierran con `?>`, pero está recomendado no usar el cierre (“close tag”)

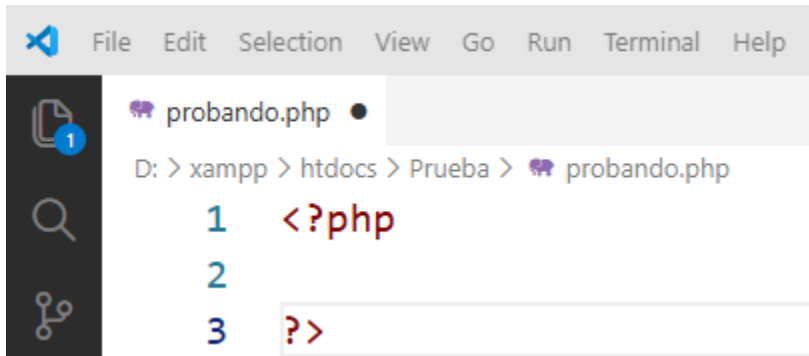


Ilustración 21: Etiquetas de inicio y cierre de script PHP

Luego, siguiendo con las recomendaciones, no se usará esa etiqueta de cierre en este manual.

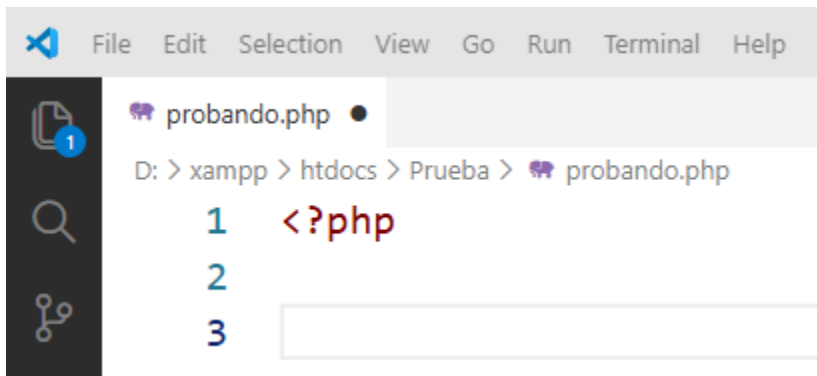


Ilustración 22: Se recomienda dejar solo la etiqueta de apertura

La razón de evitar la etiqueta de cierre es para evitar que existan líneas en blanco accidentales después del cierre.

¿Dónde se almacenan esos archivos?

Dependiendo del servidor de aplicaciones, hay un directorio o carpeta dónde se ponen los archivos. En el caso de XAMPP, la carpeta es:

D : \xampp\htdocs

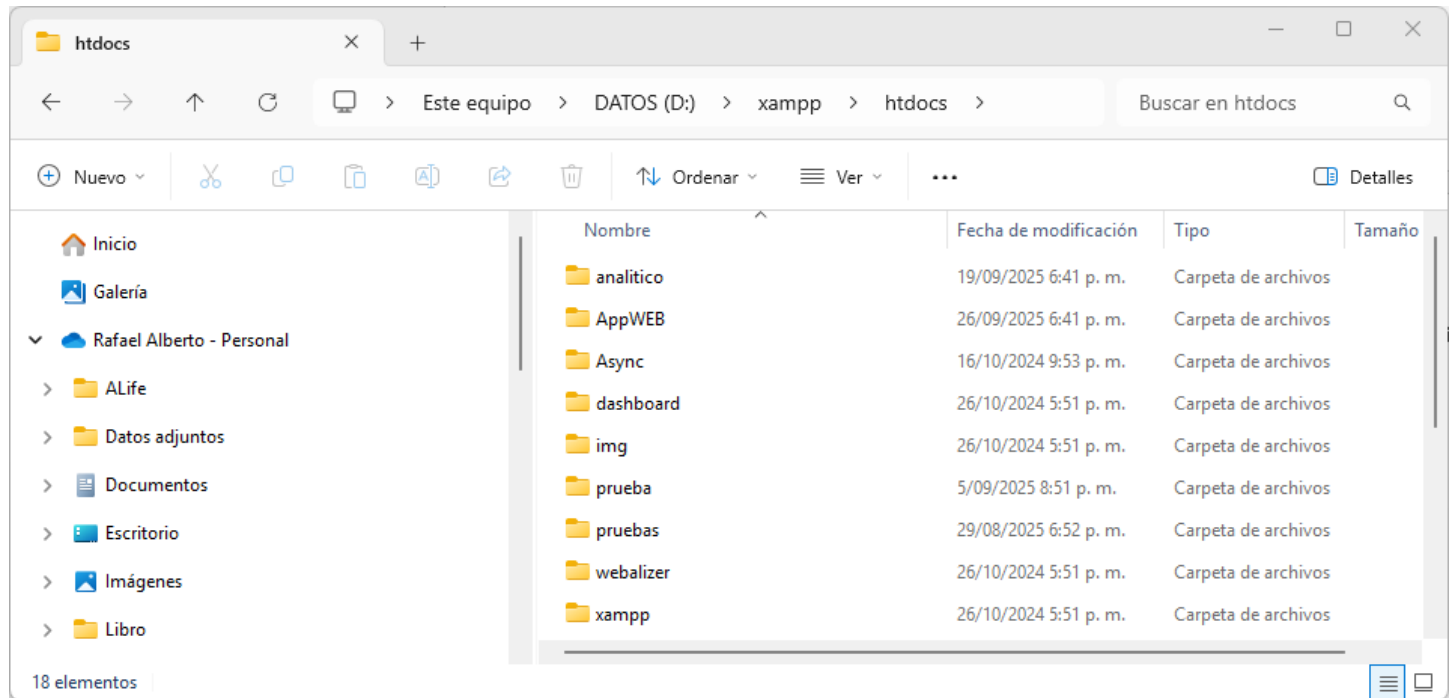


Ilustración 23: En C:\xampp\htdocs o D:\xampp\htdocs se ubican los archivos PHP

Y dentro de esa carpeta \htdocs se pueden crear subcarpetas, organizando mejor el código.

Comentarios en el código

Los comentarios son clave en la documentación del código, hacen la diferencia entre un software en que se pueda hacer mantenimiento y software que hay que desechar. Debe usarse bastante y bien. Similar a C++, si el comentario es de una sola línea, se usa // pero si se requieren varias líneas se puede encerrar entre /* y */ . Ver el código:

Directorio 01

```
<?php
//Comentario de una sola línea

/* Comentarios en
   varias
   líneas */
```

El tradicional “Hola Mundo”

Con la instrucción echo se muestran mensajes. Las instrucciones deben terminar en punto y coma (;)

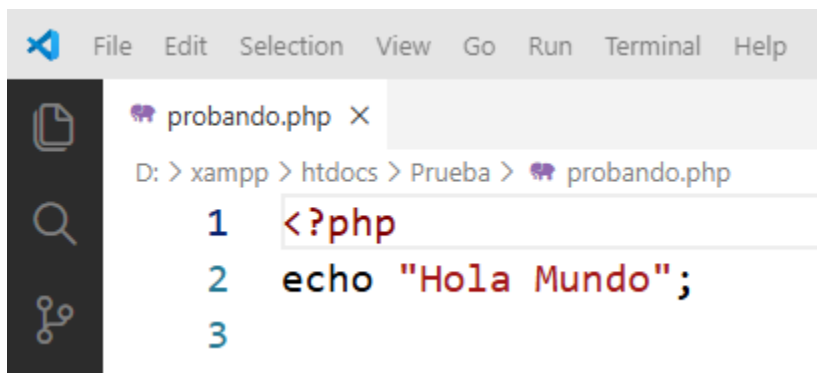


Ilustración 24: Imprimir un texto

Se guarda el documento como un archivo PHP. Hay que estar atento que en Tipo: diga PHP Hypertext Preprocessor file para guardarlo. Una vez ubicado el archivo en su lugar correcto, entonces se abre un navegador y se escribe la siguiente URL:

<http://localhost/Prueba/probando.php>

Cada parte:

http://	Es el protocolo de la aplicación Web. [8]
localhost/	Como el servidor de aplicaciones ejecuta en el propio PC, entonces la dirección URL es localhost. Aunque también se puede utilizar 127.0.0.1 [9]
Prueba/probando.php	Carpeta y el nombre que se le puso al archivo php



Ilustración 25: Inicia el navegador seleccionado para mostrar la página

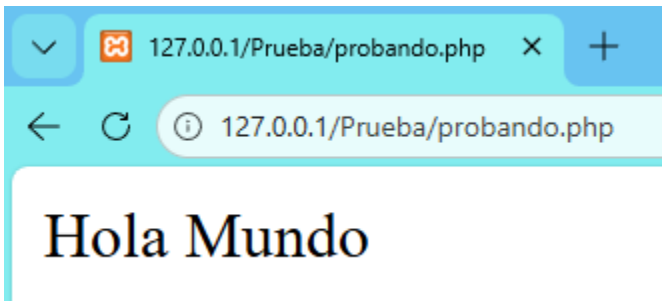


Ilustración 26: En vez de localhost, se puede usar 127.0.0.1

¡Advertencia! Se debe tener guardada la página antes de ejecutarla. Este es el código:

01/Ej02.php

```
<?php
//Imprimir una línea
echo "Hola Mundo";
```

Las comillas dobles pueden ser reemplazadas por comillas simples.

01/Ej03.php

```
<?php
//Imprimir una línea
echo 'Hola Mundo';
```

Y además es posible poner código HTML en su interior para hacer cambios en el texto

01/Ej04.php

```
<?php
//Imprimir una línea
echo '<b>Hola Mundo<b>';
```

Uso de variables

PHP declara las variables con el símbolo de \$ como prefijo, ejemplos: \$contador, \$valor, \$acumula. Es un lenguaje débilmente tipado por defecto, es decir, no se definen variables enteras, reales o de cadena. PHP detecta el tipo de dato cuando se le asigna el primer valor a la variable. Si se le asigna un texto, se considera que es una variable tipo texto (string).

01/Ej05.php

```
<?php
//Variables en PHP
$valA = 15;
$cadena = "Esta es una prueba";
$aproxima = -7.126;

echo $valA;
echo "<br>";
echo $cadena;
echo "<br>";
echo $aproxima;
```

La instrucción echo "
"; es para que haya un salto entre cada dato impreso. Se puede variar el código para impresión y que quede en una sola línea:

01/Ej06.php

```
<?php
//Variables en PHP
$valA = 78;
$cadena = "Cadena de texto";
$aproxima = -90.23;

echo $valA . "<br>";
echo $cadena . "<br>";
echo $aproxima;
```

O todo en una línea

01/Ej07.php

```
<?php
//Variables en PHP
$val = -901;
$cad = "Manual de PHP";
$aprox = 864.5;

echo $val . "<br>" . $cad . "<br>" . $aprox;
```

Operaciones matemáticas

Están disponibles suma(+), resta(-), multiplicación(*), división(/), potencia(**) y división modular(%). La división modular retorna el **residuo** de la división. Este es el código:

01/Ej08.php

```
<?php
//Aritmética
$valA = 12 + 45;
$valB = 55 - 89;
$valC = 7 * 8;
$valD = 21 / 3;
$valE = 5 ** 3; //5 al cubo
$valF = 5671 % 2; //Determina el residuo

echo $valA . "<br>" . $valB . "<br>";
echo $valC . "<br>" . $valD . "<br>";
echo $valE . "<br>" . $valF . "<br>";
```

Asignación

Se utiliza el operador = para asignar un valor a una variable. Tiene algunas variaciones para acotar operaciones

01/Ej09.php

```
<?php
//Asignación
$valA = 8;
$valB = $valA * 2;
echo $valA . " y " . $valB . "<br>";

$valA += 10; //Sumarle 10
$valB -= 5;  //Restarle 5
echo $valA . " y " . $valB . "<br>";

$valA *= 3; //Multiplique por 3
$valB /= 2; //Divida entre 2
echo $valA . " y " . $valB . "<br>";
```

Operaciones con cadenas

Se utiliza el operador punto(.) para concatenar cadenas.

01/Ej10.php

```
<?php
//Uso de cadenas
$txtA = "Una";
$txtB = "Cadena";
$txtC = $txtA . " --- " . $txtB;
echo $txtC;
```

¿Y qué sucede al combinar números y texto?

01/Ej11.php

```
<?php
//Uso de cadenas
$txtA = "Una";
$txtB = 45.98;
$txtC = $txtA . " --- " . $txtB;
echo $txtC;
```

Otro cambio, ambas variables con números reales.

01/Ej12.php

```
<?php
//Uso de cadenas
$txtA = 27.6;
$txtB = 45.98;
$txtC = $txtA . " --- " . $txtB;
echo $txtC;
```

Pero si se hace este cambio (poner a sumar las variables con valor de número real y luego concatenar una cadena):

01/Ej13.php

```
<?php
//Uso de cadenas
$txtA = 7;
$txtB = 5;
$txtC = $txtA . $txtB . "aaaaa";
echo $txtC;
```


¿Y si se hacen más operaciones matemáticas?

01/Ej14.php

```
<?php
//Uso de cadenas
$txtA = 7;
$txtB = 5;
$txtC = $txtA . $txtB . "aaaaa" . $txtA . $txtB;
echo $txtC;
```

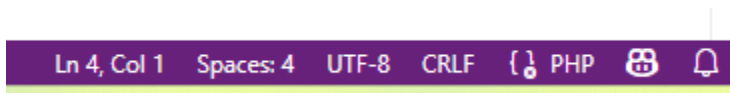
Impresión de caracteres propios del idioma español

Para imprimir caracteres como tildes, Ñs, abre interrogante o abre admiración. El código a continuación:

01/Ej15.php

```
<?php
//Uso de cadenas
echo "áéíóúÁÉÍÓÚÑñäëïöü¿?¡!";
```

Si al probar este código, no se ven bien los caracteres, entonces hay que verificar que la codificación sea UTF-8. Ver en la parte inferior de Visual Studio Code.



```
<?php
//Librería matemática. Constantes.
echo M_PI . "<br>"; // Pi
echo M_PI_2 . "<br>"; // Pi/2
echo M_PI_4 . "<br>"; // Pi/4
echo M_1_PI . "<br>"; // 1/Pi
echo M_2_PI . "<br>"; // 2/Pi
echo M_SQRTPI . "<br>"; // raíz cuadrada(Pi)
echo M_2_SQRTPI . "<br>"; // 2 / raíz cuadrada(Pi)
echo M_SQRT2 . "<br>"; //Raíz cuadrada de 2
echo M_SQRT3 . "<br>"; //Raíz cuadrada de 3
echo M_SQRT1_2 . "<br>"; // 1 / Raíz cuadrada de 2
echo M_E . "<br>"; // Número e
echo M_EULER . "<br>"; // Número Euler
echo M_LNPI . "<br>"; // Logaritmo natural de pi
echo M_LN10 . "<br>"; // Logaritmo natural de 10
echo M_LOG2E . "<br>"; // Logaritmo base 2 de E
echo M_LOG10E . "<br>"; // Logaritmo base 10 de E
```

Funciones trigonométricas

Las funciones trigonométricas deben ser con ángulos en radianes, no en grados.

```
<?php
//Librería matemática.
$angGrados = 35;
echo "Grados: " . $angGrados;
$angRadian = $angGrados*M_PI/180;
echo "<br>Radianes: " . $angRadian;
$seno = sin($angRadian); //Seno
echo "<br>Seno: " . $seno;
$coseno = cos($angRadian); //Coseno
echo "<br>Coseno: " . $coseno;
$tangente = tan($angRadian); //Tangente
echo "<br>Tangente: " . $tangente;
$arcoseno = asin($seno); //Arcoseno
echo "<br>Arcoseno: " . $arcoseno;
$arcocoseno = acos($coseno); //Arcocoseno
echo "<br>Arcocoseno: " . $arcocoseno;
$arcotangente = atan($tangente); //Arcotangente
echo "<br>Arcotangente: " . $arcotangente;
```

```
<?php
//Librería matemática.
echo "Raíz cúbica: " . pow(1331, 1/3);
echo "<br>Techo: " . ceil(4.01);
echo "<br>Exponencial: " . exp(1); //Número e
echo "<br>Piso: " . floor(4.99);
echo "<br>Log: " . log(2); //Logaritmo natural
echo "<br>Mayor: " . max(5, 9, 7, 3); //Máximo valor de una lista
echo "<br>Menor: " . min(5, 9, 7, 3); //Mínimo valor de una lista
echo "<br>Potencia: " . pow(2, 5); //base, potencia
echo "<br>Aleatorio: " . rand(); //Aleatorio entre 0 y getrandmax()
echo "<br>Máximo Aleatorio: " . getrandmax(); //Máximo aleatorio
echo "<br>Aleatorio: " . rand(20, 30); //Aleatorio entre 20 y 30 incluidos
echo "<br>Redondeo: " . round(3.7);
echo "<br>Redondeo: " . round(3.3);
echo "<br>Redondeo: " . round(3.5);
echo "<br>Raíz Cuadrada: " . sqrt(144);
echo "<br>Truncar: " . floor(8.923); //Trunca al entero
```

Ejemplos de cálculos

Cálculo del área de un triángulo dada la longitud de sus lados

```
<?php
//Cálculo del área de un triángulo según sus lados (valores de ejemplo)
$ladoA = 3;
$ladoB = 4;
$ladoC = 5;
$s = ($ladoA+$ladoB+$ladoC)/2;
$area = sqrt($s*($s-$ladoA)*($s-$ladoB)*($s-$ladoC));
echo "Área es: " . $area;
```

```
<?php
//Área y perímetro del círculo
$radio = 7; //Un valor de ejemplo de radio
$area = M_PI * $radio * $radio;
$perimetro = M_PI * $radio * 2;
echo "Área es: " . $area;
echo "<br>Perímetro es: " . $perimetro;
```

```
<?php
//Área y volumen del cilindro
$radio = 8;
$altura = 12;
$area = 2 * M_PI * $radio * ($altura + $radio);
$volumen = M_PI * $radio * $radio * $altura;
echo "Área es: " . $area;
echo "<br>Volumen es: " . $volumen;
```

Corrigiendo errores en PHP

El servidor envía para ser visualizados por el navegador los errores detectados en el código.

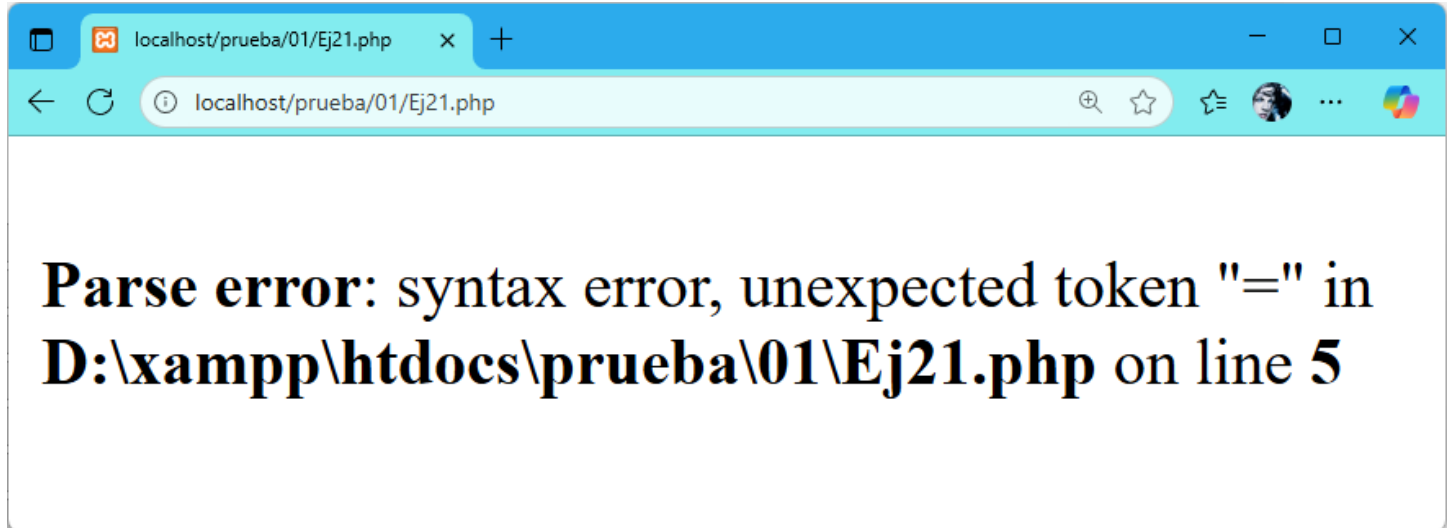


Ilustración 27: El navegador muestra los errores enviados por el servidor

```
<?php
//Si condicional
$valA = 17;
$valB = 13;
if ($valA > $valB){
    echo $valA . " es mayor que " . $valB;
}
else {
    echo $valA . " es menor o igual que " . $valB;
}
```

```
<?php
//Si condicional. Uso de else if
$valA = 17;
$valB = 17;
if ($valA > $valB){
    echo $valA . " es mayor que " . $valB;
} else if ($valA == $valB){
    echo $valA . " es igual que " . $valB;
}
else {
    echo $valA . " es menor que " . $valB;
}
```

Los operadores de comparación son:

Símbolo	Significado
>	Mayor que
>=	Mayor o igual que
<	Menor que
<=	Menor o igual que
==	Es igual a
!=	Es diferente que
===	Comparación de igualdad estricta de valor y tipo. Por ejemplo 5 === "5" sería falso En cambio 5 == "5" sería verdadero (porque == es un comparador menos estricto)
!==	Comparación de diferencia estricta de valor y tipo.

Operadores lógicos: Conjunción (Y, &&) y Disyunción (O, ||)

El operador AND, Y en PHP es &&. El operador OR, O en PHP es ||

02/Ej03.php

```
<?php
//Tipo de triángulo
$ladoA = 17;
$ladoB = 18;
$ladoC = 19;
if ($ladoA==$ladoB && $ladoB==$ladoC) {
    echo "Es equilátero";
} else if ($ladoA!=$ladoB && $ladoA!=$ladoC && $ladoB!=$ladoC) {
    echo "Es escaleno";
} else {
    echo "Es isósceles";
}
```

02/Ej04.php

```
<?php
//Tipo de triángulo
$ladoA = 17;
$ladoB = 18;
$ladoC = 19;
if ($ladoA==$ladoB && $ladoB==$ladoC) {
    echo "Es equilátero";
} else if ($ladoA==$ladoB || $ladoB==$ladoC || $ladoA==$ladoC) {
    echo "Es isósceles";
} else {
    echo "Es escaleno";
}
```

```
<?php
//Uso del switch
$valor = 9;
switch($valor){
    case 0: echo "Escribió cero"; break;
    case 1: echo "Digitó 1"; break;
    case 2: echo "Tecleó dos"; break;
    case 3: echo "Presionó 3"; break;
    default: echo "Ingresó otro valor";
}
```

Se pueden usar varias líneas en el switch. El cierre es la palabra reservada “break”.

```
<?php
//Uso del switch
$valor = 1;
switch($valor){
    case 0: echo "Escribió cero"; break;
    case 1: echo "Digitó 1";
            echo "<br>Escribió uno";
            echo "<br>Registró uno";
            break;
    case 2: echo "Tecleó dos"; break;
    case 3: echo "Presionó 3"; break;
    default: echo "Ingresó otro valor";
}
```

Ciclos. Uso del “for”

02/Ej07.php

```
<?php
//Ciclo for creciente
for ($cont = 1; $cont<=100; $cont++){
    echo $cont . "<br>";
}
```

02/Ej08.php

```
<?php
//Ciclo for decreciente
for ($cont = 100; $cont>=1; $cont--){
    echo $cont . "<br>";
}
```

02/Ej09.php

```
<?php
//Ciclo for creciente de 3 en 3
for ($cont = 0; $cont<=300; $cont+=3){
    echo $cont . "<br>";
}
```

02/Ej10.php

```
<?php
//Ciclo for creciente de 10 en 10. Se detiene antes de 500
for ($cont = 0; $cont<500; $cont+=10){
    echo $cont . "<br>";
}
```

02/Ej11.php

```
<?php
//Ciclo for creciente aumentando el doble del anterior
for ($cont = 1; $cont<=5000; $cont*=2){
    echo $cont . ", ";
}
//Resultado:1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096
```



```
<?php
//Ciclo for decreciente disminuyendo la mitad del anterior
for ($cont = 4096; $cont>=1; $cont/=2){
    echo $cont . ", ";
}
//Resultado: 4096, 2048, 1024, 512, 256, 128, 64, 32, 16, 8, 4, 2, 1,
```

```
<?php
/* Ciclo for con dos variables. Obsérvese las dos
condiciones que deben darse */
for ($x=1, $y=34; $x<=10 && $y>=28; $x++, $y--){
    echo $x . ", " . $y . "<br>";
}
/* Resultado:
1, 34
2, 33
3, 32
4, 31
5, 30
6, 29
7, 28
*/
```

```
<?php
/* Ciclo for con dos variables. Obsérvese las dos
condiciones que deben darse cambiando && por || */
for ($x=1, $y=34; $x<=10 || $y>=28; $x++, $y--){
    echo $x . ", " . $y . "<br>";
}
/* Resultado:
1, 34
2, 33
3, 32
4, 31
5, 30
6, 29
7, 28
8, 27
9, 26
10, 25
*/
```

Ciclo while

02/Ej15.php

```
<?php
//Ciclo while creciente
$cont = 1;
while ($cont<=100){
    echo $cont . "<br>";
    $cont++;
}
```

02/Ej16.php

```
<?php
//Ciclo while decreciente
$cont = 100;
while ($cont>=1){
    echo $cont . "<br>";
    $cont--;
}
```

02/Ej17.php

```
<?php
//Ciclo while creciente de 3 en 3
$cont = 0;
while ($cont<=300){
    echo $cont . "<br>";
    $cont+=3;
}
```

02/Ej18.php

```
<?php
//Ciclo while creciente de 10 en 10. Se detiene antes de 500
$cont = 0;
while ($cont<500){
    echo $cont . "<br>";
    $cont+=10;
}
```

```
<?php
//Ciclo while creciente aumentando el doble del anterior
$cont = 1;
while ($cont<=5000){
    echo $cont . ", ";
    $cont*=2;
}
//Resultado:1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096
```

```
<?php
//Ciclo while decreciente disminuyendo la mitad del anterior
$cont = 4096;
while ($cont>=1){
    echo $cont . ", ";
    $cont/=2;
}
//Resultado: 4096, 2048, 1024, 512, 256, 128, 64, 32, 16, 8, 4, 2, 1,
```

```
<?php
/* Ciclo while con dos variables. Obsérvese las dos
condiciones que deben darse */
$x=1;
$y=34;
while ($x<=10 && $y>=28){
    echo $x . ", " . $y . "<br>";
    $x++;
    $y--;
}
/* Resultado:
1, 34
2, 33
3, 32
4, 31
5, 30
6, 29
7, 28
*/
```

```
<?php
/* Ciclo while con dos variables. Obsérvese las dos
   condiciones que deben darse cambiando && por || */
$x=1;
$y=34;
while ($x<=10 || $y>=28){
    echo $x . ", " . $y . "<br>";
    $x++;
    $y--;
}
/* Resultado:
1, 34
2, 33
3, 32
4, 31
5, 30
6, 29
7, 28
8, 27
9, 26
10, 25
*/
```

Ciclo do - while

02/Ej23.php

```
<?php
//Ciclo do-while creciente
$cont = 1;
do{
    echo $cont . "<br>";
    $cont++;
}while ($cont<=100);
```

02/Ej24.php

```
<?php
//Ciclo do-while decreciente
$cont = 100;
do{
    echo $cont . "<br>";
    $cont--;
}while ($cont>=1);
```

02/Ej25.php

```
<?php
//Ciclo do-while creciente de 3 en 3
$cont = 0;
do{
    echo $cont . "<br>";
    $cont+=3;
}while ($cont<=300);
```

02/Ej26.php

```
<?php
//Ciclo do-while creciente de 10 en 10. Se detiene antes de 500
$cont = 0;
do{
    echo $cont . "<br>";
    $cont+=10;
}while ($cont<500);
```

```
<?php
//Ciclo while creciente aumentando el doble del anterior
$cont = 1;
do{
    echo $cont . ", ";
    $cont*=2;
}while ($cont<=5000);
//Resultado:1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096
```

```
<?php
//Ciclo while decreciente disminuyendo la mitad del anterior
$cont = 4096;
do{
    echo $cont . ", ";
    $cont/=2;
}while ($cont>=1);
//Resultado: 4096, 2048, 1024, 512, 256, 128, 64, 32, 16, 8, 4, 2, 1,
```

```
<?php
/* Ciclo while con dos variables. Obsérvese las dos
condiciones que deben darse */
$x=1;
$y=34;
do{
    echo $x . ", " . $y . "<br>";
    $x++;
    $y--;
}while ($x<=10 && $y>=28);
/* Resultado:
1, 34
2, 33
3, 32
4, 31
5, 30
6, 29
7, 28
*/
```

```
<?php
/* Ciclo do-while con dos variables. Obsérvese las dos
   condiciones que deben darse cambiando && por || */
$x=1;
$y=34;
do{
    echo $x . ", " . $y . "<br>";
    $x++;
    $y--;
}while ($x<=10 || $y>=28);
/* Resultado:
1, 34
2, 33
3, 32
4, 31
5, 30
6, 29
7, 28
8, 27
9, 26
10, 25
*/
```

Uso del break

02/Ej31.php

```
<?php
//Ciclo for creciente, se detiene en 50
for ($cont = 1; $cont<=100; $cont++){
    echo $cont . "<br>";
    if ($cont>=50) break; //Sale del ciclo
}
```

02/Ej32.php

```
<?php
//Ciclo while creciente. Sale cuando cont sea 50
$cont = 1;
while ($cont<=100){
    echo $cont . "<br>";
    $cont++;
    if ($cont>=50) break; //Sale del ciclo
}
```

02/Ej33.php

```
<?php
//Ciclo do-while creciente. Sale cuando cont sea 50
$cont = 1;
do{
    echo $cont . "<br>";
    $cont++;
    if ($cont>=50) break; //Sale del ciclo
}while ($cont<=100);
```


Uso del continue

La sentencia “continue” hace que el programa haga inmediatamente la siguiente iteración ignorando las instrucciones siguientes dentro del ciclo.

02/Ej34.php

```
<?php
//Ciclo for creciente, se salta los pares
for ($cont = 1; $cont<=10; $cont++){
    //Si es par, salta a la siguiente iteración
    if ($cont%2==0) continue;
    echo $cont . ", ";
}
//Resultado: 1, 3, 5, 7, 9,
```

02/Ej35.php

```
<?php
//Ciclo while creciente. Se salta los pares
$cont = 1;
while ($cont<=100){
    $cont++;
    if ($cont%2==0) continue; //Salta a la siguiente iteración
    echo $cont . "<br>";
}
```

02/Ej36.php

```
<?php
//Ciclo do-while creciente. Se salta los pares
$cont = 1;
do{
    $cont++;
    if ($cont%2==0) continue; //Salta a la siguiente iteración
    echo $cont . "<br>";
}while ($cont<=100);
```

Ciclos anidados y salir de estos

02/Ej37.php

```
<?php
//Ciclos anidados
for ($x = 1; $x <= 3; $x++){
    for ($y = 45; $y <= 50; $y++){
        for ($z = 7; $z <= 9; $z++){
            echo $x . ";" . $y . ";" . $z . "<br>";
        }
    }
}
```

Para salir de ciclos anidados se usa “break *número*” donde *número* es la cantidad de ciclos que se rompen:

02/E38.php

```
<?php
//Ciclos anidados
for ($x = 1; $x <= 3; $x++){
    for ($y = 45; $y <= 50; $y++){
        for ($z = 7; $z <= 9; $z++){
            echo $x . ";" . $y . ";" . $z . "<br>";
            if ($z==8) break 2; //Rompe ciclo de $y y $z
        }
    }
}
```

02/Ej39.php

```
<?php
//Ciclos anidados
for ($x = 1; $x <= 3; $x++){
    for ($y = 45; $y <= 50; $y++){
        for ($z = 7; $z <= 9; $z++){
            echo $x . ";" . $y . ";" . $z . "<br>";
            if ($z==8) break 3; //Rompe ciclo de $x, $y y $z
        }
    }
}
```

```
<?php
$numEntero = 13; //Tipo entero
$Primo = EsPrimo($numEntero); //Llama la función
if ($Primo)
    echo $numEntero . " es primo<br>";
else
    echo $numEntero . " NO es primo<br>";

//Función que retorna true si el número enviado por parámetro es primo
function EsPrimo($num){
    if ($num<=1) return false;
    if ($num==2) return true;
    if ($num%2==0) return false;
    for ($divide = 3; $divide <= sqrt($num); $divide+=2)
        if ($num%$divide == 0) return false;
    return true;
}
```

El anterior programa dado un número responde si ese número es primo o no.

Cuando hay una buena cantidad de funciones, es recomendable crear una librería. Se genera un archivo con extensión .php . Y este sería el código dentro de ese archivo Libreria.php

```
<?php
//Retorna true si el número es palíndromo
function EsPalindromo($num){
    if ($num==InvierteNumero($num)) return true;
    return false;
}

//Retorna las dos últimas cifras del número
function retornadosultimascifras($num){
    return $num%100;
}

//Dice cuántas cifras de determinado número hay en el número enviado
function cifrashalladas($num, $cifra){
    $acumula = 0;
    while ($num!=0){
        if ($num%10 == $cifra) $acumula++;
        $num = floor($num/10);
    }
    return $acumula;
}
```

```

}

//Invierte un número
function InvierteNumero($num) {
    $multiplica = pow(10, TotalCifras($num)-1);
    $acumula = 0;
    while($num!=0) {
        $cifra = $num%10;
        $acumula += $cifra * $multiplica;
        $multiplica /= 10;
        $num = floor($num/10);
    }
    return $acumula;
}

//Retorna la primera cifra de un número
function PrimeraCifra($num) {
    $primera = 0;
    while($num!=0) {
        $primera = $num%10;
        $num = floor($num/10);
    }
    return $primera;
}

//Retorna el número de cifras de un número
function TotalCifras($num) {
    $cuenta = 0;
    while($num!=0) {
        $cuenta++;
        $num = floor($num/10);
    }
    return $cuenta;
}

//Retorna true si un número es impar
function EsImpar($num) {
    if ($num%2==1) return true;
    return false;
}

//Retorna true si un número es par
function EsPar($num) {
    if ($num%2==0) return true;
    return false;
}

//Retorna la sumatoria de las cifras de un número
function SumaCifras($num) {
    $acum = 0;
    while($num!=0) {

```

```

        $cifra = $num%10;
        $acum += $cifra;
        $num = floor($num/10);
    }
    return $acum;
}

//Retorna la sumatoria de las cifras pares de un número
function SumaCifrasPares($num) {
    $acum = 0;
    while($num!=0) {
        $cifra = $num%10;
        if ($cifra%2==0) $acum += $cifra;
        $num = floor($num/10);
    }
    return $acum;
}

//Retorna la sumatoria de las cifras impares de un número
function SumaCifrasImpares($num) {
    $acum = 0;
    while($num!=0) {
        $cifra = $num%10;
        if ($cifra%2!=0) $acum += $cifra;
        $num = floor($num/10);
    }
    return $acum;
}

//Retorna el producto de las cifras de un número
function MultiplicaCifras($num) {
    $acum = 1;
    while($num!=0) {
        $cifra = $num%10;
        $acum *= $cifra;
        $num = floor($num/10);
    }
    return $acum;
}

//Retorna el producto de las cifras impares de un número
function MultiplicaCifrasImpares($num) {
    $acum = 1;
    while($num!=0) {
        $cifra = $num%10;
        if ($cifra%2!=0) $acum *= $cifra;
        $num = floor($num/10);
    }
    return $acum;
}

```

```

//Retorna el producto de las cifras pares de un número
function MultiplicaCifrasPares($num){
    $acum = 1;
    while($num!=0){
        $cifra = $num%10;
        if ($cifra%2==0) $acum *= $cifra;
        $num = floor($num/10);
    }
    return $acum;
}

//Retorna la antepenúltima $cifra de un número entero
function AntepenultimaCifra($num){
    return floor($num/100)%10;
}

//Retorna la penúltima $cifra de un número entero
function PenultimaCifra($num){
    return floor($num/10)%10;
}

//Retorna la última $cifra de un número entero
function UltimaCifra($num){
    return $num%10;
}

//Retorna true si el número enviado por parámetro es primo
function EsPrimo($num){
    if ($num<=1) return false;
    if ($num==2) return true;
    if ($num%2==0) return false;
    for ($divide = 3; $divide <= sqrt($num); $divide+=2)
        if ($num%$divide == 0) return false;
    return true;
}

//Retorna true si todas las cifras son pares
function TodasCifrasPares($num){
    while($num!=0){
        $cifra = $num%10;
        if ($cifra%2!=0) return false;
        $num = floor($num/10);
    }
    return true;
}

//Retorna true si todas las cifras son impares
function TodasCifrasImpares($num){
    if ($num==0) return false;
    while($num!=0){
        $cifra = $num%10;

```

```

        if ($cifra%2==0) return false;
        $num = floor($num/10);
    }
    return true;
}

//Retorna el número de cifras pares
function TotalCifrasPares($num){
    $acum = 0;
    while($num!=0){
        $cifra = $num%10;
        if ($cifra%2==0) $acum++;
        $num = floor($num/10);
    }
    return $acum;
}

//Retorna el número de cifras impares
function TotalCifrasImpares($num){
    $acum = 0;
    while($num!=0){
        $cifra = $num%10;
        if ($cifra%2!=0) $acum++;
        $num = floor($num/10);
    }
    return $acum;
}

//Retorna la $cifra más alta
function LaCifraMasAlta($num){
    $cifra = 0;
    while ($num!=0){
        if ($num%10 > $cifra) $cifra = $num%10;
        $num = floor($num/10);
    }
    return $cifra;
}

//Retorna la $cifra más baja
function LaCifraMasBaja($num){
    $cifra = 9;
    while ($num!=0){
        if ($num%10 < $cifra) $cifra = $num%10;
        $num = floor($num/10);
    }
    return $cifra;
}

//Retorna true si $num tiene sólo cifras menores o iguales de $cifra
function SoloCifrasMenorIgual($num, $cifra){
    while ($num!=0){

```

```

        if ($num%10 > $cifra) return false;
        $num = floor($num/10);
    }
    return true;
}

//Retorna true si $num tiene sólo cifras mayores o iguales de $cifra
function SoloCifrasMayorIgual($num, $cifra){
    while ($num!=0){
        if ($num%10 < $cifra) return false;
        $num = floor($num/10);
    }
    return true;
}

//Retorna true si todas las cifras son distintas
function DistintasCifras($num){
    for ($cifra=0; $cifra<=9; $cifra++){
        $numero = $num;
        $cuenta = 0;
        while ($numero!=0){
            if ($numero%10 == $cifra) $cuenta++;
            if ($cuenta > 1) return false;
            $numero = floor($numero/10);
        }
    }
    return true;
}

//Retorna si todas las cifras pares son distintas
function DistintasCifrasPares($num){
    for ($cifra=0; $cifra<=8; $cifra+=2){
        $numero = $num;
        $cuenta = 0;
        while ($numero!=0){
            if ($numero%10 == $cifra) $cuenta++;
            if ($cuenta > 1) return false;
            $numero = floor($numero/10);
        }
    }
    return true;
}

//Retorna si todas las cifras impares son distintas
function DistintasCifrasImpares($num){
    for ($cifra=1; $cifra<=9; $cifra+=2){
        $numero = $num;
        $cuenta = 0;
        while ($numero!=0){
            if ($numero%10 == $cifra) $cuenta++;
            if ($cuenta > 1) return false;
        }
    }
}

```



```

        $numero = floor($numero/10);
    }
}
return true;
}

//Eleva al cuadrado cada cifra y retorna la suma
function sumacifrascuadrado($num){
    $acumula = 0;
    while ($num!=0){
        $acumula+= ($num%10)*($num%10);
        $num = floor($num/10);
    }
    return $acumula;
}

```

Para usar la librería se llama desde el archivo principal con `include "Libreria.php";` siempre y cuando los dos archivos estén en el mismo directorio o carpeta.

03/Ej02.php

```

<?php
include "Libreria.php";

$valor = 789121;
$cifra = 3;
echo "Número es: ". $valor . "<br>";

echo "Palíndromo: ";
if (EsPalindromo($valor))
    echo "Verdadero<br>";
else echo "Falso<br>";

echo "Últimas dos cifras: ";
echo retornadosultimascifras($valor) . "<br>";

echo "Total de cifra " . $cifra . " halladas: ";
echo cifrashalladas($valor, $cifra) . "<br>";

echo "Inverso: ";
echo InvierteNumero($valor) . "<br>";

echo "Primera cifra: ";
echo PrimeraCifra($valor) . "<br>";

echo "Total cifras: ";
echo TotalCifras($valor) . "<br>";

echo "¿Impar?: ";

```

```

if (EsImpar($valor))
    echo "Verdadero<br>";
else
    echo "Falso<br>";

echo "¿Par?: ";
if (EsPar($valor))
    echo "Verdadero<br>";
else
    echo "Falso<br>";

echo "Sumatoria de cifras: ";
echo SumaCifras($valor) . "<br>";

echo "Última cifra: ";
echo UltimaCifra($valor) . "<br>";

echo "¿Primo?: ";
if (EsPrimo($valor))
    echo "Verdadero<br>";
else
    echo "Falso<br>";

echo "¿Todas sus cifras son pares?: ";
if (TodasCifrasPares($valor))
    echo "Verdadero<br>";
else
    echo "Falso<br>";

echo "¿Todas sus cifras son impares?: ";
if (TodasCifrasImpares($valor))
    echo "Verdadero<br>";
else
    echo "Falso<br>";

echo "Total cifras pares: ";
echo TotalCifrasPares($valor) . "<br>";

echo "Total cifras impares: ";
echo TotalCifrasImpares($valor) . "<br>";

echo "Cifra más alta: ";
echo LaCifraMasAlta($valor) . "<br>";

echo "Cifra más baja: ";
echo LaCifraMasBaja($valor) . "<br>";

echo "Sólo hay cifras menores o iguales a " . $cifra . ": ";
if (SoloCifrasMenorIgual($valor, $cifra))
    echo "Verdadero<br>";
else

```

```

    echo "Falso<br>";

echo "Sólo hay cifras mayores o iguales a " . $cifra . ": ";
if (SoloCifrasMayorIgual($valor, $cifra))
    echo "Verdadero<br>";
else
    echo "Falso<br>";

echo "¿Tiene distintas cifras?: ";
if (DistintasCifras($valor))
    echo "Verdadero<br>";
else
    echo "Falso<br>";

echo "¿Tiene distintas cifras pares?: ";
if (DistintasCifrasPares($valor))
    echo "Verdadero<br>";
else
    echo "Falso<br>";

echo "¿Tiene distintas cifras impares?: ";
if (DistintasCifrasImpares($valor))
    echo "Verdadero<br>";
else
    echo "Falso<br>";

echo "Sumatoria de sus cifras al cuadrado: ";
echo sumacifrascuadrado($valor) . "<br>";

```

De esa forma separa el código PHP y hace más legible el código.

Ejemplos de uso de la librería

03/Ej03.php

```

<?php
include "Libreria.php";

$minimo = 1000;
$maximo = 9999;

//Cantidad de números que la suma de las tres últimas cifras es par
$cuenta = 0;
for ($num = $minimo; $num<=$maximo; $num++){
    if (EsPar(AntepenultimaCifra($num) + UltimaCifra($num) +
PenultimaCifra($num))){
        $cuenta++;
    }
}

```

```

}

Imprime($minimo, $maximo, "Cantidad de números que la suma de las tres
últimas cifras es par", $cuenta);

//Imprime el resultado en el navegador
function Imprime($minimo, $maximo, $texto, $resultado){
    echo "Entre " . $minimo . " y " . $maximo . ". " . $texto . ": " .
    $resultado . "<br>";
}

```

03/Ej04.php

```

<?php
include "Libreria.php";

$minimo = 1000;
$maximo = 9999;

//Cantidad de números que la multiplicación de las tres últimas cifras es
igual a la suma de esas tres últimas cifras
$cuenta = 0;
for ($num = $minimo; $num<=$maximo; $num++){
    if(AntepenultimaCifra($num) * UltimaCifra($num) * PenultimaCifra($num)
    == AntepenultimaCifra($num) + UltimaCifra($num) + PenultimaCifra($num)){
        $cuenta = $cuenta + 1;
    }
}
Imprime($minimo, $maximo, "Cantidad de números que la multiplicación de
las tres últimas cifras es igual a la suma de esas tres últimas cifras",
$cuenta);

//Imprime el resultado en el navegador
function Imprime($minimo, $maximo, $texto, $resultado){
    echo "Entre " . $minimo . " y " . $maximo . ". " . $texto . ": " .
    $resultado . "<br>";
}

```

```

<?php
include "Libreria.php";

$minimo = 8000;
$maximo = 9000;

//Cantidad de números que cada una de sus tres últimas cifras es menor de
5
$cuenta = 0;
for ($num = $minimo; $num<=$maximo; $num++){
    if (AntepenultimaCifra($num) < 5 && PenultimaCifra($num) < 5 &&
UltimaCifra($num) < 5){
        echo $num . ", ";
        $cuenta = $cuenta + 1;
    }
}
echo "<br>";
Imprime($minimo, $maximo, "Cantidad de números que cada una de sus tres
últimas cifras es menor de 5", $cuenta);

//Imprime el resultado en el navegador
function Imprime($minimo, $maximo, $texto, $resultado){
    echo "Entre " . $minimo . " y " . $maximo . ". " . $texto . ": " .
$resultado . "<br>";
}

```

```

<?php
include "Libreria.php";

$minimo = 8500;
$maximo = 8700;

//Cantidad de números que al juntar la antepenúltima cifra y la última
cifra se obtenga un primo
$contar = 0;
for ($num = $minimo; $num<=$maximo; $num++){
    $nuevo = AntepenultimaCifra($num) * 10 + UltimaCifra($num);
    if (EsPrimo($nuevo)){
        echo $num . ", ";
        $contar = $contar + 1;
    }
}
echo "<br>";

```

```

Imprime($minimo, $maximo, "Cantidad de números que al juntar la
antepenúltima cifra y la última cifra se obtenga un primo", $contar);

//Imprime el resultado en el navegador
function Imprime($minimo, $maximo, $texto, $resultado){
    echo "Entre " . $minimo . " y " . $maximo . ". " . $texto . ": " .
$resultado . "<br>";
}

```

03/Ej07.php

```

<?php
include "Libreria.php";

$minimo = 4000;
$maximo = 7000;

//Cantidad de números primos que no tengan el número 3 en sus cifras
$contar = 0;
for ($num = $minimo; $num<=$maximo; $num++){
    if (cifrashalladas($num, 3) == 0 && EsPrimo($num)) {
        echo $num . ", ";
        $contar = $contar + 1;
    }
}
echo "<br>";
Imprime($minimo, $maximo, "Cantidad de números primos que no tengan el
número 3 en sus cifras", $contar);

//Imprime el resultado en el navegador
function Imprime($minimo, $maximo, $texto, $resultado){
    echo "Entre " . $minimo . " y " . $maximo . ". " . $texto . ": " .
$resultado . "<br>";
}

```

03/Ej08.php

```

<?php
include "Libreria.php";

$minimo = 1000;
$maximo = 9999;

//Cantidad de números palíndromos que tengan mínimo dos veces el 7 en sus
cifras
$contar = 0;
for ($num = $minimo; $num<=$maximo; $num++){
    if (cifrashalladas($num, 7) >= 2 && EsPalindromo($num)) {
        echo $num . ", ";
    }
}

```

```

        $contar = $contar + 1;
    }
}

echo "<br>";
Imprime($minimo, $maximo, "Cantidad de números palíndromos que tengan
mínimo dos veces el 7 en sus cifras", $contar);

//Imprime el resultado en el navegador
function Imprime($minimo, $maximo, $texto, $resultado){
    echo "Entre " . $minimo . " y " . $maximo . ". " . $texto . ": " .
$resultado . "<br>";
}

```

03/Ej09.php

```

<?php
include "Libreria.php";

$minimo = 5000;
$maximo = 9000;

//Cantidad de números que tengan solo cifras pares y al menos un número 4
$contar = 0;
for ($num = $minimo; $num<=$maximo; $num++){
    if (cifrashalladas($num, 4) >= 1 && TodasCifrasPares($num)) {
        echo $num . ", ";
        $contar = $contar + 1;
    }
}
echo "<br>";
Imprime($minimo, $maximo, "Cantidad de números que tengan solo cifras
pares y al menos un número 4", $contar);

//Imprime el resultado en el navegador
function Imprime($minimo, $maximo, $texto, $resultado){
    echo "Entre " . $minimo . " y " . $maximo . ". " . $texto . ": " .
$resultado . "<br>";
}

```

```

<?php
include "Libreria.php";

$minimo = 7700;
$maximo = 8000;

//Cantidad de números que la suma de sus cifras es impar y al menos tenga
una vez el número 7
$contar = 0;
for ($num = $minimo; $num<=$maximo; $num++){
    if (EsImpar(SumaCifras($num)) && cifrashalladas($num, 7) >= 1) {
        echo $num . ", ";
        $contar = $contar + 1;
    }
}

echo "<br>";
Imprime($minimo, $maximo, "Cantidad de números que la suma de sus cifras
es impar y al menos tenga una vez el número 7", $contar);

//Imprime el resultado en el navegador
function Imprime($minimo, $maximo, $texto, $resultado){
    echo "Entre " . $minimo . " y " . $maximo . ". " . $texto . ": " .
$resultado . "<br>";
}

```

```

<?php
include "Libreria.php";

$minimo = 1000;
$maximo = 9999;

//Cantidad de números primos que sus cifras están entre 3 y 7
$contar = 0;
for ($num = $minimo; $num<=$maximo; $num++){
    if (EsPrimo($num) && LaCifraMasBaja($num) >= 3 && LaCifraMasAlta($num)
<= 7) {
        echo $num . ", ";
        $contar = $contar + 1;
    }
}

echo "<br>";
Imprime($minimo, $maximo, "Cantidad de números primos que sus cifras están
entre 3 y 7", $contar);

//Imprime el resultado en el navegador

```



```
function Imprime($minimo, $maximo, $texto, $resultado){  
    echo "Entre " . $minimo . " y " . $maximo . ". " . $texto . ": " .  
$resultado . "<br>";  
}
```

03/Ej12.php

```
<?php  
include "Libreria.php";  
  
$minimo = 10;  
$maximo = 9999;  
  
//Cantidad de números primos y a su vez palíndromos  
$contar = 0;  
for ($num = $minimo; $num<=$maximo; $num++){  
    if (EsPrimo($num) && EsPalindromo($num)) {  
        echo $num . ", ";  
        $contar = $contar + 1;  
    }  
}  
  
echo "<br>";  
Imprime($minimo, $maximo, "Cantidad de números primos y a su vez  
palíndromos", $contar);  
  
//Imprime el resultado en el navegador  
function Imprime($minimo, $maximo, $texto, $resultado){  
    echo "Entre " . $minimo . " y " . $maximo . ". " . $texto . ": " .  
$resultado . "<br>";  
}
```

Método de Bisección

Ver: https://es.wikipedia.org/wiki/M%C3%A9todo_de_bisecci%C3%B3n

03/Ej13.php

```
<?php
$Xinicial = -3;
$Xfinal = 2;
$Ciclos = 100; //Número de veces que hará la operación de aproximación

//Verifica que entre $Xinicial y $Xfinal exista un corte
if (ecuacion($Xinicial) * ecuacion($Xfinal) > 0){
    echo "No hay punto de corte en los puntos dados";
}else{ //Si hay intersección, entonces llama a la función de Bisección
    $Xresultado = Biseccion($Xinicial, $Xfinal, $Ciclos);
    echo "Punto de corte en: " . $Xresultado;
}

//Retorna el punto X entre $Xinicial y $Xfinal que más se aproxime al
punto de corte
function Biseccion($Xinicial, $Xfinal, $Ciclos){
    $Xmedio = 0;
    if (ecuacion($Xinicial)==0) $Xmedio = $Xinicial;
    else if (ecuacion($Xfinal)==0) $Xmedio = $Xfinal;
    else
        for($scont=1; $scont<=$Ciclos; $scont++){
            $Xmedio = ($Xinicial + $Xfinal) / 2;
            if (ecuacion($Xmedio)==0) break;
            else if (ecuacion($Xinicial)*ecuacion($Xmedio) < 0) $Xfinal =
$Xmedio;
            else $Xinicial = $Xmedio;
        }
    return $Xmedio;
}

function ecuacion($x){ //En esta función se pone la ecuación
    return 4*$x*$x*$x-3*$x*$x-5*$x+2;
}
```

Ámbito (scope) de las variables y funciones

En PHP las variables son definidas dentro de las funciones o en el programa principal. No es permitido que una variable definida afuera, aparezca definida en el interior de una función.

03/Ej14.php

```
<?php
//Ámbito (scope) de una variable
$valor = 17;

//Imprime 17
echo "<br>Valor antes de la función es: " . $valor;

prueba(); //Llama a la función

//Imprime 17
echo "<br>Valor después de la función es: " . $valor;

function prueba(){
    //Mensaje de error, $valor no ha sido definida dentro de la función
    echo "<br>Valor dentro de la función es: " . $valor;
}
```

Valor antes de la función es: 17
Warning: Undefined variable \$valor in
D:\xampp\htdocs\prueba\03\Ej14.php on line 15

Valor dentro de la función es:
Valor después de la función es: 17

Ilustración 28: Ámbito (scope) de una variable

```
<?php
//Ámbito (scope) de una variable
$valor = 17;

//Imprime 17
echo "<br>Valor antes de la función es: " . $valor;

prueba(); //Llama a la función

//Imprime 17
echo "<br>Valor después de la función es: " . $valor;

function prueba(){
    $valor = 8906;
    echo "<br>Valor dentro de la función es: " . $valor;
}
```

```
Valor antes de la función es: 17
Valor dentro de la función es: 8906
Valor después de la función es: 17
```

Ilustración 29: Ámbito (scope) de una variable

```
<?php
//Ámbito (scope) de una variable
$valor = 17;

//Imprime 17
echo "<br>Valor antes de la función es: " . $valor;

prueba($valor); //Llama a la función

//Imprime 17
echo "<br>Valor después de la función es: " . $valor;

function prueba($valor){
    //Imprime 17
    echo "<br>Valor que llega a la función es: " . $valor;

    $valor = 8906;
    echo "<br>Valor después de la asignación en la función es: " . $valor;
}
```

```
Valor antes de la función es: 17
Valor que llega a la función es: 17
Valor después de la asignación en la función
es: 8906
Valor después de la función es: 17
```

Ilustración 30: Ámbito (scope) de una variable

Funciones recursivas

03/Ej15.php

```
<?php
/* Función recursiva
   Factorial(5) = 5 * Factorial(4)
   Es decir Factorial(N) = N * Factorial(N-1);
   y Factorial(0) = 1
*/
function factorial($numero) {
    if ($numero > 1 )
        return $numero * factorial($numero-1);
    else
        return 1;
}

echo "5! = " . factorial(5);
```

03/Ej16.php

```
<?php
// Función recursiva. Máximo común divisor con algoritmo de Euclides
function maximocomundivisor($numA, $numB) {
    if($numA < $numB) return maximocomundivisor($numB,$numA);
    if($numB == 0) return $numA;
    return maximocomundivisor($numB, $numA % $numB);
}

$numero = maximocomundivisor(1000, 750);
echo "Máximo común divisor de 1000 y 750 es " . $numero;
```

03/Ej17.php

```
<?php
// Función recursiva. Suma las cifras de un número
function sumacifras($num) {
    if ($num < 10) return $num;
    return $num%10 + sumacifras(floor($num/10));
}

echo "cifras suman: " . sumacifras(701);
```

```
<?php
//Convertir a binario
ConvierteBinario(70);

function ConvierteBinario($numero){
    if ($numero!=0){
        ConvierteBinario(floor($numero/2));
        echo $numero%2;
    }
}
```

```
<?php
//Potencia de un número
$valor = potencia(2, 7); //Retorna 2^7
echo $valor;

function potencia($numero, $elevado){
    if ($elevado==1) return $numero;
    return $numero * potencia($numero, $elevado - 1);
}
```

Uso de eval como evaluador de expresiones algebraicas

La instrucción eval permite evaluar instrucciones PHP almacenadas dentro de un string, ejemplo:

03/Ej20.php

```
<?php
//Ejecuta un comando PHP
eval("echo 'hola mundo';");
```

Es un comando en cierta manera peligroso, por lo que se desaconseja su uso.

03/Ej21.php

```
<?php
//Uso de eval como evaluador de expresiones algebraicas
$valor = eval("return 3+2*5;");
echo $valor;
```

03/Ej22.php

```
<?php
//Uso de eval como evaluador de expresiones algebraicas
$valor = eval("return sin(15)-cos(21);");
echo $valor;
```

03/Ej23.php

```
<?php
//Uso de eval como evaluador de expresiones algebraicas
$a=3;
$b=5;
$c=2;
$d=7;
$expresion = "return " . $a . "+" . $b . "-" . $c . "*" . $d . ";";
$valor = eval($expresion);
echo $valor;
```


Números aleatorios

En PHP se recomienda mucho hacer uso de `mt_rand()` para generar números aleatorios. Hace uso del algoritmo Mersenne Twister [10] .

03/Ej24.php

```
<?php
//Números aleatorios
$numero = mt_rand(); //Un número entero entre 0 y mt_getrandmax() incluido
echo $numero . "<br>";

echo "Máximo aleatorio: ";
echo mt_getrandmax();
```

03/Ej25.php

```
<?php
//Números aleatorios. Ciclo.
for ($num = 1; $num <= 100; $num++)
    echo mt_rand() . ", ";
```

Distribución Normal

03/Ej26.php

```
<?php
/* Variable aleatoria
   Distribución Normal. Generar una variable aleatoria con media M y
   desviación D
   variable = M + D * c
   donde c = cos(2*PI*r2)*raizcuadrada(-2*LogarimoNatural(r1))
   r1 y r2 son números aleatorios */
$M = 100;
$D = 7;
for ($cont=1; $cont<=100; $cont++){
    $r1 = mt_rand() / mt_getrandmax();
    $r2 = mt_rand() / mt_getrandmax();
    $c = cos(2*M_PI*$r2)*sqrt(-2*log($r1));
    $variable = $M + $D * $c;
    echo $variable . ", ";
```

Distribución Triangular

03/Ej27.php

```
<?php
/* Distribución Triangular. Generar una variable aleatoria con valor
mínimo A,
    valor más probable B y valor máximo C
    Si  $r < (B-A)/(C-A)$ 
        Variable =  $A + \text{raizcuadrada}(r * (C-A) * (B-A))$ 
    de lo contrario
        Variable =  $C - \text{raizcuadrada}((1-r) * (C-A) * (C-B))$ 
    */
$A = 150; $B = 190; $C = 230; $variable = 0;
for ($cont=1; $cont<=100; $cont++){
    $r = mt_rand() / mt_getrandmax();
    if ($r < ($B-$A)/($C-$A))
        $variable = $A + sqrt($r * ($C-$A) * ($B-$A));
    else
        $variable = $C - sqrt((1-$r) * ($C-$A) * ($C-$B));
    echo $variable . ", ";
}
```

Distribución Uniforme

03/Ej28.php

```
<?php
/* Variable aleatoria
Distribución uniforme. Generar un número entero entre A y B
variable =  $r * (B - A) + A$  */
$A = 10;
$B = 50;
for ($cont=1; $cont<=100; $cont++){
    $r = mt_rand() / mt_getrandmax();
    $variable = $r * ($B - $A) + $A;
    echo $variable . ", ";
}
```

Manejo de errores con números

is_nan

is_nan significa "is Not a Number". Si un resultado no es válido, retorna true como un arcoseno de un número mayor de 1 o una raíz cuadrada de un número negativo.

03/Ej29.php

```
<?php
//Chequea si un número es correcto
$numero = acos(15); //Arcoseno de 15 es erróneo
if (is_nan($numero))
    echo "No es un número";
else
    echo "Número es: " . $numero;
```

03/Ej30.php

```
<?php
if (is_nan(sqrt(3))) //Raíz cuadrada de un número positivo
    echo "1. Inválido <br>";
else
    echo "1. Válido <br>";

if (is_nan(sqrt(-3))) //Raíz cuadrada de un número negativo
    echo "2. Inválido <br>";
else
    echo "2. Válido <br>";
```

is_finite

03/Ej31.php

```
<?php
if (is_finite(sqrt(3))) //Raíz cuadrada de un número positivo
    echo "1. Válido <br>";
else
    echo "1. inválido <br>";

if (is_finite(sqrt(-3))) //Raíz cuadrada de un número negativo
    echo "2. Válido <br>";
else
    echo "2. inválido <br>";
```

Try...catch

Captura el error de división entre cero

03/Ej32.php

```
<?php
//Función que captura el error matemático
function Captura_Error($NumeroError, $Mensaje){
    throw new Exception($Mensaje);
}

//Esa función se matricula
set_error_handler('Captura_Error');

//Hace una división entre cero
try {
    $resultado = 4 / 0;
    echo $resultado;
}
catch(Exception $e ){
    echo "División entre cero<br>";
}
```

03/Ej33.php

```
<?php
try{
    $valorA = 15;
    $valorB = 0;
    $valorC = Divide($valorA, $valorB);
    echo "Resultado es: " . $valorC. "<br>";
} catch(Exception $unerror){
    echo 'Error capturado: '. $unerror->getMessage(). "<br>";
}
echo "Finaliza";

//Función que divide
function Divide($valA, $valB){
    if ($valB==0) throw new Exception("División entre cero");
    $valC = $valA / $valB;
    return $valC;
}
```

Arreglos unidimensionales o vectores

En PHP los arreglos unidimensionales siempre inician en la posición cero.

04/Ej01.php

```
<?php
//Arreglo unidimensional
$arreglo = [17,32,73,24,85,96,37,18,59,160];

echo "<br>Posición 0 es: " . $arreglo[0];
echo "<br>Posición 3 es: " . $arreglo[3];
```

Con `count($arreglo)` se obtiene el número de elementos del arreglo

04/Ej02.php

```
<?php
//Arreglo unidimensional
$arreglo = [17,32,73,24,85,96,37,18,59,160];

//Recorrido con un ciclo
for ($cont=0; $cont < count($arreglo); $cont++)
    echo $arreglo[$cont] . ", ";
```

Otra forma de recorrerlo

04/Ej03.php

```
<?php
//Arreglo unidimensional
$arreglo = [17,32,73,24,85,96,37,18,59,160];

//Otra forma de recorrerlo
foreach ($arreglo as $valor){
    echo $valor . ", ";
}
```

Uso de llaves

En PHP se pueden crear llaves en los arreglos unidimensionales

04/Ej04.php

```
<?php
//Arreglo unidimensional con llaves
$arreglo = [    "A" => "Tiburón",
                "B" => "Delfín",
                "C" => "Ballena",
                "D" => "Pez volador",
                "E" => "Anémona" ];

//Imprime el contenido con la llave "E"
echo $arreglo["E"];
```

04/Ej05.php

```
<?php
//Arreglo unidimensional con llaves de tipo número entero
$arreglo = [    7 => "Tiburón",
                12 => "Delfín",
                5 => "Ballena",
                23 => "Pez volador",
                45 => "Anémona" ];

echo $arreglo[7];
```

Adicionar elementos con array_push

04/Ej06.php

```
<?php
//Crea el arreglo
$lenguajes = array("C++");

//Adiciona elementos con array_push
array_push($lenguajes, "C#");
array_push($lenguajes, "Visual Basic .Net", "PHP", "JavaScript");
array_push($lenguajes, "Object Pascal");

//Recorre el arreglo
foreach ($lenguajes as $valor){
    echo $valor . ", ";
}
```

```
<?php
$valores = array();
array_push($valores, 15000);
array_push($valores, 18000);
array_push($valores, 34000);
array_push($valores, 17000);
array_push($valores, 8000);

//Operación con vectores: acumulado
$acumula = 0;
for ($cont=0; $cont < count($valores); $cont++){
    $acumula += $valores[$cont];
}

echo "Acumulado es: " . $acumula;
```

```
<?php
$valores = array();
array_push($valores, 15000);
array_push($valores, 18000);
array_push($valores, 34000);
array_push($valores, 17000);
array_push($valores, 8000);

//Operación con vectores: acumulado
$acumula = 0;
for ($cont=0; $cont < count($valores); $cont++){
    $acumula += $valores[$cont];
}

//Calcula el promedio
$promedio = $acumula / count($valores);
echo "Promedio es: " . $promedio;
```

```

<?php
$valores = array();
array_push($valores, 15000);
array_push($valores, 18000);
array_push($valores, 34000);
array_push($valores, 17000);
array_push($valores, 8000);

$mayor = $valores[0]; //Inicia con el primer elemento como el mayor
for ($cont=0; $cont < count($valores); $cont++){
    if ($valores[$cont] > $mayor) $mayor = $valores[$cont];
}

echo "Mayor es: " . $mayor;

```

Ordenar arreglos

```

<?php
$arreglo = [32,15.4,73,24,85,96,37,18,-59,16.8];

sort($arreglo); //Ordena de menor a mayor

foreach ($arreglo as $valor){
    echo $valor . ", ";
}

```

```

<?php
$arreglo = [32,17,-73,24,85,96,37,18,59,6];

sort($arreglo); //Ordena de menor a mayor
$nuevo = array_reverse($arreglo); //Invierte el arreglo

foreach ($nuevo as $valor){
    echo $valor . ", ";
}

```



```

<?php
//Crea el arreglo
$lenguajes = array("C++");

//Adiciona elementos con array_push
array_push($lenguajes, "C#");
array_push($lenguajes, "Visual Basic .Net", "PHP", "JavaScript");
array_push($lenguajes, "Object Pascal");

sort($lenguajes);

for ($cont=0; $cont < count($lenguajes); $cont++){
    echo $lenguajes[$cont] . ", ";
}

```

Fusionar, extraer y retirar elementos de un arreglo

```

<?php
$valorA = [2, 4, 6, 1, 3, 7];
$valorB = [8, 10, 12, 32, 42, 52];

//Une los dos arreglos en un tercero
$nuevo = array_merge($valorA, $valorB);
Imprime("Fusionado: ", $nuevo);

//Extrae los 4 primeros valores de un arreglo y los guarda en un tercero
$nuevo = array_slice($valorA, 0, 4);
Imprime("Parcial: ", $nuevo);

//Borra los dos primeros valores de un arreglo
array_splice($valorB, 0, 2);
Imprime("Cortado: ", $valorB);

function Imprime($texto, $arreglo){
    echo "<br>" . $texto;
    for ($cont=0; $cont < count($arreglo); $cont++){
        echo $arreglo[$cont] . ", ";
    }
}

```

```
<?php
//Dado un arreglo extraer los valores únicos
$numeros = [5,8,1,3,3,2,7,8,1,9,9,1,1,2,4,7,2,3,8,4,3,1,1,3,2,8,8,7];
$extrae = array();

for ($cont=0; $cont<count($numeros); $cont++)
    if ( BuscarEnArreglo( $numeros[$cont], $extrae) == false )
        array_push($extrae, $numeros[$cont] );

print_r($extrae); //Muestra el arreglo

//Muestra elemento a elemento del arreglo
echo "<br>";
for ($cont=0; $cont < count($extrae); $cont++){
    echo $extrae[$cont] . ", ";
}

function BuscarEnArreglo($num, $arreglo){ //Retorna true si encuentra num
en arreglo
    for ($cont=0; $cont<count($arreglo); $cont++)
        if ($num==$arreglo[$cont])
            return true;
    return false;
}
```

```
<?php
//Función genérica para hallar menor y mayor valor
$pagar = [1120, 6040, 1570, 5205, 3147, 7361, 166, 1002, 1572, 6567];

$menor = BuscaMenorValor($pagar);
$mayor = BuscaMayorValor($pagar);
echo "Menor cuantía es: " . $menor;
echo "<br>Mayor cuantía es: " . $mayor;

function BuscaMenorValor($arreglo){ //Función genérica
    $minimo = $arreglo[0];
    for($cont=1; $cont < count($arreglo); $cont++)
        if ($arreglo[$cont] < $minimo) $minimo = $arreglo[$cont];
    return $minimo;
}

function BuscaMayorValor($arreglo){ //Función genérica
    $maximo = $arreglo[0];
    for($cont=1; $cont < count($arreglo); $cont++)
        if ($arreglo[$cont] > $maximo) $maximo = $arreglo[$cont];
    return $maximo;
}
```

```
<?php
// Separe los elementos pares e impares de una lista, y retorne ambas
listas
$numeros = [10, 13, 20, 24, 23, 2, 23, 29, 5, 19, 8, 6, 16, 2, 6, 27];
$pares = retornaPares($numeros);
$impares = retornaImpares($numeros);
echo "Pares: ";
print_r($pares);
echo "<br>Impares: ";
print_r($impares);

function retornaPares($valores) {
    $par = array();
    for ($cont=0; $cont<count($valores); $cont++)
        if ($valores[$cont]%2==0)
            array_push($par, $valores[$cont]);
    return $par;
}

function retornaImpares($valores) {
    $impar = array();
    for ($cont=0; $cont<count($valores); $cont++)
        if ($valores[$cont]%2!=0)
            array_push($impar, $valores[$cont]);
    return $impar;
}
```

Implementación de algoritmos de ordenación

¡OJO! Por defecto los arreglos se pasan por valor en PHP, luego para enviarlo por referencia en el parámetro de la función se hace uso de &

Algoritmo de burbuja

04/Ej17.php

```
<?php
$letras = ['r', 'v', 'e', 'g', 't', 'u', 'd', 'f', 'f', 'y', 'v'];
Imprime("Original: ", $letras);
OrdenarBurbuja($letras);
Imprime("Ordenado: ", $letras);

//Ordenación por el método de burbuja, el arreglo se recibe por referencia
function OrdenarBurbuja(& $arreglo) {
    for ($i=0; $i<count($arreglo)-1; $i++)
        for ($j=0; $j<count($arreglo)-$i-1; $j++)
            if ($arreglo[$j]>$arreglo[$j+1]){
                $aux = $arreglo[$j];
                $arreglo[$j] = $arreglo[$j+1];
                $arreglo[$j+1] = $aux;
            }
}

function Imprime($texto, $arreglo) {
    echo "<br>" . $texto;
    for ($cont=0; $cont < count($arreglo); $cont++){
        echo $arreglo[$cont] . ", ";
    }
}
```

04/Ej18.php

```
<?php
$valores = [-363, -1793, -2601, -2541, 6376, 6053, 503, 1601, -2994, 6480,
998];
Imprime("Original: ", $valores);
OrdenarBurbuja($valores);
Imprime("Ordenado: ", $valores);

//Ordenación por el método de burbuja, el arreglo se recibe por referencia
function OrdenarBurbuja(& $arreglo) {
    for ($i=0; $i<count($arreglo)-1; $i++)
        for ($j=0; $j<count($arreglo)-$i-1; $j++)
            if ($arreglo[$j]>$arreglo[$j+1]){
                $aux = $arreglo[$j];
                $arreglo[$j] = $arreglo[$j+1];
                $arreglo[$j+1] = $aux;
            }
}
```

```

}

function Imprime($texto, $arreglo){
    echo "<br>" . $texto;
    for ($cont=0; $cont < count($arreglo); $cont++){
        echo $arreglo[$cont] . ", ";
    }
}

```

Algoritmo de ordenación por selección

04/Ej19.php

```

<?php
$valores = [-363, -1793, -2601, -2541, 6376, 6053, 503, 1601, -2994, 6480,
998];
Imprime("Original: ", $valores);
OrdenarSeleccion($valores);
Imprime("Ordenado: ", $valores);

//Ordenación por selección
function OrdenarSeleccion(& $arreglo){
    for($i=0; $i<count($arreglo)-1; $i++){
        $tmp = $i;
        for ($j=$i+1; $j<count($arreglo); $j++)
            if ($arreglo[$j] < $arreglo[$tmp])
                $tmp = $j;

        $temporal = $arreglo[$tmp];
        $arreglo[$tmp] = $arreglo[$i];
        $arreglo[$i] = $temporal;
    }
}

function Imprime($texto, $arreglo){
    echo "<br>" . $texto;
    for ($cont=0; $cont < count($arreglo); $cont++){
        echo $arreglo[$cont] . ", ";
    }
}

```

Algoritmo de ordenación por inserción

04/Ej20.php

```
<?php
$valores = [-363, -1793, -2601, -2541, 6376, 6053, 503, 1601, -2994, 6480,
998];
Imprime("Original: ", $valores);
OrdenarInsercion($valores);
Imprime("Ordenado: ", $valores);

//Algoritmo de ordenación por inserción
function OrdenarInsercion(& $arreglo){
    for ($i = 0; $i < count($arreglo); ++$i) {
        $tmp = $arreglo[$i];
        for ($j = $i - 1; $j >= 0 && $arreglo[$j] > $tmp; --$j)
            $arreglo[$j + 1] = $arreglo[$j];
        $arreglo[$j + 1] = $tmp;
    }
}

function Imprime($texto, $arreglo){
    echo "<br>" . $texto;
    for ($cont=0; $cont < count($arreglo); $cont++){
        echo $arreglo[$cont] . ", ";
    }
}
```

Algoritmo de ordenación QuickSort

04/Ej21.php

```
<?php
$valores = [-363, -1793, -2601, -2541, 6376, 6053, 503, 1601, -2994, 6480,
998];
Imprime("Original: ", $valores);
$nuevo = quicksort($valores);
Imprime("Ordenado: ", $nuevo);

function quicksort($arreglo){
    if(count($arreglo) == 0) return $arreglo;
    $izq = $der = array();
    $pivote = $arreglo[0];

    for ($i = 1; $i < count($arreglo); $i++)
        if ($arreglo[$i] < $pivote)
            $izq[] = $arreglo[$i];
        else
```

```

        $der[] = $arreglo[$i];

    return array_merge(quicksort($izq), array(0=>$pivote), quicksort($der));
}

function Imprime($texto, $arreglo) {
    echo "<br>" . $texto;
    for ($cont=0; $cont < count($arreglo); $cont++){
        echo $arreglo[$cont] . ", ";
    }
}

```

Algoritmo de ordenación MergeSort

04/Ej22.php

```

<?php
$valores = [8,7,6,5,4,3,2,1];
Imprime("Original: ", $valores); echo "<br>";
$nuevo = mergesort($valores);
Imprime("Ordenado: ", $nuevo);

function mergesort($arreglo) {
    if(count($arreglo) <= 1) return $arreglo;
    $mitad = floor(count($arreglo) / 2);
    $izquierdo = array_slice($arreglo, 0, $mitad);
    $derecho = array_slice($arreglo, $mitad, count($arreglo));
    return merge(mergesort($izquierdo), mergesort($derecho));
}

function merge($izquierdo, $derecho) {
    $ordenado = array();
    while (count($izquierdo) > 0 && count($derecho) > 0) {
        $b = $izquierdo[0] <= $derecho[0];
        if($b) $ordenado[]=$izquierdo[0]; else $ordenado[]=$derecho[0];
        if($b) array_splice($izquierdo, 0, 1); else
array_splice($derecho, 0, 1);
    }
    return array_merge($ordenado, $izquierdo, $derecho);
}

function Imprime($texto, $arreglo) {
    echo "<br>" . $texto;
    for ($cont=0; $cont < count($arreglo); $cont++){
        echo $arreglo[$cont] . ", ";
    }
}

```



```
<?php
// Establecer la zona horaria predeterminada a usar.
date_default_timezone_set('America/Bogota');

echo "Año es: " . date("Y") . "<br>";
echo "Mes es: " . date("m") . "<br>";
echo "Día es: " . date("d") . "<br>";
echo "Hora es: " . date("h") . "<br>";
echo "Minuto es: " . date("i") . "<br>";
echo "Segundo es: " . date("s") . "<br>";
```

Manejo de cadenas o strings

05/Ej01.php

```
<?php
//Concatenar cadenas
$cadena1 = "Dragón";
$cadena2 = " de ";
$cadena3 = "Komodo";
$cadena4 = $cadena1 . $cadena2 . $cadena3;
echo $cadena4;
```

05/Ej02.php

```
<?php
//Concatenar cadenas y valores numéricos
$cadena1 = 3;
$cadena2 = "abcde";
$cadena3 = 5;
$cadena4 = $cadena1 . $cadena2 . $cadena3 . "efg";
echo $cadena4;
```

05/Ej03.php

```
<?php
/* Extraer una letra de una cadena. Las cadenas se comportan como arreglos
unidimensionales, el primer lelemento está en la posición 0 */
$cadena = "Esta es una prueba";
$letra = $cadena[0]; //Trae la primera letra
echo $letra;
```

05/Ej04.php

```
<?php
//Longitud de una cadena (incluye los espacios)
$cadena = " Esta es una prueba ";
$longitud = strlen($cadena);
echo $longitud;
```

05/Ej05.php

```
<?php
//Convertir a mayúsculas y minúsculas
$cadena = "ÉsTa Es UNa PrUEBa dE CAMBIO de MAYúscULas y MINúscULas";
$mayuscula = strtoupper($cadena); //Convierte a mayúsculas
$minuscula = strtolower($cadena); //Convierte a minúsculas
echo $mayuscula;
echo "<br>";
echo $minuscula;
```

Se observa que las vocales tildadas **no** son convertidas a mayúsculas o minúsculas de forma satisfactoria.

```
<?php
//Comparación de cadenas
$cadena1 = "FraseA";
$cadena2 = "FraseA";
if ($cadena1 === $cadena2) //Comparación estricta al usar ===
    echo "iguales";
else
    echo "diferentes";
```

Vocales tildadas, Ñs y otros caracteres del idioma español

Código con caracteres pertenecientes al alfabeto inglés

```
<?php
//Uso de caracteres de habla inglesa
$cadena = "aeiou";
echo "Cadena: [" . $cadena . "]<br>";
echo "Longitud calculada: " . strlen($cadena) . "<br>";
echo "Longitud codificada: " . mb_strlen($cadena, 'UTF-8') . "<br>";

echo "Recorriendo: ";
for ($cont=0; $cont<strlen($cadena); $cont++){
    $letra = $cadena[$cont];
    echo "[" . $letra . "] ";
}
```

Ahora se varía el código con letras propias del idioma español

```
<?php
//Uso de caracteres de habla española
$cadena = "áéíóú";
echo "Cadena: [" . $cadena . "]<br>";
echo "Longitud calculada: " . strlen($cadena) . "<br>";
echo "Longitud codificada: " . mb_strlen($cadena, 'UTF-8') . "<br>";

echo "Recorriendo: ";
for ($cont=0; $cont<strlen($cadena); $cont++){
    $letra = $cadena[$cont];
    echo "[" . $letra . "] ";
}
```

Este es el resultado:



Ilustración 31: Comportamiento con letras del idioma español

Como puede observarse el comportamiento es completamente distinto y lo mismo pasará con caracteres propios del idioma español ¿Qué hacer? El término es “multibyte character” y hay que hacer uso de funciones propias para tratarlo. Este sería la forma de hacer el programa:

05/Ej09.php

```
<?php
//Uso de caracteres de habla española
$cadena = "áéíóú";
echo "Cadena: [" . $cadena . "<br>";
echo "Longitud codificada: " . mb_strlen($cadena, 'UTF-8') . "<br>";

echo "Recorriendo: ";
for ($cont=0; $cont<mb_strlen($cadena, 'UTF-8'); $cont++){
    $letra = mb_substr($cadena, $cont, 1, 'UTF-8');
    echo "[" . $letra . "<br>";
}
```

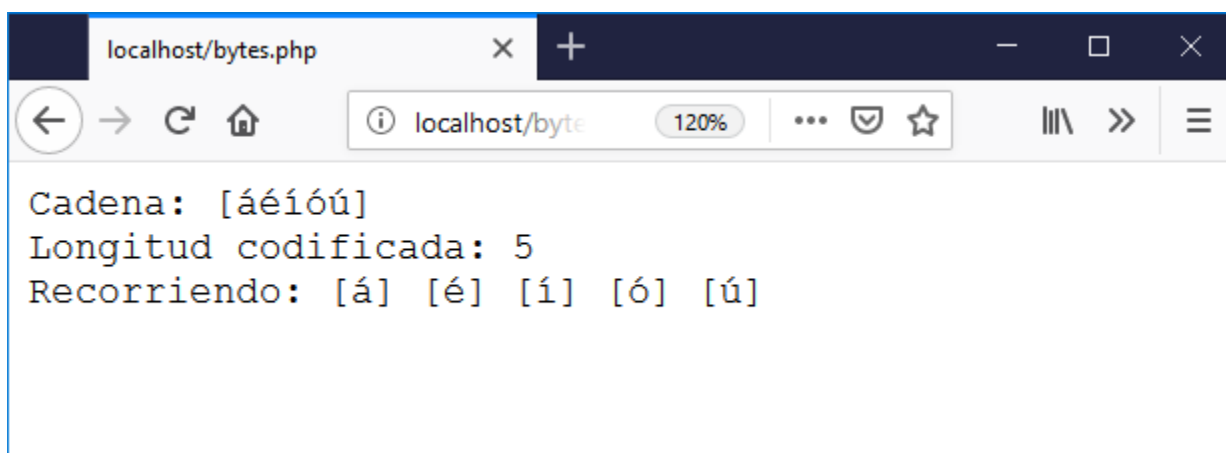


Ilustración 32: Recorriendo la cadena con caracteres multibyte

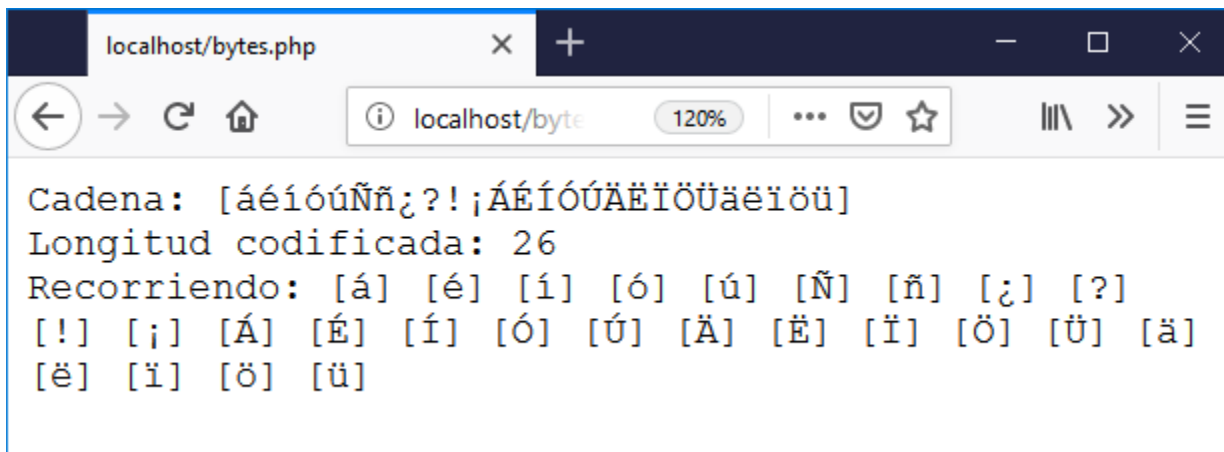
Se puede crecer el programa para que cubra otras letras y símbolos propios del español

05/Ej10.php

```
<?php
//Uso de caracteres de habla española
$cadena = "áéíóúÑñ¿?!;ÁÉÍÓÚÄËÏÖÜäëïöü";
echo "Cadena: [" . $cadena . "]<br>";
echo "Longitud codificada: " . mb_strlen($cadena, 'UTF-8') . "<br>";

echo "Recorriendo: ";
for ($cont=0; $cont<mb_strlen($cadena, 'UTF-8'); $cont++){
    $letra = mb_substr($cadena, $cont, 1, 'UTF-8');
    echo "[" . $letra . "] ";
}
```

Y el resultado es el esperado:



```
Cadena: [áéíóúÑñ¿?!;ÁÉÍÓÚÄËÏÖÜäëïöü]
Longitud codificada: 26
Recorriendo: [á] [é] [í] [ó] [ú] [Ñ] [ñ] [¿] [?]
[!] [;] [Á] [É] [Í] [Ó] [Ú] [Ä] [Ë] [Ï] [Ö] [Ü] [ä]
[ë] [ï] [ö] [ü]
```

Ilustración 33: Recorriendo la cadena con caracteres multibyte

Ejemplos de programas usando cadenas

Invertir Cadena

06/Ej01.php

```
<?php
//Invierte la cadena
$cadena = "Rafael áéíóúÑñ¿?!;ÁÉÍÓÚÄËÏÖÜäëïöü";
$nueva = "";
for ($cont=0; $cont<mb_strlen($cadena, 'UTF-8'); $cont++){
    $letra = mb_substr($cadena, $cont, 1, 'UTF-8');
    $nueva = $letra . $nueva;
}

echo "Original: " . $cadena . "<br>";
echo "Invierte: " . $nueva . "<br>";
```

Quitar los espacios de una cadena

06/Ej02.php

```
<?php
//Quita espacios
$cadena = "   Rafael   Moreno   áéíóúÑñ   ¿?   !; ÁÉÍÓÚÄËÏÖÜäëïöü   Parra ";
$nueva = "";
for ($cont=0; $cont<mb_strlen($cadena, 'UTF-8'); $cont++){
    $letra = mb_substr($cadena, $cont, 1, 'UTF-8');
    if ($letra != ' ')
        $nueva .= $letra;
}

echo "Original: " . $cadena . "<br>";
echo "Sin espacios: " . $nueva . "<br>";
```

Quitar las vocales de una cadena

06/Ej03.php

```
<?php
//Quita vocales
$cadena = "    Rafael  Moreno  áéíóúÑñ  ¿?  !; ÁÉÍÓÚÄËÏÖÜäëïöü  Parra ";
$nueva = "";
for ($cont=0; $cont<mb_strlen($cadena, 'UTF-8'); $cont++){
    $letra = mb_substr($cadena, $cont, 1, 'UTF-8');
    if (!EsVocal($letra)) $nueva .= $letra;
}

echo "Original: " . $cadena . "<br>";
echo "Sin vocales: " . $nueva . "<br>";

//Retorna true si el caracter es una vocal
function EsVocal($caracter){
    $vocales = "aeiouAEIOUÁÉÍÓÚáéíóúäëïöüÄËÏÖÜ";
    for ($cont=0; $cont < mb_strlen($vocales, 'UTF-8'); $cont++){
        $letra = mb_substr($vocales, $cont, 1, 'UTF-8');
        if ($caracter == $letra)
            return true;
    }
    return false;
}
```

Quitar caracteres no permitidos

06/Ej04.php

```
<?php
//Retirar caracteres no permitidos de una cadena
$cadena = "Esto es una <b><big>prueba</big></b>";
$nueva = "";
for ($cont=0; $cont<mb_strlen($cadena, 'UTF-8'); $cont++){
    $letra = mb_substr($cadena, $cont, 1, 'UTF-8');
    if (EsPermitido($letra)) $nueva .= $letra;
}

echo "Original: " . $cadena . "<br>";
echo "Con los caracteres permitidos: " . $nueva . "<br>";

//Retorna true si el caracter está permitido
function EsPermitido($caracter){
    $permite = "ÁÉÍÓÚáéíóúäëïöü";
    $permite .= "abcdefghijklmnopqrstuvwxyz";
    $permite .= "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    $permite .= " 1234567890";
}
```

```

for ($cont=0; $cont < mb_strlen($permite, 'UTF-8'); $cont++){
    $letra = mb_substr($permite, $cont, 1, 'UTF-8');
    if ($caracter == $letra)
        return true;
}
return false;
}

```

Un sencillo cifrador / descifrador

06/Ej05.php

```

<?php
//Cifrador sencillo
$cadena = "Esta es una prueba de cifrado";
$cifrada = Cifrar($cadena);
echo "Original: " . $cadena . "<br>";
echo "Cifrado: " . $cifrada . "<br>";

$descifra = DesCifrar($cifrada);
echo "Descifrado: " . $descifra . "<br>";

//Cifra la cadena moviendo un caracter hacia adelante
function Cifrar($cadena){
    $cifrada = "";
    for ($cont=0; $cont<mb_strlen($cadena, 'UTF-8'); $cont++){
        $num = ord(mb_substr($cadena, $cont, 1, 'UTF-8')); //Trae el valor
        ASCII de la letra
        $num++; //Incrementa en uno ese valor
        $cifrada .= chr($num); //Convierte el valor a su respectivo
        caracter
    }
    return $cifrada;
}

//DesCifra la cadena cifrada moviendo un caracter hacia atrás
function DesCifrar($cadena){
    $descifrada = "";
    for ($cont=0; $cont<mb_strlen($cadena, 'UTF-8'); $cont++){
        $num = ord(mb_substr($cadena, $cont, 1, 'UTF-8')); //Trae el valor
        ASCII de la letra
        $num--; //Incrementa en uno ese valor
        $descifrada .= chr($num); //Convierte el valor a su respectivo
        caracter
    }
    return $descifrada;
}

```



```
<?php
//Cifrador sencillo con clave de cifrado
$cadena = "Esta es una prueba de cifrado con clave";
$clave = "sesamo";

$scifrada = Cifrar($cadena, $clave);
echo "Original: " . $cadena . "<br>";
echo "Cifrado: " . $scifrada . "<br>";

$descifra = DesCifrar($scifrada, $clave);
echo "Descifrado: " . $descifra . "<br>";

function Cifrar($cadena, $clave){
    $scifrada = "";
    $clavecont = 0;
    for ($scont=0; $scont<mb_strlen($cadena, 'UTF-8'); $scont++){
        //Cada letra de la clave genera un desplazamiento
        $desplaza = ord(mb_substr($clave, $clavecont, 1, 'UTF-8'))%20;
        $clavecont++;
        if ($clavecont >= mb_strlen($clave, 'UTF-8')) $clavecont = 0; //Si
se llega al final de la clave
        $num = ord(mb_substr($cadena, $scont, 1, 'UTF-8')); //Trae el valor
ASCII de la letra
        $num += $desplaza;
        $scifrada .= chr($num);
    }
    return $scifrada;
}

function DesCifrar($cadena, $clave){
    $scifrada = "";
    $clavecont = 0;
    for ($scont=0; $scont<mb_strlen($cadena, 'UTF-8'); $scont++){
        //Cada letra de la clave genera un desplazamiento
        $desplaza = ord(mb_substr($clave, $clavecont, 1, 'UTF-8'))%20;
        $clavecont++;
        if ($clavecont >= mb_strlen($clave, 'UTF-8')) $clavecont = 0; //Si
se llega al final de la clave
        $num = ord(mb_substr($cadena, $scont, 1, 'UTF-8')); //Trae el valor
ASCII de la letra
        $num -= $desplaza;
        $scifrada .= chr($num);
    }
    return $scifrada;
}
```

Sumar largos números enteros almacenados en cadenas

06/Ej07.php

```
<?php
//Suma dos números enteros positivos almacenados en cadenas. Como son solo
números dentro de strings
//se hace uso de las funciones genéricas de cadena (no es necesario UTF-8)
$numeroA = "395763487398655335"; //Primer número
$numeroB = "8888888888888888888"; //Segundo número
$resultado = ""; //Guarda el resultado
$ultimaPosA = strlen($numeroA)-1; //Posición del último dígito del primer
número
$ultimaPosB = strlen($numeroB)-1;
$llevar = false; //Se activa si la suma de dígitos es 10 o mas
while($ultimaPosA>=0 || $ultimaPosB>=0){ //Suma del dígito menos
significativo al más significativo
    $suma = 0; //Suma de dígitos
    if ($ultimaPosA>=0 && $ultimaPosB>=0){ //Si ambos dígitos existen en
esas posiciones
        $suma = intval($numeroA[$ultimaPosA]) +
intval($numeroB[$ultimaPosB]);
    }
    else if ($ultimaPosA>=0){ //Si sólo existe dígito en primer número
        $suma = intval($numeroA[$ultimaPosA]);
    }
    else { //solo existe dígito en segundo numero
        $suma = intval($numeroB[$ultimaPosB]);
    }
    if ($llevar==true) $suma++; //Si llevaba aumenta en 1 la suma
    $llevar = false; //La bandera de llevar se apaga
    if ($suma>=10){ $llevar=true; $suma-=10; } //Si la suma es 10 o más,
entonces lleva, le quita 10
    $resultado = $suma . $resultado; //Agrega al inicio de la cadena
    $ultimaPosA--; //Va hacia la izquierda del primer número
    $ultimaPosB--; //Va hacia la izquierda del segundo número
}
if ($llevar==true) $resultado = "1" . $resultado; //Si la suma de los
primeros dígitos es 10 o más
echo $numeroA . " + <br>" . $numeroB . "<br>-----<br>"
. $resultado;
```

Arreglos bidimensionales

Definiendo arreglos bidimensionales

07/Ej01.php

```
<?php
//Un arreglo bidimensional de cuatro filas y tres columnas
$tablero = array(
    array(902, 178, 347),
    array(782, 490, 111),
    array(213, 887, 147),
    array(553, 673, 821)
);

//Fila, columna
echo 'Posición 0,0 es: ' . $tablero[0][0] . '<br>';
echo 'Posición 2,1 es: ' . $tablero[2][1] . '<br>';
echo 'Posición 3,0 es: ' . $tablero[3][0] . '<br>';
```

Recorriendo el arreglo bidimensional

07/Ej02.php

```
<?php
$tablero = array(
    array(902, 178, 347),
    array(782, 490, 111),
    array(213, 887, 147),
    array(553, 673, 821)
);

//Recorrido es:
for ($fila=0; $fila<count($tablero); $fila++){
    echo "<br>";
    for ($columna=0; $columna<count($tablero[$fila]); $columna++){
        echo $tablero[$fila][$columna] . ', ';
    }
}
```

Un arreglo bidimensional no necesariamente es rectangular, puede haber filas con diferente número de columnas

```

<?php
//Es bidimensional pero con diferente número de columnas
$tablero = array(
    array(902, 178, 347),
    array(782, 490, 111, 512, 775, 231),
    array(213, 887, 147),
    array(553, 673, 821, 489)
);

//Recorrido es:
for ($fila=0; $fila<count($tablero); $fila++){
    echo "<br>";
    for ($columna=0; $columna<count($tablero[$fila]); $columna++)
        echo $tablero[$fila][$columna] . ', ';
}

```

```

<?php
//Crea el arreglo bidimensional
$tabla = array(); //Crea las filas

//Crea las columnas
for ($fila=0; $fila < 8; $fila++)
    $tabla[$fila] = array();

echo "Filas: " . count($tabla) . "<br>";

//Llena el arreglo bidimensional de puntos
for ($fila=0; $fila < count($tabla); $fila++)
    for ($columna=0; $columna < 20; $columna++)
        $tabla[$fila][$columna]='.';

echo "Columnas: " . count($tabla[0]) . "<br>";

//Pone una X en una posición al azar
$posF = rand(0, 7);
$posC = rand(0, 19);
$tabla[$posF][$posC] = 'X';

//Imprime la tabla
for ($fila=0; $fila < count($tabla); $fila++){
    echo "<br>";
    for ($columna=0; $columna < count($tabla[$fila]); $columna++)
        echo $tabla[$fila][$columna];
}

```

```

<?php
//Llama a función que genera arreglo bidimensional
$tabla = GenerarArregloBidimensional(8);

echo "Filas: " . count($tabla) . "<br>";

//Llena el arreglo bidimensional de puntos
for ($fila=0; $fila < count($tabla); $fila++)
    for ($columna=0; $columna < 20; $columna++)
        $tabla[$fila][$columna]='.';

echo "Columnas: " . count($tabla[0]) . "<br>";

//Pone una X en una posición al azar
$posF = rand(0, 7);
$posC = rand(0, 19);
$tabla[$posF][$posC] = 'X';

//Imprime la tabla
for ($fila=0; $fila < count($tabla); $fila++){
    echo "<br>";
    for ($columna=0; $columna < count($tabla[$fila]); $columna++)
        echo $tabla[$fila][$columna];
}

//Función genérica para crear arreglo bidimensional
function GenerarArregloBidimensional($totalfilas) {
    $arreglo = array();
    for ($fila=0; $fila < $totalfilas; $fila++) $arreglo[$fila] = array();
    return $arreglo;
}

```

```

<?php
//Llama a función que genera arreglo bidimensional
$tabla = GeneraArregloBidimensional(10, 15, '.');

//Imprime la tabla
ImprimeArregloBidimensional($tabla);

/* Función genérica para crear arreglos bidimensionales
   con el número de filas y columnas dado y llena cada celda
   de un valor */
function GeneraArregloBidimensional($filas, $columnas, $valor) {
    $arreglo = array();
    for ($fila=0; $fila < $filas; $fila++) {
        $arreglo[$fila] = array();
        for ($columna=0; $columna<$columnas; $columna++)
            $arreglo[$fila][$columna]=$valor;
    }
    return $arreglo;
}

//Función genérica para imprimir un arreglo bidimensional
function ImprimeArregloBidimensional($arreglo){
    for ($fila=0; $fila < count($arreglo); $fila++){
        echo "<br>";
        for ($columna=0; $columna < count($arreglo[$fila]); $columna++)
            echo $arreglo[$fila][$columna];
    }
}

```

Ejemplos de programas usando arreglos bidimensionales

Poner una torre al azar en un tablero de ajedrez y mostrar su ataque

07/Ej07.php

```
<?php
/* Crear un tablero de ajedrez de 8*8, poner en una posición al
   azar una Torre y mostrar su ataque */
$tabla = GeneraArregloBidimensional(8, 8, '.');
$tabla = PoneTorreAzar($tabla);
ImprimeArregloBidimensional($tabla);

//Pone una torre al azar y muestra su ataque
function PoneTorreAzar($arreglo){
    $filaAzar = rand(0, count($arreglo)-1);
    $coluAzar = rand(0, count($arreglo[0])-1);
    for ($fil=0; $fil<count($arreglo); $fil++) $arreglo[$fil][$coluAzar] =
'x';
    for ($col=0; $col<count($arreglo[0]); $col++) $arreglo[$filaAzar][$col]
= 'x';
    $arreglo[$filaAzar][$coluAzar] = 'T';
    return $arreglo;
}

/* Función genérica para crear arreglos bidimensionales
   con el número de filas y columnas dado y llena cada celda
   de un valor */
function GeneraArregloBidimensional($filas, $columnas, $valor) {
    $arreglo = array();
    for ($fila=0; $fila < $filas; $fila++) {
        $arreglo[$fila] = array();
        for ($columna=0; $columna<$columnas; $columna++)
            $arreglo[$fila][$columna]=$valor;
    }
    return $arreglo;
}

//Función genérica para imprimir un arreglo bidimensional
function ImprimeArregloBidimensional($arreglo){
    for ($fila=0; $fila < count($arreglo); $fila++){
        echo "<br>";
        for ($columna=0; $columna < count($arreglo[$fila]); $columna++)
            echo $arreglo[$fila][$columna];
    }
}
```

Poner un Rey al azar en un tablero de ajedrez y mostrar su ataque

07/Ej08.php

```
<?php
/* Crear un tablero de ajedrez de 8*8, poner en una posición al
   azar un Rey y mostrar su ataque */
$tabla = GeneraArregloBidimensional(8, 8, '.');
$tabla = PoneReyAzar($tabla);
ImprimeArregloBidimensional($tabla);

//Pone un rey al azar y muestra su ataque
function PoneReyAzar($arreglo) {
    $filaAzar = rand(0, count($arreglo)-1);
    $coluAzar = rand(0, count($arreglo[0])-1);
    if ($filaAzar>0) $arreglo[$filaAzar-1][$coluAzar]='x';
    if ($filaAzar>0 && $coluAzar>0) $arreglo[$filaAzar-1][$coluAzar-1]='x';
    if ($coluAzar>0) $arreglo[$filaAzar][$coluAzar-1]='x';
    if ($coluAzar<count($arreglo[0])-1)
$arreglo[$filaAzar][$coluAzar+1]='x';
    if ($filaAzar>0 && $coluAzar<count($arreglo[0])-1) $arreglo[$filaAzar-
1][$coluAzar+1]='x';
    if ($filaAzar<count($arreglo)-1 && $coluAzar<count($arreglo[0])-1)
$arreglo[$filaAzar+1][$coluAzar+1]='x';
    if ($filaAzar<count($arreglo)-1) $arreglo[$filaAzar+1][$coluAzar]='x';
    if ($filaAzar<count($arreglo)-1 && $coluAzar>0)
$arreglo[$filaAzar+1][$coluAzar-1]='x';
    $arreglo[$filaAzar][$coluAzar] = 'R';
    return $arreglo;
}

/* Función genérica para crear $arreglos bidimensionales
   con el número de filas y columnas dado y llena cada celda
   de un valor */
function GeneraArregloBidimensional($filas, $columnas, $valor) {
    $arreglo = array();
    for ($fila=0; $fila < $filas; $fila++) {
        $arreglo[$fila] = array();
        for ($columna=0; $columna<$columnas; $columna++)
            $arreglo[$fila][$columna]=$valor;
    }
    return $arreglo;
}

//Función genérica para imprimir un $arreglo bidimensional
function ImprimeArregloBidimensional($arreglo){
    for ($fila=0; $fila < count($arreglo); $fila++){
        echo "<br>";
        for ($columna=0; $columna < count($arreglo[$fila]); $columna++)
            echo $arreglo[$fila][$columna];
    }
}
```



```
}
```

Poner un alfil al azar en un tablero de ajedrez y mostrar su ataque

07/Ej09.php

```
<?php
/* Crear un tablero de ajedrez de 8*8, poner en una posición al
   azar un alfil y mostrar su ataque */
$tabla = GeneraArregloBidimensional(8, 8, '.');
$tabla = PoneAlfilAzar($tabla);
ImprimeArregloBidimensional($tabla);

//Pone un rey al azar y muestra su ataque
function PoneAlfilAzar($arreglo){
    $filaAzar = rand(0, count($arreglo)-1);
    $coluAzar = rand(0, count($arreglo[0])-1);
    $tmpFil=$filaAzar;
    $tmpCol=$coluAzar;
    while($tmpFil>=0 && $tmpCol>=0) { $arreglo[$tmpFil][$tmpCol]='x';
    $tmpFil--; $tmpCol--; }
    $tmpFil=$filaAzar; $tmpCol=$coluAzar;
    while($tmpFil>=0 && $tmpCol<8) { $arreglo[$tmpFil][$tmpCol]='x';
    $tmpFil--; $tmpCol++; }
    $tmpFil=$filaAzar; $tmpCol=$coluAzar;
    while($tmpFil<8 && $tmpCol<8) { $arreglo[$tmpFil][$tmpCol]='x';
    $tmpFil++; $tmpCol++; }
    $tmpFil=$filaAzar; $tmpCol=$coluAzar;
    while($tmpFil<8 && $tmpCol>=0) { $arreglo[$tmpFil][$tmpCol]='x';
    $tmpFil++; $tmpCol--; }
    $arreglo[$filaAzar][$coluAzar] = 'A';
    return $arreglo;
}

/* Función genérica para crear $arreglos bidimensionales
   con el número de filas y columnas dado y llena cada celda
   de un valor */
function GeneraArregloBidimensional($filas, $columnas, $valor) {
    $arreglo = array();
    for ($fila=0; $fila < $filas; $fila++) {
        $arreglo[$fila] = array();
        for ($columna=0; $columna<$columnas; $columna++)
            $arreglo[$fila][$columna]=$valor;
    }
    return $arreglo;
}

//Función genérica para imprimir un $arreglo bidimensional
function ImprimeArregloBidimensional($arreglo){
```

```

    for ($fila=0; $fila < count($arreglo); $fila++){
        echo "<br>";
        for ($columna=0; $columna < count($arreglo[$fila]); $columna++)
            echo $arreglo[$fila][$columna];
    }
}

```

Poner una Reina al azar en un tablero de ajedrez y mostrar su ataque

07/Ej10.php

```

<?php
/* Crear un tablero de ajedrez de 8*8, poner en una posición al
   azar una reina y mostrar su ataque */
$tabla = GeneraArregloBidimensional(8, 8, '.');
$tabla = PoneReinaAzar($tabla);
ImprimeArregloBidimensional($tabla);

//Pone una reina al azar y muestra su ataque
function PoneReinaAzar($arreglo) {
    $filaAzar = rand(0, count($arreglo)-1);
    $coluAzar = rand(0, count($arreglo[0])-1);
    $tmpFil=$filaAzar;
    $tmpCol=$coluAzar;
    while($tmpFil>=0 && $tmpCol>=0) { $arreglo[$tmpFil][$tmpCol]='x';
    $tmpFil--; $tmpCol--; }
    $tmpFil=$filaAzar; $tmpCol=$coluAzar;
    while($tmpFil>=0 && $tmpCol<8) { $arreglo[$tmpFil][$tmpCol]='x';
    $tmpFil--; $tmpCol++; }
    $tmpFil=$filaAzar; $tmpCol=$coluAzar;
    while($tmpFil<8 && $tmpCol<8) { $arreglo[$tmpFil][$tmpCol]='x';
    $tmpFil++; $tmpCol++; }
    $tmpFil=$filaAzar; $tmpCol=$coluAzar;
    while($tmpFil<8 && $tmpCol>=0) { $arreglo[$tmpFil][$tmpCol]='x';
    $tmpFil++; $tmpCol--; }
    for ($tmpFil=0; $tmpFil<8; $tmpFil++) $arreglo[$tmpFil][$coluAzar]='x';
    for ($tmpCol=0; $tmpCol<8; $tmpCol++) $arreglo[$filaAzar][$tmpCol]='x';
    $arreglo[$filaAzar][$coluAzar] = 'R';
    return $arreglo;
}

/* Función genérica para crear $arreglos bidimensionales
   con el número de filas y columnas dado y llena cada celda
   de un valor */
function GeneraArregloBidimensional($filas, $columnas, $valor) {
    $arreglo = array();
    for ($fila=0; $fila < $filas; $fila++) {
        $arreglo[$fila] = array();
        for ($columna=0; $columna<$columnas; $columna++)
            $arreglo[$fila][$columna]=$valor;
    }
}

```

```

    }
    return $arreglo;
}

//Función genérica para imprimir un $arreglo bidimensional
function ImprimeArregloBidimensional($arreglo){
    for ($fila=0; $fila < count($arreglo); $fila++){
        echo "<br>";
        for ($columna=0; $columna < count($arreglo[$fila]); $columna++){
            echo $arreglo[$fila][$columna];
        }
    }
}

```

Ruta del menor costo

Dada una tabla (arreglo bidimensional) de costos que muestra cuánto cuesta ir de una ciudad a otra. El reto es encontrar una ruta que visite todas las ciudades, sin repetir ninguna y que tenga el menor coste. Habría que probar $N!$ (N factorial) rutas para encontrar la de menor costo, un proceso muy costoso computacionalmente y no terminaría en un tiempo justo. El método mostrado es usando una técnica indeterminista que se acerca a la ruta menos costosa.

07/Ej11.php

```

<?php
/* Hay N ciudades a recorrer (0 a N-1). Sólo se puede visitar una ciudad
por vez.
    En la tabla aparece cuanto cuesta ir de una ciudad (origen) a otra
ciudad (destino).
    ¿Qué ruta tomar para visitar todas las ciudades con el mínimo costo? */

$ciudad = 20; //Número de ciudades
$minValor=15; //Valor mínimo que tendrá ir de una ciudad a otra
$maxValor=85; //Valor máximo que tendrá ir de una ciudad a otra

//Genera valores de viaje al azar
$valorviajes = inicializavalorviajes($ciudad, $minValor, $maxValor);

//Imprime los valores
ImprimeArregloBidimensional($valorviajes);

//Inicia con una ruta predeterminada 0, 1, 2, 3, .... N ... 0
$ruta = iniciaRuta($ciudad);

//Deduce el costo de esa ruta predeterminada
$costo = DeduceCosto($ruta, $valorviajes);

//Imprime
echo "<br>" . CadenaRuta($ruta) . " Cuesta: $" . $costo;

```

```

//Usando el método Monte Carlo se buscarán otras rutas con menor costo
for($pruebas=1; $pruebas<=50000; $pruebas++){
    $dest1 = rand(0, count($ruta)-1);
    do{
        $dest2 = rand(0, count($ruta)-1);
    }while ($dest2==$dest1);
    $ruta = ModificaRuta($ruta, $dest1, $dest2);
    $costoNuevo = DeduceCosto($ruta, $valorviajes);
    if ($costoNuevo < $costo){
        $costo = $costoNuevo;
        echo "<br>" . CadenaRuta($ruta) . " Cuesta: $" . $costo;
    }
    else
        $ruta = ModificaRuta($ruta, $dest1, $dest2); //Dejar la ruta como
antes
}

function CadenaRuta($ruta){
    $retorna = "Ruta: ";
    for ($cont=0; $cont<count($ruta); $cont++)
        $retorna .= $ruta[$cont] . ", ";
    return $retorna;
}

//Modifica la ruta de viaje
function ModificaRuta($ruta, $dest1, $dest2){
    $tmp = $ruta[$dest1];
    $ruta[$dest1]=$ruta[$dest2];
    $ruta[$dest2]=$tmp;
    return $ruta;
}

//Inicia el arreglo bidimensional de rutas
function iniciaRuta($limite){
    $ruta = array();
    for($cont=0; $cont<$limite; $cont++) array_push($ruta, $cont);
    return $ruta;
}

//Deduce el costo de la ruta de viaje
function DeduceCosto($ruta, $costos){
    $acum=0;
    for ($cont=0; $cont<count($ruta)-1; $cont++)
        $acum += $costos[$ruta[$cont]][$ruta[$cont+1]];
    return $acum;
}

//Llena de valores al azar la tabla de costos de viajes de una ciudad a
otra
function inicializavalorviajes($ciudad, $minValor, $maxValor){
    $tablero = array();

```

```

    for ($fila=0; $fila<$ciudad; $fila++){
        $tablero[$fila] = array();
        for ($columna=0; $columna<$ciudad; $columna++)
            $tablero[$fila][$columna]=rand($minValor, $maxValor);
    }
    for ($cont=0; $cont<$ciudad; $cont++) $tablero[$cont][$cont]=0;
    return $tablero;
}

/* Función genérica para crear $arreglos bidimensionales
con el número de filas y columnas dado y llena cada celda
de un valor */
function GeneraArregloBidimensional($filas, $columnas, $valor) {
    $arreglo = array();
    for ($fila=0; $fila < $filas; $fila++) {
        $arreglo[$fila] = array();
        for ($columna=0; $columna<$columnas; $columna++)
            $arreglo[$fila][$columna]=$valor;
    }
    return $arreglo;
}

//Función genérica para imprimir un $arreglo bidimensional
function ImprimeArregloBidimensional($arreglo){
    for ($fila=0; $fila < count($arreglo); $fila++){
        echo "<br>";
        for ($columna=0; $columna < count($arreglo[$fila]); $columna++)
            echo $arreglo[$fila][$columna] . " ";
    }
}

```

```
<?php
//El script puede tardar un buen tiempo
ini_set('max_execution_time', 300); //300 segundos = 5 minutos

$sudoku = /* El Sudoku como es planteado. Los ceros(0) son las casillas
vacías a completar */
[ [ 5, 3, 0, 0, 7, 0, 0, 0, 0],
  [ 6, 0, 0, 1, 9, 5, 0, 0, 0],
  [ 0, 9, 8, 0, 0, 0, 0, 6, 0],
  [ 8, 0, 0, 0, 6, 0, 0, 0, 3],
  [ 4, 0, 0, 8, 0, 3, 0, 0, 1],
  [ 7, 0, 0, 0, 2, 0, 0, 0, 6],
  [ 0, 6, 0, 0, 0, 0, 2, 8, 0],
  [ 0, 0, 0, 4, 1, 9, 0, 0, 5],
  [ 0, 0, 0, 0, 8, 0, 0, 7, 9]
];

ImprimeArregloBidimensional($sudoku);

$finalizo=false; /* Mantiene el ciclo hasta que resuelva el Sudoku */
$ciclos=0; /* Lleva el número de iteraciones */

$solucion = array(); /* Tablero en el que se trabaja */
for ($fila=0; $fila<9; $fila++){
    $solucion[$fila] = array();
    for ($columna=0; $columna<9; $columna++)
        $solucion[$fila][$columna]=0;
}
$DESTRUYE = 700; /* Cada cuantos ciclos borra números para destrabar */

/* Ciclo que llenará el sudoku completamente */
do{
    for ($fila=0; $fila<9; $fila++){ /* Se copia el sudoku sobre el
tablero a evaluar */
        for ($columna=0; $columna<9; $columna++)
            if ($sudoku[$fila][$columna] != 0) $solucion[$fila][$columna] =
$sudoku[$fila][$columna];
    }

    $numValido=true;
    do { /* Busca un número al azar para colocar en alguna celda */
        $posX = rand(0,8); /* Una posición X de 0 a 8 */
        $posY = rand(0,8); /* Una posición Y de 0 a 8 */
        $numero = rand(1,9); /* Un número al azar de 1 a 9 */
        $numValido=true; /* Chequea si el número no se repite ni
vertical ni horizontalmente */
    } while ($numValido==false);
    $solucion[$posY][$posX] = $numero;
    $ciclos++;
} while ($ciclos<$DESTRUYE);

if ($finalizo==false)
    echo "No se pudo resolver el sudoku en el tiempo establecido";
else
    echo "Se resolvió el sudoku en " . $ciclos . " ciclos";

```

```

        for ($cont=0; $cont<9; $cont++) if
($solucion[$cont][$posY]==$numero || $solucion[$posX][$cont]==$numero)
$numValido=false;
        if ($numValido) $solucion[$posX][$posY]=$numero; /* Si el número no
se repite entonces lo coloca en el tablero */
        }while(!$numValido);

/* Chequea que NO se viole la regla de que cada uno de los 9 cuadros
internos no repita número */
for ($cuadroX=0; $cuadroX<=6; $cuadroX+=3)
for ($cuadroY=0; $cuadroY<=6; $cuadroY+=3) {
    $numRepite=0;
    for ($valor=1; $valor<=9; $valor++){
        $numRepite=0;
        for ($posX=0; $posX<3; $posX++)
            for ($posY=0; $posY<3; $posY++){
                if
($solucion[$cuadroX+$posX][$cuadroY+$posY]==$valor) $numRepite++;
                if ($numRepite>1) break;
            }

            if ($numRepite>1) /* Si detecta repetición, entonces borra
todos los números repetidos */
                for ($posX=0; $posX<3; $posX++)
                    for ($posY=0; $posY<3; $posY++)
                        if
($solucion[$cuadroX+$posX][$cuadroY+$posY]==$valor)
$solucion[$cuadroX+$posX][$cuadroY+$posY]=0;
                    }
            }

    $finalizo=true; /* Chequea si se completó el sudoku completamente */
    for ($posX=0; $posX<9; $posX++)
        for ($posY=0; $posY<9; $posY++)
            if ($solucion[$posX][$posY]==0) $finalizo=false;

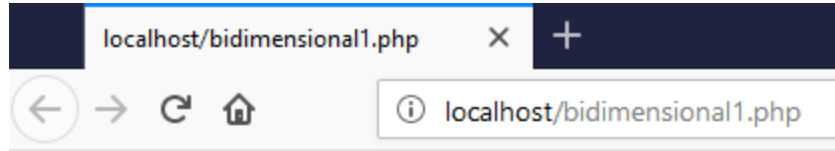
    $ciclos++; /* Cada ciertos ciclos para destrabar, borra la tercera
parte de lo completado */
    if ($ciclos%$DESTRUYE==0)
        for ($posX=0; $posX<9; $posX++)
            for ($posY=0; $posY<9; $posY++)
                if ( rand(0,2) == 0) $solucion[$posX][$posY]=0;
}while (!$finalizo);

echo "<p style='font-family:courier new; '>";
echo "Ciclos totales: " . $ciclos . "</p>";
ImprimeArregloBidimensional($solucion);

//Función genérica para imprimir un $arreglo bidimensional

```

```
function ImprimeArregloBidimensional($arreglo){
    for ($fila=0; $fila < count($arreglo); $fila++){
        echo "<br>";
        for ($columna=0; $columna < count($arreglo[$fila]); $columna++){
            echo $arreglo[$fila][$columna] . " ";
        }
    }
}
```



```
5 3 0 0 7 0 0 0 0
6 0 0 1 9 5 0 0 0
0 9 8 0 0 0 0 6 0
8 0 0 0 6 0 0 0 3
4 0 0 8 0 3 0 0 1
7 0 0 0 2 0 0 0 6
0 6 0 0 0 0 2 8 0
0 0 0 4 1 9 0 0 5
0 0 0 0 8 0 0 7 9
```

Ciclos totales: 14535

```
5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 9
```

Ilustración 34: Sudoku resuelto

Poner los barcos en el juego batalla naval

07/Ej13.php

```
<?php
/* Hacer un algoritmo que ponga los 5 barcos
   al azar en el tablero del juego de batalla naval */

//Crear el tablero del juego
$tableroJuego = inicializaTablero(14, 10);

//Poner cada barco: arreglo, tipo barco, número de huecos
$tableroJuego = poneBarco($tableroJuego, 'P', 5);
$tableroJuego = poneBarco($tableroJuego, 'G', 4);
$tableroJuego = poneBarco($tableroJuego, 'C', 3);
$tableroJuego = poneBarco($tableroJuego, 'S', 3);
$tableroJuego = poneBarco($tableroJuego, 'D', 2);
ImprimeArregloBidimensional($tableroJuego);

//Genera el tablero
function inicializaTablero($filas, $columnas){
    $tablero = array();
    for ($fila=0; $fila<$filas; $fila++){
        $tablero[$fila] = array();
        for ($columna=0; $columna<$columnas; $columna++)
            $tablero[$fila][$columna]='.';
    }
    return $tablero;
}

//Función genérica para imprimir un arreglo bidimensional
function ImprimeArregloBidimensional($arreglo){
    for ($fila=0; $fila < count($arreglo); $fila++){
        echo "<br>";
        for ($columna=0; $columna < count($arreglo[$fila]); $columna++)
            echo $arreglo[$fila][$columna] . " ";
    }
}

//Pone el barco en una posición al azar
function poneBarco($tablero, $simbolo, $huecos){
    do{
        /* ¿Vertical u horizontal?
           0 es vertical, 1 es horizontal */
        $orienta = rand(0,1);

        $fil = rand(0, count($tablero)-1);
        $col = rand(0, count($tablero[0])-1);
        $posValida = true; //La posición del barco es válida

        //Chequea si no se sale del tablero
```

```

if ($fil+$huecos >= count($tablero) && $orienta==0)
    $posValida=false; //La posición del barco es inválida
else if ($col+$huecos >= count($tablero[0]) && $orienta==1)
    $posValida=false; //La posición del barco es inválida
else { //Chequea si ya ha sido ocupada esa parte
    if ($orienta==0)
        for($cont=0; $cont<$huecos; $cont++){
            if ($tablero[$fil+$cont][$col]!='.')
                $posValida=false;
        }
    else
        for($cont=0; $cont<$huecos; $cont++){
            if ($tablero[$fil][$col+$cont]!='.')
                $posValida=false;
        }
    }
while($posValida==false);

if ($orienta==0)
    for($cont=0; $cont<$huecos; $cont++){
        $tablero[$fil+$cont][$col]=$simbolo;
    }
else
    for($cont=0; $cont<$huecos; $cont++){
        $tablero[$fil][$col+$cont]=$simbolo;
    }

return $tablero;
}

```

Funciones que reciben distinto número de parámetros

08/Ej01.php

```
<?php
//Funciones que reciben distinto número de parámetros
UnaFuncion();
UnaFuncion("esto", "es", "una", "prueba");
UnaFuncion(1,2,3,4,5,6,7,8,9);
UnaFuncion("palabra", 5, "texto", 7, 8.56);

function UnaFuncion(){
    //total parámetros
    $numParametros = func_num_args();
    echo "Total parámetros:" . $numParametros . "<br>";
}
```

08/Ej02.php

```
<?php
//Funciones que reciben distinto número de parámetros
UnaFuncion("paso", "varios", "argumentos", 7, 1, "numeros", 3, "y texto");

function UnaFuncion(){
    //total parámetros
    $ListadoParametros = func_get_args();

    for ($cont=0; $cont < count($ListadoParametros); $cont++)
        echo $ListadoParametros[$cont] . "<br>";
}
```

Funciones que reciben a su vez funciones por parámetros

También son llamadas funciones anónimas (Anonymous functions)

08/Ej03.php

```
<?php
//Se envía una función que retorna el resultado de a - b
$MiFuncion = function($a,$b){ return $a-$b; };
$prueba = Ejecuta($MiFuncion);
echo $prueba . "<br>";

//Se envía una función que retorna el resultado de a * b
$MiFuncion = function($a,$b){ return $a*$b; };
$prueba = Ejecuta($MiFuncion);
echo $prueba . "<br>";

//Se envía una función que retorna el resultado de a / b
$MiFuncion = function($a,$b){ return $a/$b; };
$prueba = Ejecuta($MiFuncion);
echo $prueba . "<br>";

//Esta función recibe como parámetro: ;una función!
function Ejecuta($MiFuncion){
    $valorA = 15;
    $valorB = 10;
    //Dependiendo de la función recibida como parámetro, ejecuta la acción
    $resultado = $MiFuncion($valorA, $valorB);
    return $resultado;
}
```

```
<?php
//Se envía una función que retorna el resultado de a - b
$prueba = Ejecuta(5, 7, function($a,$b){ return $a-$b; });
echo $prueba . "<br>";

//Se envía una función que retorna el resultado de a * b
$prueba = Ejecuta(10, 3, function($a,$b){ return $a*$b; });
echo $prueba . "<br>";

//Se envía una función que retorna el resultado de a / b
$prueba = Ejecuta(9, 12, function($a,$b){ return $a/$b; });
echo $prueba . "<br>";

//Esta función recibe como parámetro: ;una función!
function Ejecuta($valorA, $valorB, $MiFuncion){
    //Dependiendo de la función recibida como parámetro, ejecuta la acción
    $resultado = $MiFuncion($valorA, $valorB);
    return $resultado;
}
```

Operaciones de bit

Convertir un entero a su representación binaria

09/Ej01.php

```
<?php
//Convertir un entero a su representación binaria
$valor = 17;
$resultado = decbin($valor);
echo $valor . " en binario es: " . $resultado . "<br>";

//Conversión directa
$convierte = decbin(890);
echo "En binario el número 890 es " . $convierte . "<br>";
```

Operaciones OR, AND y XOR directas

09/Ej02.php

```
<?php
//Operaciones de bit
$valA = 25;
$valB = 20;
echo $valA . " en binario es: " . decbin($valA) . "<br>";
echo $valB . " en binario es: " . decbin($valB) . "<br>";

$valC = $valA | $valB; //Operación OR
echo $valC . " en binario es: " . decbin($valC) . "<br>";
```

09/Ej03.php

```
<?php
//Operaciones de bit
$valA = 25;
$valB = 20;
echo $valA . " en binario es: " . decbin($valA) . "<br>";
echo $valB . " en binario es: " . decbin($valB) . "<br>";

$valC = $valA & $valB; //Operación AND
echo $valC . " en binario es: " . decbin($valC) . "<br>";
```

```

<?php
//Operaciones de bit
$valA = 25;
$valB = 20;
echo $valA . " en binario es: " . decbin($valA) . "<br>";
echo $valB . " en binario es: " . decbin($valB) . "<br>";

$valC = $valA ^ $valB; //Operación XOR
echo $valC . " en binario es: " . decbin($valC) . "<br>";

```

Multiplicar usando operaciones de bit

```

<?php
//Operaciones de bit
$valA = 2;
echo $valA . " en binario es: " . decbin($valA) . "<br>";

$valB = $valA << 1; //Multiplica por 2
echo $valB . " en binario es: " . decbin($valB) . "<br>";

$valC = $valA << 2; //Multiplica por 4
echo $valC . " en binario es: " . decbin($valC) . "<br>";

$valD = $valA << 3; //Multiplica por 8
echo $valD . " en binario es: " . decbin($valD) . "<br>";

```

Dividir usando operaciones de bit

09/Ej06.php

```
<?php
//Operaciones de bit
$valA = 64;
echo $valA . " en binario es: " . decbin($valA) . "<br>";

$valB = $valA >> 1; //Divide entre 2
echo $valB . " en binario es: " . decbin($valB) . "<br>";

$valC = $valA >> 2; //Divide entre 4
echo $valC . " en binario es: " . decbin($valC) . "<br>";

$valD = $valA >> 3; //Divide entre 8
echo $valD . " en binario es: " . decbin($valD) . "<br>";
```

Intercambiar valores usando operaciones de bit

09/Ej07.php

```
<?php
$valA = 64;
$valB = 17;
echo "valores: " . $valA . " , " . $valB . "<br>";

//Para intercambiar los valores de esas variables se puede hacer uso de
operadores de bit (XOR)
$valA^=$valB;
$valB^=$valA;
$valA^=$valB;
echo "valores: " . $valA . " , " . $valB . "<br>";
```


Programación orientada a objetos

Definiendo clases y atributos

10/Ej01.php

```
<?php
//Define una clase
class MiClase{
    //Propiedades
    public $valorPublico; //Acceso público
    private $datoPrivado; //Acceso privado
}

$instancia = new MiClase(); //Instancia la clase
$instancia->valorPublico = 17; //Se puede hacer por ser pública
echo $instancia->valorPublico . "<br>";

//Estas instrucciones fallarían
$instancia->datoPrivado = 1235; //NO se puede acceder directamente a una
propiedad privada
echo $instancia->datoPrivado;
```

Definiendo métodos para acceder a atributos privados

10/Ej02.php

```
<?php
class MiClase{
    private $datoPrivado; //Acceso privado
    public function PonerDato($valor){
        $this->datoPrivado = $valor;
    }
    public function DevolverDato(){
        return $this->datoPrivado;
    }
}

$instancia = new MiClase(); //Instancia la clase
$instancia->PonerDato(1235); //Llama al método público
echo $instancia->DevolverDato(); //Llama al otro método público
```

Dos variables, misma instancia

10/Ej03.php

```
<?php
class MiClase{
    public $valorPublico; //Acceso público
}

$instancia = new MiClase(); //Instancia la clase
$instancia->valorPublico = 17;

//¡OJO! Refiere a la misma instancia
$otravariante = $instancia;

echo "otravariante: " . $otravariante->valorPublico . "<br>";
echo "instancia: " . $instancia->valorPublico . "<br>";
```

Dos variables y dos instancias distintas

10/Ej04.php

```
<?php
class MiClase{
    public $valorPublico; //Acceso público
}

$instancia = new MiClase(); //Instancia la clase
$instancia->valorPublico = 17;

//¡OJO! son diferentes instancias
$otravariante = new $instancia;
$otravariante->valorPublico = 2853;

echo "otravariante: " . $otravariante->valorPublico . "<br>";
echo "instancia: " . $instancia->valorPublico . "<br>";
```

```
<?php
class MiClase{
    //Constante
    const VALORCONSTANTE = 2.7812431;
}

$instancia = new MiClase(); //Instancia la clase

//Así se accede desde afuera a una constante
echo $instancia::VALORCONSTANTE . "<br>";

//Esto no se puede hacer
//$instancia::VALORCONSTANTE = 77;
```

Constructores y destructores

El constructor se ejecuta cuando se instancia el objeto.

```
<?php
class MiClase{
    //Constructor
    function __construct(){
        echo "Ejecuta al instanciar la clase<br>";
    }
}

$objeto = new MiClase();
```

```
<?php
class MiClase{
    public $valorpublico;

    //Constructor con parámetros
    function __construct($valor){
        $this->valorpublico = $valor;
    }
}

//Se instancia la clase con un valor
$objeto = new MiClase("hola mundo");
echo $objeto->valorpublico;
```

En el caso del destructor, se ejecuta cuando el objeto no puede usarse otra vez.

```
<?php
class MiClase{
    //Constructor
    function __construct(){
        echo "Se ha construido el objeto<br>";
    }
    //Destructor
    function __destruct(){
        echo "Se ha destruido el objeto<br>";
    }
}

//Se instancia la clase
$objeto = new MiClase();
```

Un constructor con parámetros con valores por defecto

Si el constructor tiene varios parámetros y es llamado con menos parámetros, es posible hacerlo siempre y cuando los parámetros tengan valores por defecto.

10/Ej09.php

```
<?php
class MiClase{
    public $valorA;
    public $valorB;
    public $valorC;
    //Constructor con parámetros con valores por defecto
    public function __construct($valA, $valB='Pez Espada',
$valC='Sardina'){
        $this->valorA = $valA;
        $this->valorB = $valB;
        $this->valorC = $valC;
    }
    public function Imprime(){
        echo $this->valorA . " .. " . $this->valorB . " .. " . $this-
>valorC . "<br>";
    }
}

$probar = new MiClase('Bagre');
$probar->Imprime();
```

10/Ej10.php

```
<?php
class MiClase{
    public $valorA;
    public $valorB;
    public $valorC;
    //Constructor con parámetros con valores por defecto
    public function __construct($valA, $valB='Pez Espada',
$valC='Sardina'){
        $this->valorA = $valA;
        $this->valorB = $valB;
        $this->valorC = $valC;
    }
    public function Imprime(){
        echo $this->valorA . " .. " . $this->valorB . " .. " . $this-
>valorC . "<br>";
    }
}

$probar = new MiClase('Bagre', 'Delfín');
$probar->Imprime();
```

Instancia un objeto dentro de otra clase y accede a los atributos

Por ejemplo, un atributo es la instancia de una clase externa. Luego a través de ese atributo en particular es posible acceder a los atributos y métodos públicos de esa clase externa.

10/Ej11.php

```
<?php
class MiClase{
    public $valorPublico; //Un sólo atributo público
}

class OtraClase{
    public $objeto; //Atributo público
    public function unMetodo($valor){
        $this->objeto = new MiClase(); //Crea un objeto de tipo MiClase
        $this->objeto->valorPublico = $valor; //Da un valor a la propiedad
de MiClase
    }
}

$probar = new OtraClase();
$probar->unMetodo(3.1415926537);
echo $probar->objeto->valorPublico; //Muestra el valor de la propiedad de
MiClase
```

10/Ej12.php

```
<?php
class MiClase{
    public function Aviso(){
        echo "Método de MiClase";
    }
}

class OtraClase{
    public $objeto; //Atributo público
    public function unMetodo(){
        $this->objeto = new MiClase(); //Crea un objeto de tipo MiClase
    }
}

$probar = new OtraClase();
$probar->unMetodo();
echo $probar->objeto->Aviso(); //Llama al método de MiClase
```

Recibe un objeto como parámetro de una función

10/Ej13.php

```
<?php
class MiClase{
    public $valorPublico; //Un sólo atributo público
}

//Esta función recibe un objeto como parámetro
function Imprimir($objeto_instanciado){
    echo $objeto_instanciado->valorPublico;
}

$probar = new MiClase();
$probar->valorPublico = "Esta es una prueba...";
Imprimir($probar); //Envía la instancia como parámetro
```

Clonando objetos

Se copia el valor de los atributos de una instancia a otra.

10/Ej14.php

```
<?php
class MiClase{
    public $valorA;
    public $valorB;
}

//Se instancia y se le dan unos valores a los atributos
$objeto = new MiClase();
$objeto->valorA = "Hola mundo";
$objeto->valorB = 3.14159265;

//Se crea un objeto nuevo y se copian los valores al nuevo objeto
$oTRO = clone($objeto);

//Se cambia un valor del objeto original
$objeto->valorB = 90123.7;

//Imprime el objeto original y el objeto copia
echo "Valores con el original son: " . $objeto->valorA . ", " . $objeto->valorB . "<br>";
echo "Valores con la copia son: " . $oTRO->valorA . ", " . $oTRO->valorB . "<br>";
```

Detectar instancias

10/Ej15.php

```
<?php
class MiClase{
    public $valorPublico; //Un sólo atributo público
}

class OtraClase{
    public function Imprimir(){
        echo "esto es una prueba";
    }
}

//Instancia una determinada clase
$objeto = new OtraClase();

//Si condicional para saber si un objeto es instancia de una determinada
clase
if ($objeto instanceof MiClase)
    echo "objeto es instancia de MiClase";
else
    echo "objeto NO es instancia de MiClase";
```

Métodos estáticos

10/Ej16.php

```
<?php
//Una clase con métodos estáticos
class MiClase{
    public static function Imprimir(){
        echo "Hola Mundo<br><br>";
    }

    public static function Sumar($valA, $valB){
        return $valA + $valB;
    }
}

MiClase::Imprimir(); //No necesita instanciar, se usa directamente

$resultado = MiClase::Sumar(17, 50); //Uso directo sin instanciar
echo $resultado;
```


Serializar un objeto

Una instancia se puede convertir en una cadena (string), guardarse como tal, luego recuperar esa cadena y reconstruir el objeto.

10/Ej17.php

```
<?php
class MiClase{
    public $valorA;
    public $valorB;
    public $valorC;
}

//Instancia la clase y le da unos valores a los atributos
$objeto = new MiClase();
$objeto->valorA = 11;
$objeto->valorB = "esta es una prueba";
$objeto->valorC = 3333;

//Convierte el objeto en una cadena
$cadena = serialize($objeto);
echo $cadena;

//Crea una nueva instancia sin valores
$o otro = new MiClase();

//Toma la cadena de la anterior instancia y la convierte a valores del
objeto nuevo
$o otro = unserialize($cadena);
echo '<br>' . $o otro->valorB;
```

```
<?php
class MiClase{
    public $valorPublico; //Un sólo atributo público
}

class OtraClase{
    public $objeto; //Atributo público
    public function unMetodo($valor){
        $this->objeto = new MiClase(); //Crea un objeto de tipo MiClase
        $this->objeto->valorPublico = $valor; //Da un valor a la propiedad
de MiClase
    }
}

//Instancia
$probar = new OtraClase();
$probar->unMetodo(3.1415926537);

//Convierte el objeto a una cadena
$cadena = serialize($probar);
echo $cadena . "<br><br>";

//Crea una nueva instancia
$nuevo = new OtraClase();
$nuevo = unserialize($cadena);
echo $nuevo->objeto->valorPublico;
```

Comparando objetos

Compara el valor de los atributos de un objeto con otro

10/Ej19.php

```
<?php
class MiClase{
    public $valorA;
    public $valorB;
}
$objeto = new MiClase(); //Instancia la clase
$objeto->valorA = 11;
$objeto->valorB = 222;

$otro = new MiClase(); //Instancia otro objeto de la misma clase
$otro->valorA = 11;
$otro->valorB = 222;

if ($objeto == $otro)
    echo "objetos iguales";
else
    echo "objetos diferentes";
```

```
<?php
//Clase padre
class MiClase{
    public $valorPublico; //Acceso público
}

//Clase hija, hereda de la clase padre
class ClaseHija extends MiClase{ //Herencia
    public $propiedadhija;
}

//Instancia la clase hija
$objeto = new ClaseHija();

//Da valor a un atributo que se encuentra en la clase padre
$objeto->valorPublico = 888;

//Da valor a un atributo que se encuentra en la clase hija
$objeto->propiedadhija = 555;

//Imprime los valores de los atributos
echo $objeto->valorPublico . "<br>";
echo $objeto->propiedadhija . "<br>";
```

```
<?php
class MiClase{
    //Atributo puede ser accedido desde afuera
    public $valorA = 'público';

    //Atributo solo puede ser accedido por clases hijas
    protected $valorB = 'protegido';

    //Atributo que solo puede ser accedido por esta clase, nadie mas.
    private $valorC = 'privado';
}

class claseHija extends MiClase{
    public function Metodo(){
        echo $this->valorA . "<br>";
        echo $this->valorB . "<br>";
        echo $this->valorC . "<br>"; //Dará error: undefined
    }
}

$probar = new claseHija();
$probar->Metodo();
```

```
<?php
//Clase padre
class MiClase{
    //El método Imprimir en el padre
    public function Imprimir(){
        echo "Texto de la clase padre";
    }
}

//Clase hija
class claseHija extends MiClase{
    //El método Imprimir en la clase hija
    public function Imprimir(){
        echo "Imprime de la clase hija";
    }
}

//Instancia la clase hija
$probar = new claseHija();

/* Llama al método Imprimir: el de la hija */
$probar->Imprimir();
```

Se añade una instrucción en el método de la clase hija para llamar al método de la clase padre

10/Ej23.php

```
<?php
//Clase padre
class MiClase{
    //El método Imprimir en el padre
    public function Imprimir(){
        echo "Texto de la clase padre<br>";
    }
}

//Clase hija
class claseHija extends MiClase{
    //El método Imprimir en la clase hija
    public function Imprimir(){
        echo "Imprime de la clase hija<br>";
        parent::Imprimir(); //Llama al método de la clase padre
    }
}

//Instancia la clase hija
$probar = new claseHija();

/* Llama al método Imprimir
¿Cuál de ellos? El de la hija y luego la del padre */
$probar->Imprimir();
```

Un cuidado a tener es que los métodos deben tener el mismo número de parámetros, ver el siguiente código:

10/Ej24.php

```
<?php
class MiClase{
    public function UnaFuncion($valor){
        echo "Padre, recibe: " . $valor . "<br>";
    }
}

class claseHija extends MiClase{
    public function UnaFuncion($valorA, $valorB){
        echo "Hija, recibe: " . $valorA . ", ". $valorB . "<br>";
    }
}

$probar = new claseHija();
$probar->UnaFuncion(56, 78); //Falla el llamado
```

Reacción a constructores en clase padre e hija

10/Ej25.php

```
<?php
class MiClase{
    public function __construct($valorA, $valorB){
        echo "Padre, recibe: " . $valorA . ", ". $valorB . "<br>";
    }
}

class claseHija extends MiClase{
    public function __construct($valorA, $valorB){
        echo "Hija, recibe: " . $valorA . ", ". $valorB . "<br>";
    }
}

//¿Qué constructor se ejecuta?
$probar = new claseHija(56, 78);
```



```
<?php
//Clase padre
class MiClase{
    //Constructor con un sólo parámetro
    public function __construct($valor){
        echo "Padre, recibe: " . $valor . "<br>";
    }
}

//Clase hija
class claseHija extends MiClase{
    //Constructor con dos parámetros
    public function __construct($valorA, $valorB){
        echo "Hija, recibe: " . $valorA . ", " . $valorB . "<br>";

        //Llama al constructor del padre
        parent::__construct($valorA);
    }
}

//Instancia la clase hija
$probar = new claseHija(56, 78);
```

```
<?php
//Clase abstracta. No se puede instanciar.
abstract class MiClase{
    public function operar($valorA, $valorB){
        echo "Padre, recibe: " . $valorA . ", " . $valorB . "<br>";
    }
}

//Si se puede heredar
class claseHija extends MiClase{
    public function Imprimir(){
        echo "Hola Mundo <br>";
    }
}

//Se puede hacer: instanciar la clase hija
$probar = new claseHija();
$probar->Imprimir();
$probar->operar(123, 678);

//Generará un error. No se puede instanciar.
$otro = new MiClase();
```

```
<?php
//Clases y métodos abstractos
abstract class MiClase{
    //Este método obligatoriamente tiene que implementarse en la clase hija
    public abstract function operar();
    /* ¡OJO! Sólo clases abstractas pueden tener métodos abstractos */
}

class claseHija extends MiClase{
    public function Imprimir($valorA, $valorB){
        echo "Hija, recibe: " . $valorA . ", ". $valorB . "<br>";
    }

    //Aquí se implementa el método abstracto de la clase padre
    public function operar(){
        echo "método implementado obligatoriamente en la clase hija";
    }
}

$probar = new claseHija();
$probar->Imprimir(234, 678);
$probar->operar();
```

```
<?php
//Clases y métodos abstractos
abstract class MiClase{
    //Este método obligatoriamente tiene que implementarse en la clase hija
    public abstract function operar();
    /* ¡OJO! Sólo clases abstractas pueden tener métodos abstractos */
}

//La clase hija no implementa el método abstracto dando como resultado un error
class claseHija extends MiClase{
    public function Imprimir($valorA, $valorB){
        echo "Hija, recibe: " . $valorA . ", " . $valorB . "<br>";
    }
}

$probar = new claseHija();
$probar->Imprimir(234, 678);
```

Evitar la herencia

Una clase se marca como “final” para evitar que hereden de esta:

```
<?php
//Clase que no se puede heredar
final class MiClase{
    public function Imprimir(){
        echo "Texto de la clase padre";
    }
}

//Se intenta hacer herencia y genera un error
class claseHija extends MiClase{
    public function Imprimir(){
        echo "sobreescribo el método";
    }
}

$probar = new claseHija();
$probar->Imprimir();
```

```
<?php
//Cada ítem de la lista será Nodo
class Nodo {
    private $valor;
    private $letra;
    private $cadena;

    function __construct($valor, $letra, $cadena){
        $this->valor = $valor;
        $this->letra = $letra;
        $this->cadena = $cadena;
    }

    function Imprime(){
        echo "Valor: " . $this->valor . " caracter: " . $this->letra . "
cadena: " . $this->cadena . "<br>";
    }
}

//Declara la lista que almacenará objetos Nodo
$Objetos = array();

//Adiciona elementos a la lista
array_push($Objetos, new Nodo(7, 'k', "prueba"));
array_push($Objetos, new Nodo(-9.4, 'r', "texto"));
array_push($Objetos, new Nodo(1.836, 's', "test"));

//Trae un elemento. ¡OJO! Es una referencia (como un acceso directo)
$tmp = $Objetos[1];
$tmp->Imprime();

//Llama objeto directamente de la lista
$Objetos[0]->Imprime();
```

```
<?php
//Cada ítem de la lista será Nodo
class Nodo {
    private $valor;
    private $letra;
    private $cadena;

    function __construct($valor, $letra, $cadena){
        $this->valor = $valor;
        $this->letra = $letra;
        $this->cadena = $cadena;
    }

    function Imprime(){
        echo "Valor: " . $this->valor . " caracter: " . $this->letra . "
cadena: " . $this->cadena . "<br>";
    }
}

//Declara la lista que almacenará objetos Nodo
$Objetos = array();

//Adiciona elementos a la lista
$Objetos[] = new Nodo(7, 'k', "prueba");
$Objetos[] = new Nodo(-9.4, 'r', "texto");
$Objetos[] = new Nodo(1.836, 's', "test");

//Trae un elemento. ¡OJO! Es una referencia (como un acceso directo)
$tmp = $Objetos[1];
$tmp->Imprime();

//Llama objeto directamente de la lista
$Objetos[0]->Imprime();
```

```
<?php
//Cada ítem de la lista será Nodo
class Nodo {
    private $valor;
    private $letra;
    private $cadena;

    function __construct($valor, $letra, $cadena){
        $this->valor = $valor;
        $this->letra = $letra;
        $this->cadena = $cadena;
    }

    function Imprime(){
        echo "Valor: " . $this->valor . " caracter: " . $this->letra . "
cadena: " . $this->cadena . "<br>";
    }

    function setCadena($cadena) {
        $this->cadena = $cadena;
    }
}

//Declara la lista que almacenará objetos Nodo
$Objetos = array();

//Adiciona elementos a la lista
array_push($Objetos, new Nodo(7, 'k', "prueba"));
array_push($Objetos, new Nodo(-9.4, 'r', "texto"));
array_push($Objetos, new Nodo(1.836, 's', "test"));

//Llama objeto directamente de la lista
$Objetos[0]->Imprime();

//Cambia el valor
$Objetos[0]->setCadena("zzzzzzzzzzzzzzzzzzzz");
$Objetos[0]->Imprime();
```

Polimorfismo

El polimorfismo, entendido como tener varios métodos con el mismo nombre pero que los diferencia es el número de parámetros, no existe en PHP. El siguiente código falla:

10/Ej34.php

```
<?php
//Este código fallará
class MiClase {
    function MetodoA() {
        document.write("<br>Llama al método A con cero parámetros");
    }
    function MetodoA($param1, $param2) {
        document.write("<br>Llama al método A con dos parámetros");
    }
    function MetodoA($param1) {
        document.write("<br>Llama al método A con un parámetro");
    }
    function MetodoA($param1, $param2, $param3) {
        document.write("<br>Llama al método A con tres parámetros: " +
$param1 + ", " + $param2 + ", " + $param3);
    }
}

$prueba = new MiClase();
$prueba->MetodoA();
$prueba->MetodoA(2, 7);
$prueba->MetodoA(8);
$prueba->MetodoA(5, 9, 3);
```



```
<?php
class MiClase {
    function MetodoA() {
        document.write("<br>Llama al método A");
    }
}
$valorA = new MiClase();
echo "<br>1. Tipo de valorA es " . gettype($valorA);

$valorA = 157;
echo "<br>2. Tipo de valorA es " . gettype($valorA) . ": " . $valorA;
$valorA = "Había una vez...";
echo "<br>3. Tipo de valorA es " . gettype($valorA) . ": " . $valorA;
$valorA = 'K';
echo "<br>4. Tipo de valorA es " . gettype($valorA) . ": " . $valorA;
$valorA = true;
echo "<br>5. Tipo de valorA es " . gettype($valorA) . ": " . $valorA;
$valorA = asin(39); //NAN
echo "<br>6. Tipo de valorA es " . gettype($valorA) . ": " . $valorA;
$valorA = pi();
echo "<br>7. Tipo de valorA es " . gettype($valorA) . ": " . $valorA;
$valorA = array();
echo "<br>8. Tipo de valorA es " . gettype($valorA);
$valorA = NULL;
echo "<br>9. Tipo de valorA es " . gettype($valorA);
```

Árbol binario

Con tres nodos se puede crear un árbol binario: raíz, rama izquierda, rama derecha

11/Ej01.php

```
<?php
//Cada ítem del arbol binario será Nodo
class Nodo {
    public $valor;
    public $izquierda;
    public $derecha;

    function __construct($valor, $izquierda, $derecha){
        $this->valor = $valor;
        $this->izquierda = $izquierda;
        $this->derecha = $derecha;
    }
}

$NodoRaiz = new Nodo(1, null, null); //Nodo raíz
$NodoIzq = new Nodo(2, null, null); //Rama izquierda
$NodoDer = new Nodo(3, null, null); //Rama derecha

//El nodo raíz conecta a la izquierda y derecha
$NodoRaiz->izquierdo = $NodoIzq;
$NodoRaiz->derecho = $NodoDer;

//Imprime los valores del árbol
echo $NodoRaiz->valor . "<br>";
echo $NodoRaiz->izquierdo->valor . "<br>";
echo $NodoRaiz->derecho->valor . "<br>";
```

Recorrido en pre-orden

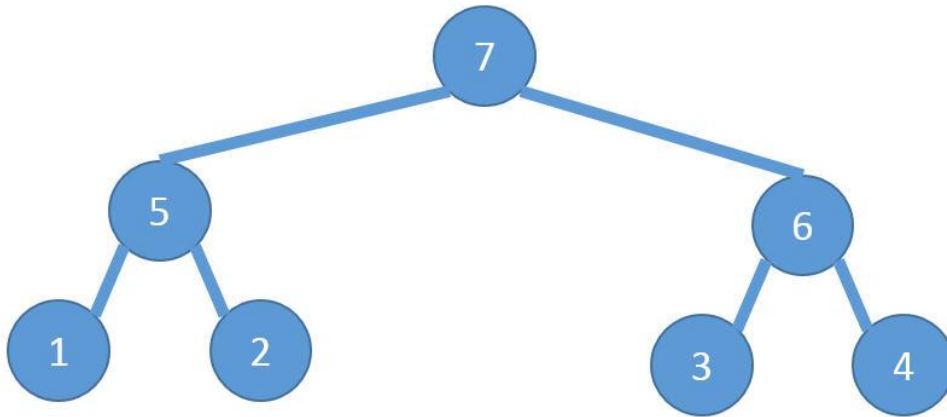


Ilustración 35: Árbol binario

11/Ej02.php

```
<?php
//Cada ítem del árbol binario será Nodo
class Nodo {
    public $valor;
    public $izquierda;
    public $derecha;

    function __construct($valor, $izquierda, $derecha){
        $this->valor = $valor;
        $this->izquierda = $izquierda;
        $this->derecha = $derecha;
    }
}

//Se genera el árbol binario
$Nodo1 = new Nodo(1, null, null);
$Nodo2 = new Nodo(2, null, null);
$Nodo3 = new Nodo(3, null, null);
$Nodo4 = new Nodo(4, null, null);
$Nodo5 = new Nodo(5, $Nodo1, $Nodo2);
$Nodo6 = new Nodo(6, $Nodo3, $Nodo4);
$Nodo7 = new Nodo(7, $Nodo5, $Nodo6);

//Lo recorre en preorden
preorden($Nodo7); //7,5,1,2,6,3,4

function preorden($nodo) {
    if ($nodo==null) return;
```

```
echo " , " . $nodo->valor;  
preorden($nodo->izquierda);  
preorden($nodo->derecha);  
}
```

, 7 , 5 , 1 , 2 , 6 , 3 , 4

Ilustración 36: Recorrido en pre-orden

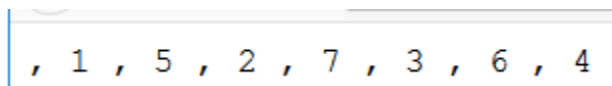
```
<?php
//Cada ítem del arbol binario será Nodo
class Nodo {
    public $valor;
    public $izquierda;
    public $derecha;

    function __construct($valor, $izquierda, $derecha){
        $this->valor = $valor;
        $this->izquierda = $izquierda;
        $this->derecha = $derecha;
    }
}

//Se genera el árbol binario
$Nodo1 = new Nodo(1, null, null);
$Nodo2 = new Nodo(2, null, null);
$Nodo3 = new Nodo(3, null, null);
$Nodo4 = new Nodo(4, null, null);
$Nodo5 = new Nodo(5, $Nodo1, $Nodo2);
$Nodo6 = new Nodo(6, $Nodo3, $Nodo4);
$Nodo7 = new Nodo(7, $Nodo5, $Nodo6);

//Lo recorre en inOrden
inorden($Nodo7); //1,5,2,7,3,6,4

function inorden($nodo){
    if ($nodo==null) return;
    inorden($nodo->izquierda);
    echo " , " . $nodo->valor;
    inorden($nodo->derecha);
}
```



, 1 , 5 , 2 , 7 , 3 , 6 , 4

Ilustración 37: Recorrido en in-orden

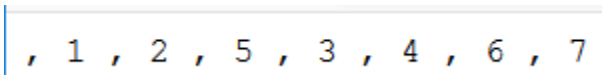
```
<?php
//Cada ítem del árbol binario será Nodo
class Nodo {
    public $valor;
    public $izquierda;
    public $derecha;

    function __construct($valor, $izquierda, $derecha){
        $this->valor = $valor;
        $this->izquierda = $izquierda;
        $this->derecha = $derecha;
    }
}

//Se genera el árbol binario
$Nodo1 = new Nodo(1, null, null);
$Nodo2 = new Nodo(2, null, null);
$Nodo3 = new Nodo(3, null, null);
$Nodo4 = new Nodo(4, null, null);
$Nodo5 = new Nodo(5, $Nodo1, $Nodo2);
$Nodo6 = new Nodo(6, $Nodo3, $Nodo4);
$Nodo7 = new Nodo(7, $Nodo5, $Nodo6);

//Lo recorre en inOrden
postorden($Nodo7); //1,2,5,3,4,6,7

function postorden($nodo) {
    if ($nodo==null) return;
    postorden($nodo->izquierda);
    postorden($nodo->derecha);
    echo " , " . $nodo->valor;
}
```



, 1 , 2 , 5 , 3 , 4 , 6 , 7

Ilustración 38: Recorrido en post-orden

```
<?php
//Cada ítem del arbol binario será Nodo
class Nodo {
    public $valor;
    public $izquierdo;
    public $derecho;

    function __construct($valor, $izquierdo, $derecho){
        $this->valor = $valor;
        $this->izquierdo = $izquierdo;
        $this->derecho = $derecho;
    }
}

//Un arreglo con valores no ordenados
$arreglo = [ 15, 21, 17, 32, 48, 13, 29, 44, 19, 16, 3, -1, 9, 0];

//Genera el árbol binario
$NodoRaiz = new Nodo($arreglo[0], null, null);
for($cont=1; $cont<count($arreglo); $cont++){
    $pasea = $NodoRaiz;
    while ($pasea!=null){
        if ($arreglo[$cont] < $pasea->valor){ //Pone valor en la rama
            izquierda
            if ($pasea->izquierdo!=null)
                $pasea = $pasea->izquierdo;
            else{ //Crea nodo con el nuevo valor
                $pasea->izquierdo = new Nodo($arreglo[$cont], null, null);
                break;
            }
        }
        else{ //Pone valor en la rama derecha
            if ($pasea->derecho!=null)
                $pasea = $pasea->derecho;
            else{ //Crea nodo con el nuevo valor
                $pasea->derecho = new Nodo($arreglo[$cont], null, null);
                break;
            }
        }
    } //Cierra el while que navega por el árbol binario
} //Cierra el for que pasea por todo el arreglo unidimensional no ordenado

//Recorre en in-orden el árbol binario y los datos están ordenados
inorden($NodoRaiz);

function inorden($nodo){
```

```
if ($nodo==null) return;  
inorden($nodo->izquierdo);  
echo " , " . $nodo->valor;  
inorden($nodo->derecho);  
}
```

, -1 , 0 , 3 , 9 , 13 , 15 , 16 , 17 , 19 , 21 , 29 , 32 , 44 , 48

Ilustración 39: Ordenación usando un árbol binario


```
<?php
//Cada ítem de la lista será Nodo
class Nodo {
    private $valor;
    private $letra;
    private $cadena;

    function __construct($valor, $letra, $cadena){
        $this->valor = $valor;
        $this->letra = $letra;
        $this->cadena = $cadena;
    }

    function Imprime(){
        echo "<br>Valor: " . $this->valor;
    }
}

//Declara la lista que almacenará objetos Nodo
$Objetos = array();

//Adiciona elementos a la lista
array_push($Objetos, new Nodo(7, 'k', "prueba"));
array_push($Objetos, new Nodo(-9.4, 'r', "texto"));
array_push($Objetos, new Nodo(1.836, 's', "test"));

//Total
echo "<br>Total elementos: " . count($Objetos);
foreach($Objetos as $pasea){ $pasea->Imprime(); }

//Quita último elemento ingresado
array_pop($Objetos);

//Total
echo "<br>Total elementos: " . count($Objetos);
foreach($Objetos as $pasea){ $pasea->Imprime(); }
```

```
<?php
//Cada ítem de la lista será Nodo
class Nodo {
    private $valor;
    private $letra;
    private $cadena;

    function __construct($valor, $letra, $cadena){
        $this->valor = $valor;
        $this->letra = $letra;
        $this->cadena = $cadena;
    }

    function Imprime(){
        echo "<br>Valor: " . $this->valor;
    }
}

//Declara la lista que almacenará objetos Nodo
$Objetos = array();

//Adiciona elementos a la lista
array_push($Objetos, new Nodo(7, 'k', "prueba"));
array_push($Objetos, new Nodo(-9.4, 'r', "texto"));
array_push($Objetos, new Nodo(1.836, 's', "test"));

//Total
echo "<br>Total elementos: " . count($Objetos);
foreach($Objetos as $pasea){ $pasea->Imprime(); }

//Quita el primer elemento de la lista
array_shift($Objetos);

//Total
echo "<br>Total elementos: " . count($Objetos);
foreach($Objetos as $pasea){ $pasea->Imprime(); }
```

```

<?php
//Cada ítem de la lista será Nodo
class Nodo {
    private $valor;
    private $letra;
    private $cadena;

    function __construct($valor, $letra, $cadena){
        $this->valor = $valor;
        $this->letra = $letra;
        $this->cadena = $cadena;
    }

    function Imprime(){
        echo "<br>Valor: " . $this->valor;
    }
}

//Declara la lista que almacenará objetos Nodo
$Objetos = array();

//Adiciona elementos a la lista
array_push($Objetos, new Nodo(7, 'k', "prueba"));
array_push($Objetos, new Nodo(-9.4, 'r', "texto"));
array_push($Objetos, new Nodo(1.836, 's', "test"));

//Total
echo "<br>Total elementos: " . count($Objetos);
foreach($Objetos as $pasea){ $pasea->Imprime(); }

//Agrega un nodo al inicio de la lista
array_unshift($Objetos, new Nodo(55.5, 'x', "al inicio"));

//Total
echo "<br>Total elementos: " . count($Objetos);
foreach($Objetos as $pasea){ $pasea->Imprime(); }

```

Archivos Planos

Uso de JSON

12/Ej01.php

```
<?php
class Nodo {
    public $valor;
    public $letra;
    public $cadena;

    function __construct($valor, $letra, $cadena){
        $this->valor = $valor;
        $this->letra = $letra;
        $this->cadena = $cadena;
    }
}

//Instancia
$objeto = new Nodo(7, 'k', "prueba");

//Codifica en JSON
$codificado = json_encode($objeto);

//Imprime
echo $codificado;
```

12/Ej02.php

```
<?php
$arreglo = array("Sally", "Vikingo", "Suini", "Grisú", "Capuchina",
    "Milú", "Tom", "Cremoso", "Grisáceo", "Medianoche");

$codificacion = json_encode($arreglo);

echo $codificacion;
```

Los archivos son generados y almacenados en el servidor, **NO** en el cliente.

12/Ej03.php

```
<?php
//Archivos planos. Abre un archivo para escritura
$archivo = fopen("datos.txt", "w");
fwrite($archivo, "Esta es una prueba");
fwrite($archivo, "para escribir datos en");
fwrite($archivo, "un archivo plano.");
fclose($archivo);
echo "Termina";
```

Añadiendo saltos de línea

12/Ej04.php

```
<?php
//Archivos planos. Abre un archivo para escritura y aplica saltos de línea
$archivo = fopen("datos.txt", "w");
fwrite($archivo, "Esta es una prueba" . PHP_EOL);
fwrite($archivo, "para escribir datos en" . PHP_EOL);
fwrite($archivo, "un archivo plano." . PHP_EOL);
fclose($archivo);
echo "Finaliza";
```

Añadiendo datos a un archivo

12/Ej05.php

```
<?php
//Archivos planos. Abre un archivo para escritura
$archivo = fopen("datos.txt", "a");
fwrite($archivo, "Añade líneas al archivo" . PHP_EOL);
fwrite($archivo, "plano y no borra los" . PHP_EOL);
fwrite($archivo, "datos que estaban antes." . PHP_EOL);
fclose($archivo);
echo "Fin";
```

Leyendo un archivo plano

Se leen los archivos planos que están en el servidor, **NO** en el cliente.

12/Ej06.php

```
<?php
//Archivos planos. Leyendo el archivo
$archivo = fopen("datos.txt", "r");
while(!feof($archivo)){ //Mientras no encuentre el fin de archivo
    $cadena = fgets($archivo); //Trae la cadena del archivo
    echo $cadena . "<br>";
}
fclose($archivo);
```

Modelo Vista Controlador (MVC)

Para mostrar resultados al cliente, se hace uso de la instrucción “echo”, por ejemplo:

13/Parte1/Ej01.php

```
<?php
/* Hallar la hipotenusa de un triángulo rectángulo */
$catA = 10;
$catB = 30;
$Hipotenusa = sqrt($catA*$catA+$catB*$catB);

/* Impresión simple */
echo "Cateto A: " . $catA . "<br>";
echo "Cateto B: " . $catB . "<br>";
echo "Hipotenusa: " . $Hipotenusa . "<br>";
```

Al ejecutar se obtiene,

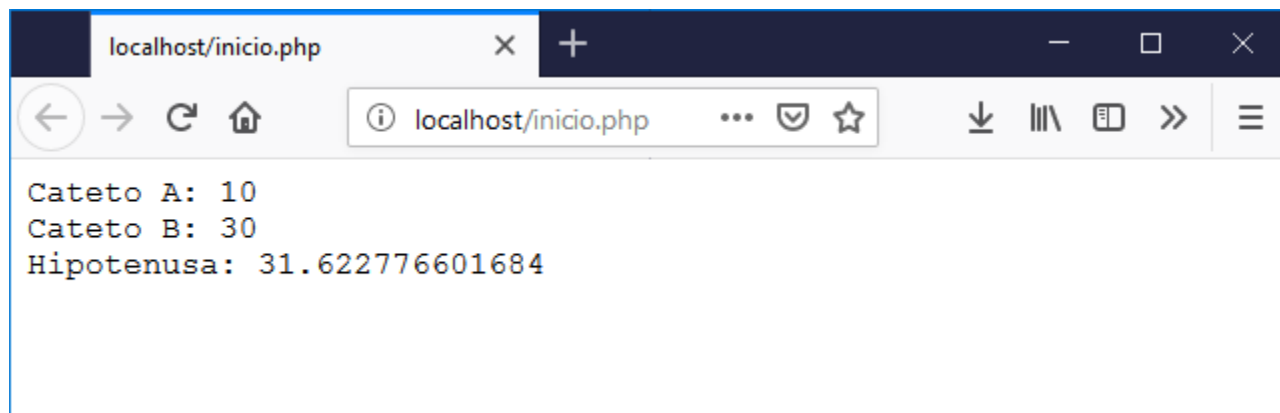


Ilustración 40: Página mostrada

Sin embargo, el código HTML enviado al cliente es este:

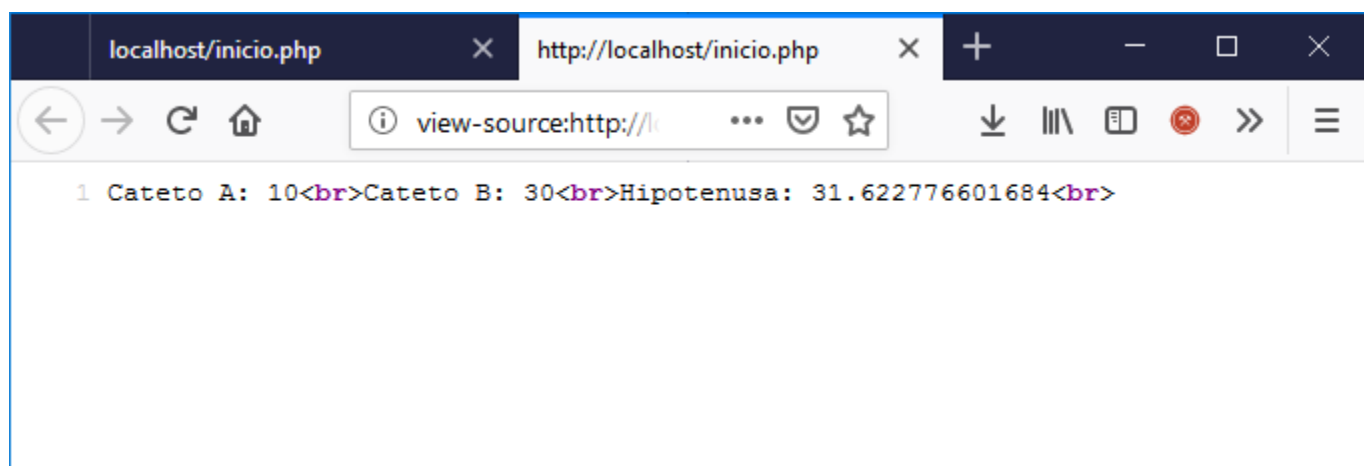


Ilustración 41: Código HTML de la página mostrada

Es una página HTML no correcta, no tiene las etiquetas mínimas necesarias, y eso puede dar problemas.

Una corrección a eso podría ser:

13/Parte1/Ej02.php

```
<?php
/* Hallar la hipotenusa de un triángulo rectángulo */
$catA = 10;
$catB = 30;
$Hipotenusa = sqrt($catA*$catA+$catB*$catB);

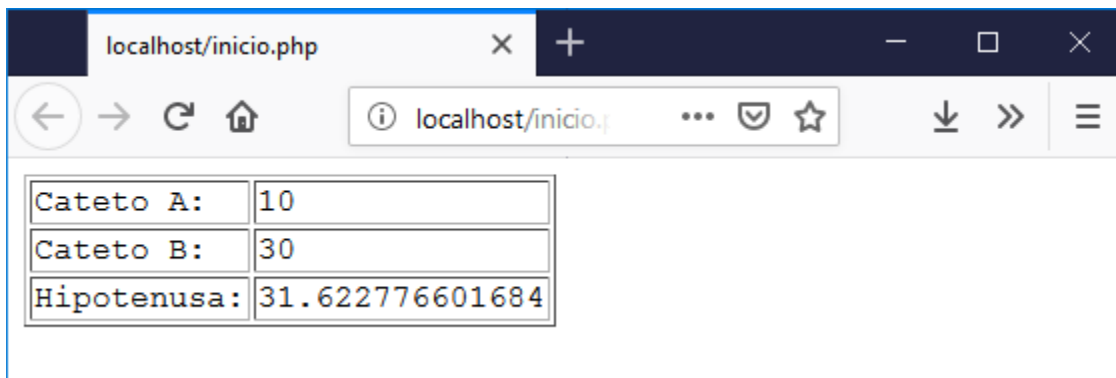
/* Impresión */
echo "<!DOCTYPE HTML><html><body>";
echo "Cateto A: " . $catA . "<br>";
echo "Cateto B: " . $catB . "<br>";
echo "Hipotenusa: " . $Hipotenusa . "<br>";
echo "</body></html>";
```

Un comportamiento mejor



Ilustración 42: Mejora del código en HTML

¿Pero si se requiere algo más elaborado como el uso de tablas?



Cateto A:	10
Cateto B:	30
Hipotenusa:	31.622776601684

Ilustración 43: Mejorando con una tabla

Requiere el siguiente código:

13/Parte1/Ej03.php

```
<?php
/* Hallar la hipotenusa de un triángulo rectángulo */
$catA = 10;
$catB = 30;
$Hipotenusa = sqrt($catA*$catA+$catB*$catB);

/* Impresión */
echo "<!DOCTYPE HTML><html><body>";
echo "<table border='1'>";
echo "<tr><td>Cateto A:</td><td>" . $catA . "</td></tr>";
echo "<tr><td>Cateto B:</td><td>" . $catB . "</td></tr>";
echo "<tr><td>Hipotenusa:</td><td>" . $Hipotenusa . "</td></tr>";
echo "</table></body></html>";
```

Como puede observarse, se está poniendo cada vez más y más código HTML para la presentación dentro de las instrucciones PHP. A medida que los requisitos de presentación aumenten (tipo y tamaño de letra, colores, encabezados, pies de página, JavaScript, etc.), se obtiene un código PHP muy difícil de entender y modificar.

En la actualidad, la parte visual en las aplicaciones Web es una tarea ejercida principalmente por profesionales del diseño web y muy poco por desarrolladores de software, luego es muy importante separar la parte lógica, de la parte visual.

Primero está el código HTML (lo visual) en su archivo separado.

13/Parte2/Partevisual.html

```
<!DOCTYPE HTML><html>
<head><title>Geometr&iacute;a b&aacute;sica</title></head>
<body>
<h1 style="color: #000000; font-family: 'Times New Roman'; font-style:
normal; font-variant-ligatures: normal; font-variant-caps: normal; letter-
spacing: normal; orphans: 2; text-indent: 0px; text-transform: none;
white-space: normal; widows: 2; word-spacing: 0px; -webkit-text-stroke-
width: 0px; text-decoration-style: initial; text-decoration-color:
initial; text-align: center;"><span style="font-family: verdana,
geneva;">Geometr&iacute;a B&aacute;sica</span></h1>
<p style="color: #000000; font-family: 'Times New Roman'; font-size:
medium; font-style: normal; font-variant-ligatures: normal; font-variant-
caps: normal; font-weight: 400; letter-spacing: normal; orphans: 2; text-
align: start; text-indent: 0px; text-transform: none; white-space: normal;
widows: 2; word-spacing: 0px; -webkit-text-stroke-width: 0px; text-
decoration-style: initial; text-decoration-color: initial;"><span
style="font-family: tahoma, arial, helvetica, sans-serif;">C&aacute;lculo
de la hipotenusa usando el Teorema de Pit&aacute;goras</span></p>
<table style="color: #000000; font-family: 'Times New Roman'; font-size:
medium; font-style: normal; font-variant-ligatures: normal; font-variant-
caps: normal; font-weight: 400; letter-spacing: normal; text-align: start;
text-indent: 0px; text-transform: none; white-space: normal; word-spacing:
0px; -webkit-text-stroke-width: 0px; text-decoration-style: initial; text-
decoration-color: initial; border-color: blue; width: 730px; height:
188px;" border="1">
<tbody>
<tr style="background-color: #1e2ae0;">
<td style="width: 226.417px; text-align: center;">
<h3><span style="font-family: tahoma, arial, helvetica, sans-serif; font-
size: x-large; color: #ffff00;">Cateto A</span></h3>
</td>
<td style="width: 226.417px; text-align: center;">
<h3><span style="font-family: tahoma, arial, helvetica, sans-serif; font-
size: x-large; color: #ffff00;">Cateto B</span></h3>
</td>
<td style="width: 299.167px; text-align: center;">
```

```

<h3><span style="font-family: tahoma, arial, helvetica, sans-serif; font-size: x-large; color: #ffffff;">Hipotenusa</span></h3>
</td>
</tr>
<tr>
<td style="width: 226.417px;"><span style="font-family: tahoma, arial, helvetica, sans-serif; font-size: large;">{catetoA}</span></td>
<td style="width: 226.417px;"><span style="font-family: tahoma, arial, helvetica, sans-serif; font-size: large;">{catetoB}</span></td>
<td style="width: 299.167px;"><span style="font-family: tahoma, arial, helvetica, sans-serif; font-size: large;">{hipotenusa}</span></td>
</tr>
</tbody>
</table>
</body></html>

```

Y ahora el código PHP con la lógica

13/Parte2/Inicio.php

```

<?php /* Separar HTML (parte visual) de la parte lógica */
//La lógica
$catA = 10;
$catB = 30;
$Hipotenusa = sqrt($catA*$catA+$catB*$catB);

//Llama la parte visual
$plantilla = file_get_contents('Partevisual.html');

//Actualiza la parte visual con los datos
$resultado = str_replace('{catetoA}', $catA, $plantilla);
$resultado = str_replace('{catetoB}', $catB, $resultado);
$resultado = str_replace('{hipotenusa}', $Hipotenusa, $resultado);

//Muestra el resultado al usuario
echo $resultado;

```

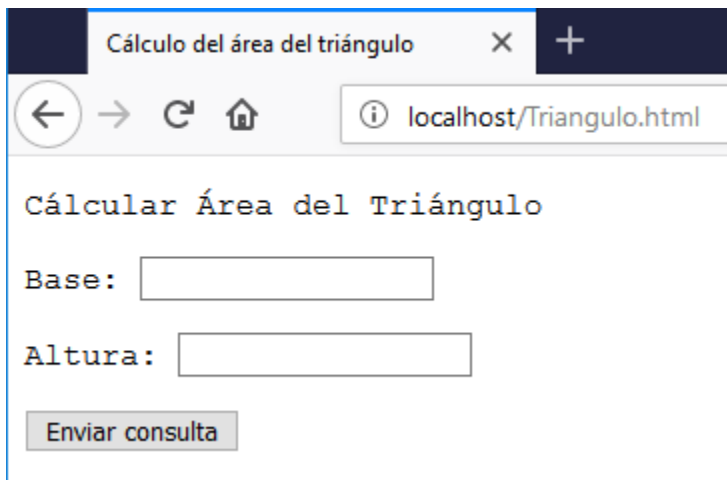
En el archivo HTML hay unas secciones encerradas con llaves {catetoA} {catetoB} {hipotenusa}, el resto es código HTML normal. El diseñador web puede cambiar completamente la página Web respetando sólo lo que hay entre los símbolos de llave { }. En el programa en PHP, lo que se hace es cargar el archivo HTML como si fuese una cadena (string) y se reemplaza lo que está entre las llaves { } con los valores generados por el algoritmo.

Se obtiene el siguiente resultado:

Formularios

Método GET

Para poder enviarle a PHP datos de entrada, se hacen uso de los formularios HTML. Por ejemplo, calcular el área de un triángulo dada la base y la altura. Luego el usuario final en un formulario digita esa base y altura, da clic en el botón de enviar y recibe a continuación el área del triángulo como respuesta.



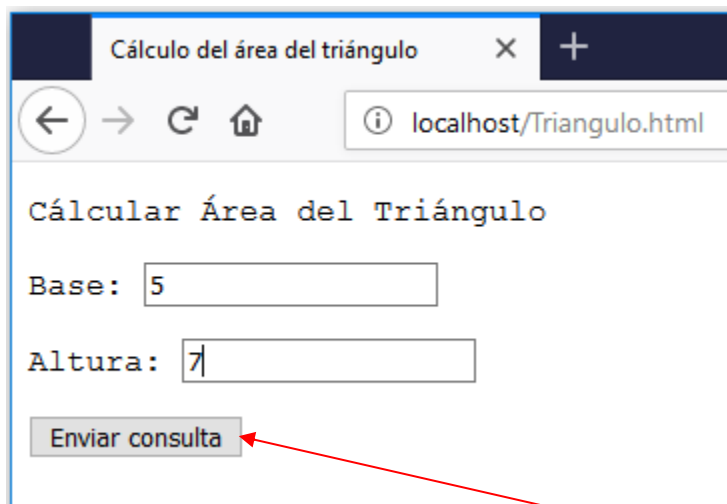
Cálculo del área del triángulo

Base:

Altura:

Enviar consulta

Ilustración 45: Formulario donde el usuario digita los datos de base y altura



Cálculo del área del triángulo

Base:

Altura:

Enviar consulta

Ilustración 46: Se digitan los datos y se presiona el botón "Enviar consulta"

Y esta es la respuesta

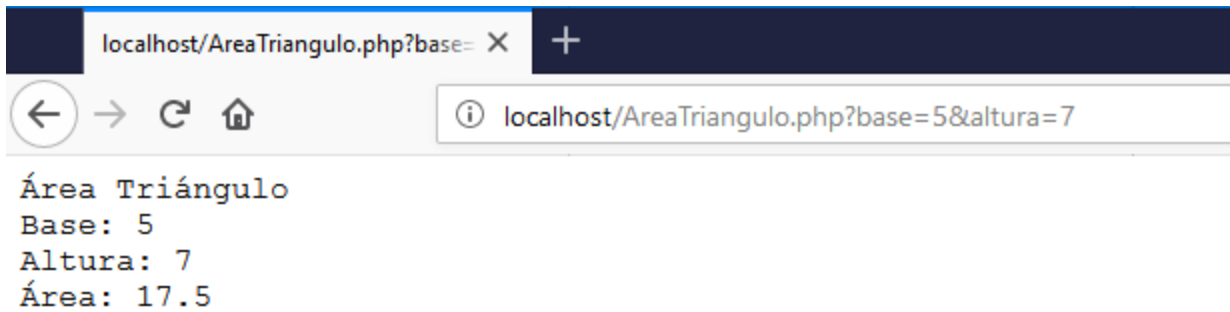


Ilustración 47: Esta es la respuesta del programa

Los datos son enviados por la URL al programa en PHP

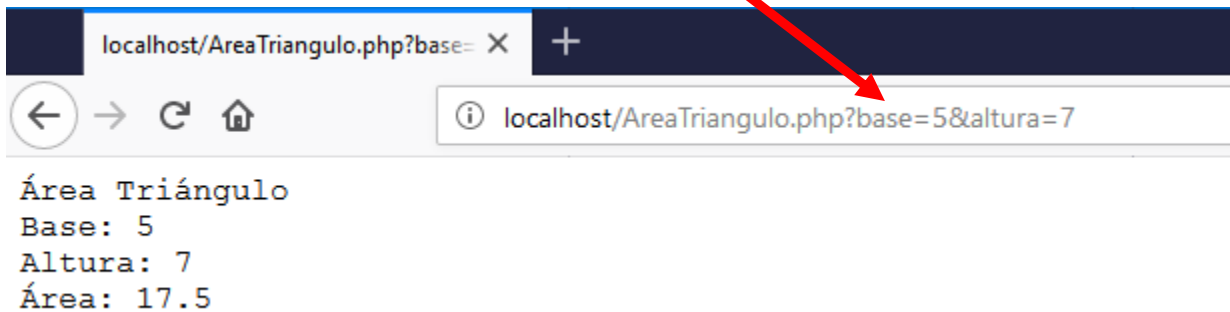


Ilustración 48: Datos enviados por la URL

Se observa que en la URL aparecen los datos digitados en el formulario. Tiene esta forma:

Programa.php?parametroA=valor1¶metroB=valor2¶metroC=valor3

La lista de parámetros inicia con el símbolo “?” y luego sigue el nombre del parámetro, el signo “=” y después el valor, luego sigue el símbolo “&” para separar los parámetros. ¿Cuántos parámetros puede haber? Los que se desee, el límite estaría en el máximo tamaño de la URL soportado por el navegador y el servidor de aplicaciones.

El paso de parámetros por la URL se le conoce como método GET.

Para el ejemplo anterior se usaron dos archivos en el servidor de aplicaciones:

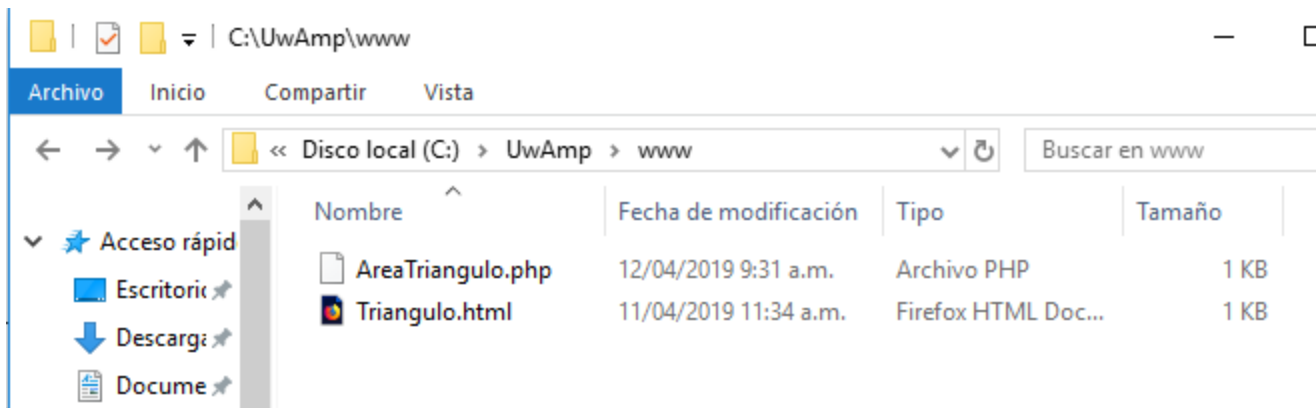


Ilustración 49: Dos archivos en el servidor de aplicaciones

Este es el código del formulario HTML (Triangulo.html)

13/Parte3/Triangulo.html

```
<!DOCTYPE HTML><html>
<head><title>Cálculo del área del triángulo</title></head>
<body>
<p>Calcular Área del Triángulo</p>

<!-- Genera el formulario y llama a AreaTriangulo.php cuando
      se de clic en el botón "Enviar consulta" -->
<form action="AreaTriangulo.php" method="get">
  <p>Base: <input type="text" name="base" /></p>
  <p>Altura: <input type="text" name="altura" /></p>
  <p><input type="submit" /></p>
</form>

</body></html>
```

El formulario: inicia con “<form action=”, luego sigue el nombre del archivo en PHP que llamará cuando se presiona el botón de “Enviar consulta”, cierra esa línea con el método de envío que es “get” (hay otro método de envío llamado POST que se explica más adelante).

Palabras reservadas Programa PHP que se llamará Método de envío de datos

```
<form action="AreaTriangulo.php" method="get">
```

Sigue entonces los campos de entrada del formulario. Usualmente se usa un campo texto (`type="text"`) y el nombre único (`name`) que tendrá cada campo.

```
<p>Base: <input type="text" name="base" /></p>
<p>Altura: <input type="text" name="altura" /></p>
```

Tipo de campo de entrada

Nombre único de campo que será usado luego por PHP

Está el botón de “Enviar consulta” que es `input type="submit"`

```
<p><input type="submit" /></p>
```

El programa en PHP es:

13/Parte3/AreaTriangulo.php

```
<?php
//Lee los parámetros que llegan por la línea URL
$baseTriangulo = $_GET['base'];
$alturaTriangulo = $_GET['altura'];

//Hace el cálculo
$area = $baseTriangulo * $alturaTriangulo / 2;

//Muestra el resultado
echo "Área Triángulo<br>";
echo "Base: " . $baseTriangulo . "<br>";
echo "Altura: " . $alturaTriangulo . "<br>";
echo "Área: " . $area;
```

Para capturar el valor del campo se hace uso de `$_GET['nombre del campo']`

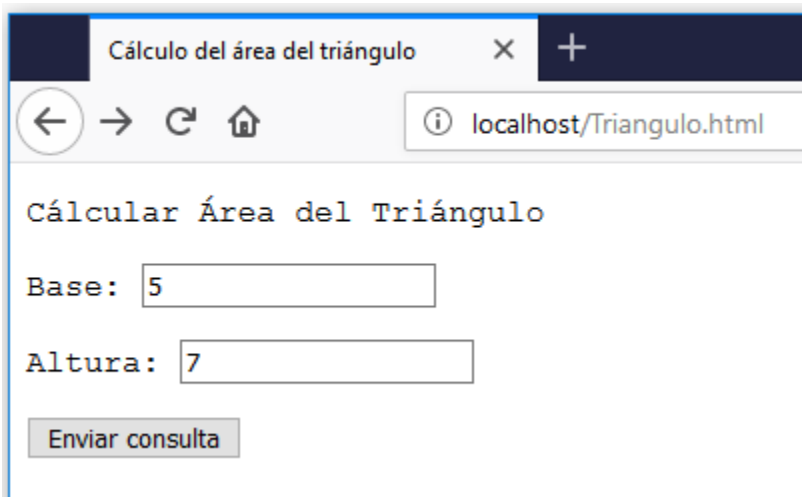
Lee el valor del parámetro enviado por GET, es decir por la URL

```
$baseTriangulo = $_GET['base'];
$alturaTriangulo = $_GET['altura'];
```

Luego debe haber coincidencia entre el “name” del formulario en HTML y lo que se lee con `$_GET`

Método POST

Existe otro método para el envío de parámetros y es el POST, así funciona:



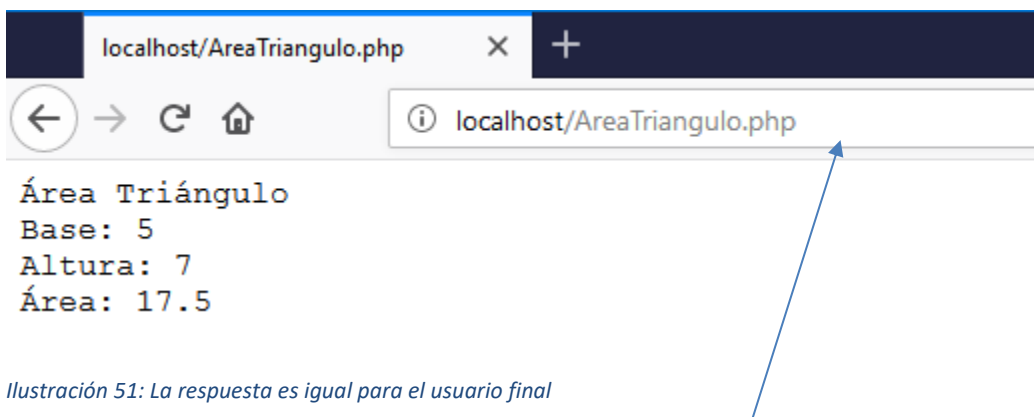
Cálculo del área del triángulo

Cálcular Área del Triángulo

Base:

Altura:

Ilustración 50: El formulario luce igual para el usuario final



localhost/AreaTriangulo.php

Área Triángulo

Base: 5

Altura: 7

Área: 17.5

Ilustración 51: La respuesta es igual para el usuario final

¿Cuál es la diferencia? En la URL de la respuesta: NO se ven los parámetros, porque los parámetros se envían de forma oculta.

El código es el siguiente:

```
<!DOCTYPE HTML><html>
<head><title>Cálculo del área del triángulo</title></head>
<body>
<p>Cálcular Área del Triángulo</p>

<!-- Genera el formulario y llama a AreaTriangulo.php cuando
      se de clic en el botón "Enviar consulta" -->
<form action="AreaTriangulo.php" method="post">
  <p>Base: <input type="text" name="base" /></p>
  <p>Altura: <input type="text" name="altura" /></p>
  <p><input type="submit" /></p>
</form>

</body></html>
```

Uso de "POST"

```
<?php
//Lee los parámetros que llegan por variables ocultas
$baseTriangulo = $_POST['base'];
$alturaTriangulo = $_POST['altura'];

//Hace el cálculo
$area = $baseTriangulo * $alturaTriangulo / 2;

//Muestra el resultado
echo "Área Triángulo<br>";
echo "Base: " . $baseTriangulo . "<br>";
echo "Altura: " . $alturaTriangulo . "<br>";
echo "Área: " . $area;
```

Uso de bases de datos

En 14/Educator.sql está la base de datos con la que se trabajarán los siguientes ejemplos. Es un script que genera una base de datos llamada “educar” y trae varias tablas y registros.

Aparece así en SQLYog Community Edition:

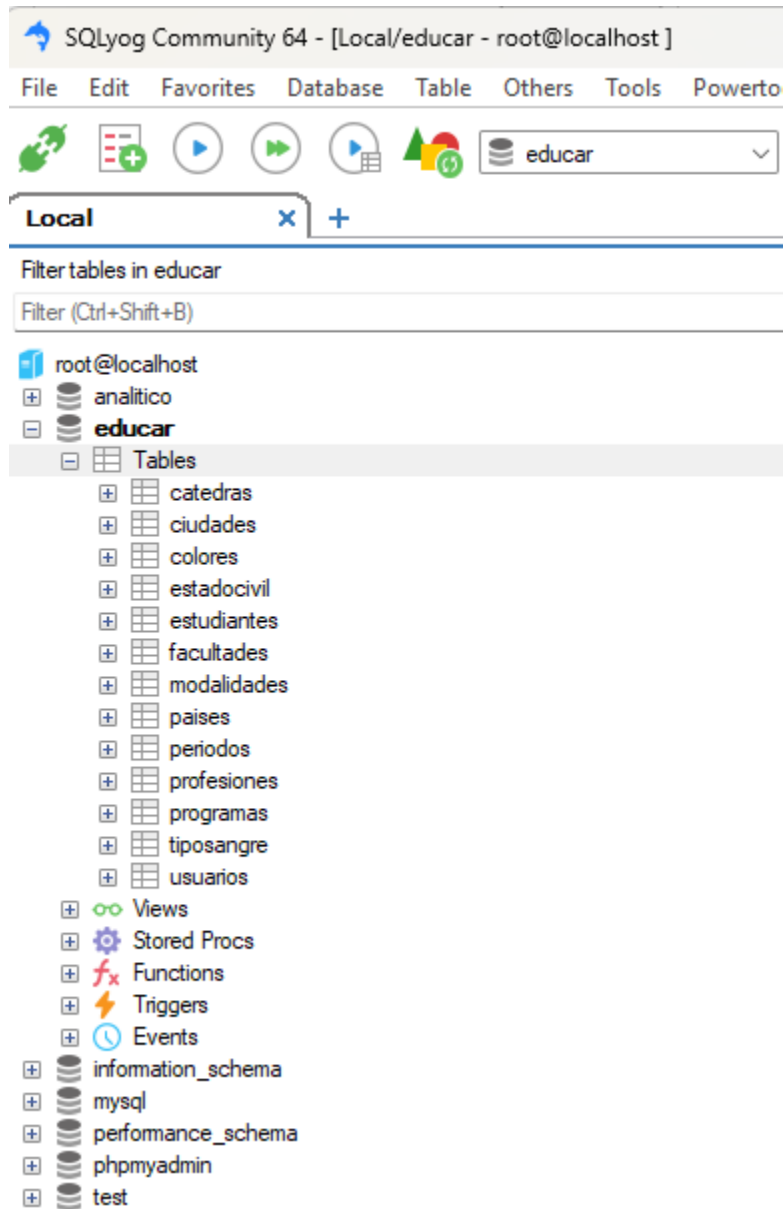


Ilustración 52: Base de datos "educar"

Versiones de elaboración de CRUDs

Versión 1	Conexión a la base de datos. Uso de POO.
Versión 2	Añade paginación
Versión 3	Añade orden al código
Versión 4	Añade página de detalles
Versión 5	Añade página de borrar
Versión 6	Añade página de adicionar
Versión 7	Añade página de editar
Versión 8	Se cambia colores.php a index.php Cuando adiciona y edita, ese registro se muestra al inicio del grid.
Versión 9	Se adiciona Buscar ¡OJO! Es vulnerable a un ataque de inyección SQL, Usar: p%'; drop table estudiantes;
Versión 10	Buscar, quita vulnerabilidad al ataque de inyección SQL, probarlo con: p%'; drop table estudiantes;
Versión 11	CRUD de tabla estudiantes (más compleja)
Versión 12	Trayendo las descripciones de los campos referenciados de la tabla estudiantes
Versión 13	Uso del componente Select2 para adicionar
Versión 14	Componente Trumbowyg para texto enriquecido
Versión 15	Trumbowyg + Edición
Versión 16	Trumbowyg limitado para minimizar ataques de inyección HTML
Versión 17	Subir imágenes
Versión 18	Lista de valores en cascada
Versión 19	Separando el código de persistencia
Versión 20	Menú y Sesiones
Versión 21	Muestra las bases de datos, tablas y campos por tabla

Referencias

- [1] W3Schools, «PHP Tutorial,» 2026. [En línea]. Available: <https://www.w3schools.com/php/default.asp>. [Último acceso: 2026].
- [2] PHP, «PHP,» 2026. [En línea]. Available: <http://php.net/>. [Último acceso: 2026].
- [3] W3Techs, «Usage of server-side programming languages for websites,» 2026. [En línea]. Available: https://w3techs.com/technologies/overview/programming_language/all. [Último acceso: 2026].
- [4] Mozilla, «What is a web server?,» 2026. [En línea]. Available: https://developer.mozilla.org/en-US/docs/Learn/Common_questions/Web_mechanics/What_is_a_web_server. [Último acceso: 2026].
- [5] M. Rouse, «database management system (DBMS),» 2026. [En línea]. Available: <https://searchsqlserver.techtarget.com/definition/database-management-system>. [Último acceso: 2026].
- [6] MariaDB, «MariaDB,» 2026. [En línea]. Available: <https://mariadb.org/>. [Último acceso: 2026].
- [7] MySQL, «MySQL,» 2026. [En línea]. Available: <https://www.mysql.com/>. [Último acceso: 2026].
- [8] W3C, «HTTP - Hypertext Transfer Protocol,» 2026. [En línea]. Available: <https://www.w3.org/Protocols/>. [Último acceso: 2026].
- [9] J. A. Pascual, «¿Qué es localhost o IP 127.0.0.1 y para qué se utiliza?,» 2026. [En línea]. Available: <https://computerhoy.com/noticias/internet/que-es-localhost-ip-127001-que-utiliza-48046>. [Último acceso: 2026].
- [10] M. Matsumoto, «Mersenne Twister Home Page,» 2026. [En línea]. Available: <http://www.math.sci.hiroshima-u.ac.jp/~m-mat/MT/emt.html>. [Último acceso: 2026].